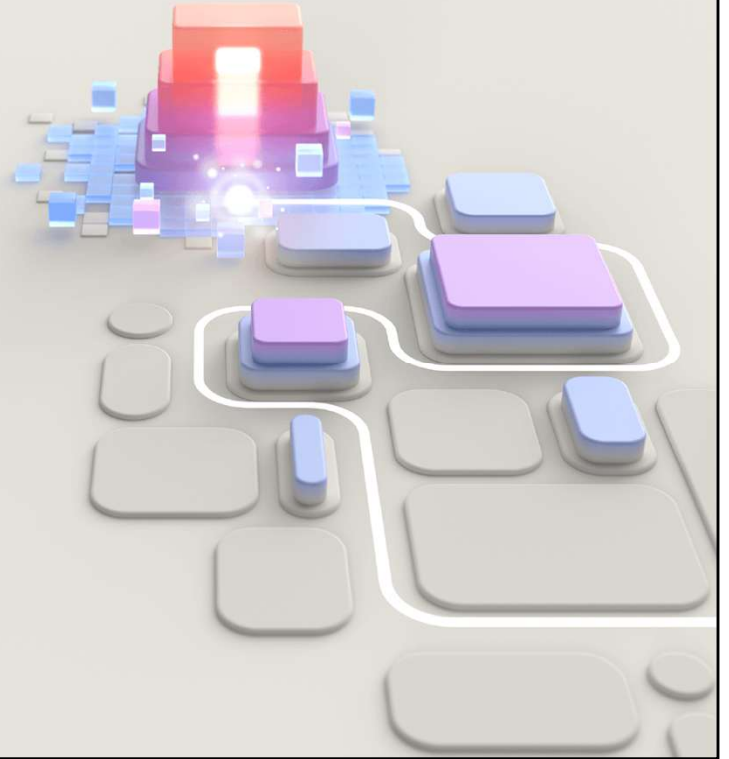




DP-300

Implement scalable database solutions using Azure SQL

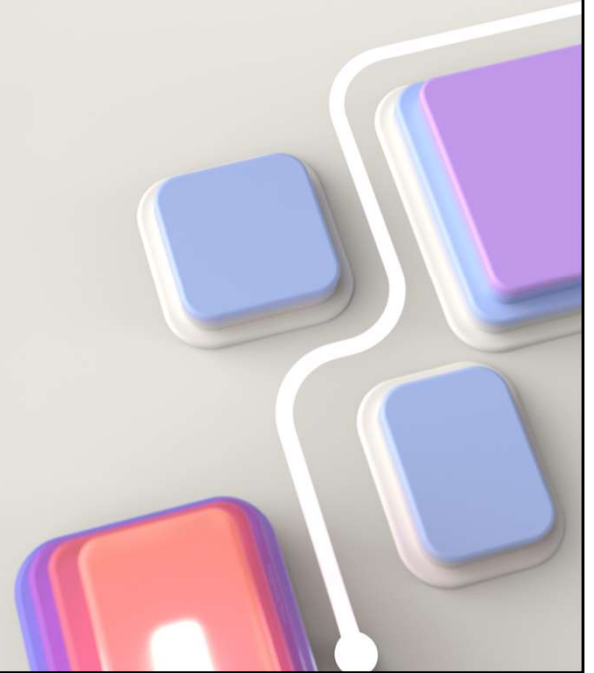


© Copyright Microsoft Corporation. All rights reserved.

Plan and implement data platform resources

<https://learn.microsoft.com/training/paths/plan-implement-data-platform-resources/>

© Copyright Microsoft Corporation. All rights reserved.



This Learning Path introduces methods for deploying data platform resources in Azure. You will learn about options for both upgrading and migrating existing SQL databases to Azure. You will learn how to set up Azure resources to host SQL Server on a Virtual Machine, a Managed Instance, and Azure SQL Database. You will learn how to determine which options are best based on specific requirements including the High Availability and Disaster Recovery (HADR) needs. They will learn to calculate resource requirements and create templates for their deployments.

<https://learn.microsoft.com/training/paths/plan-implement-data-platform-resources/>

Objectives

- Describe the key differences between the SQL Server-based database options in Azure
- Deploy resources using manual methods
- Recommend an appropriate database offering based on requirements
- Configure Azure SQL resources
- Evaluate and implement a strategy for moving a database to Azure SQL

© Copyright Microsoft Corporation. All rights reserved.

This Learning Path introduces methods for deploying data platform resources in Azure. You will learn about options for both upgrading and migrating existing SQL databases to Azure. You will learn how to set up Azure resources to host SQL Server on a Virtual Machine, a Managed Instance, and Azure SQL Database. You will learn how to determine which options are best based on specific requirements including the High Availability and Disaster Recovery (HADR) needs. They will learn to calculate resource requirements and create templates for their deployments.

Prepare to maintain SQL Server-based databases on Azure

<https://learn.microsoft.com/training/modules/prepare-to-maintain-sql-databases-azure/>

© Copyright Microsoft Corporation. All rights reserved.

NOTE: In this section, students will explore the conceptual aspects of SQL Server-based databases on Azure. This serves as a foundational introduction to the upcoming sections.

The range of options for deploying SQL Server on Azure are wide and comprehensive, and challenging for many administrators to navigate though. Many Database Administrators are tasked with not just maintaining and supporting SQL Server, but also providing assistance with making architectural decisions as they relate to the relational database management system (RDBMS) platform.

The decisions vary by the nature of the application, whether the application is developed in-house or by an independent software vendor (ISV). While most of the Azure Data Platform offerings are quite similar in their implementation, there are nuanced differences which require changes in application code which may not be possible if you do not own the application code. This unit will be focused on applications that moved into Azure Virtual Machines running SQL Server.

Objectives



Describe Microsoft Intelligent Data Platform roles



Understand SQL Server in an Azure virtual machine



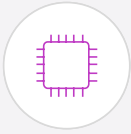
Design Azure SQL Database for cloud-native applications



Explore Azure SQL Managed Instance

© Copyright Microsoft Corporation. All rights reserved.

Azure Database Administrator



Azure Database Administrator:

The Azure Database Administrator implements and manages the operational aspects of cloud-native and hybrid data platform solutions built on Microsoft Azure data services and Microsoft SQL Server

Other Azure data roles:

Azure Data Engineer

Azure Data Scientist

Azure Data Analyst

Azure Artificial Intelligence Engineer

© Copyright Microsoft Corporation. All rights reserved.

Azure Database Administrator

Ansvarar för drift, säkerhet och prestanda i databaser på Azure och SQL Server.

Arbetsuppgifter: backup/restore, övervakning, patchning, åtkomstkontroll, optimering av databasmiljöer.

Azure Data Engineer

Bygger och underhåller pipelines som samlar in, bearbetar och lagrar stora datamängder i molnet.

Arbetsuppgifter: ETL-processer, Azure Data Factory, datalager, integration mellan system.

Azure Data Scientist

Analyserar data och utvecklar maskininlärningsmodeller för att skapa prediktioner och insikter.

Arbetsuppgifter: modellträning, feature engineering, experiment i Azure ML, utvärdering av AI-resultat.

Azure Data Analyst

Fokuserar på att tolka data och skapa rapporter och dashboards för verksamheten.

Arbetsuppgifter: Power BI-rapporter, datavisualisering, KPI-analys, stöd till beslutsfattare.

Azure Artificial Intelligence Engineer

Implementerar AI-lösningar och integrerar kognitiva tjänster i applikationer.

Arbetsuppgifter: bygga chatbots, använda Azure Cognitive Services, deployment av AI-modeller, AI-integration i system.

Azure Data Engineer:

Design and implement the management, monitoring, security, and privacy of data systems

Azure Data Analyst:

Enable businesses to maximize the value of their data assets by using Microsoft Power BI

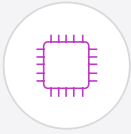
Azure Data Scientist:

Applies their knowledge of data science and machine learning to implement and run machine learning workloads on Azure

Azure Artificial Intelligence Engineer:

Use Cognitive Services, Machine Learning, and Knowledge Mining to build AI solutions

Azure Database Administrator



Azure Database Administrator:

The Azure Database Administrator implements and manages the operational aspects of cloud-native and hybrid data platform solutions built on Microsoft Azure data services and Microsoft SQL Server

Other Azure data roles:

Azure Data Engineer

Azure Data Scientist

Azure Data Analyst

Azure Artificial Intelligence Engineer

© Copyright Microsoft Corporation. All rights reserved.

Azure Data Engineer:

Design and implement the management, monitoring, security, and privacy of data systems

Azure Data Analyst:

Enable businesses to maximize the value of their data assets by using Microsoft Power BI

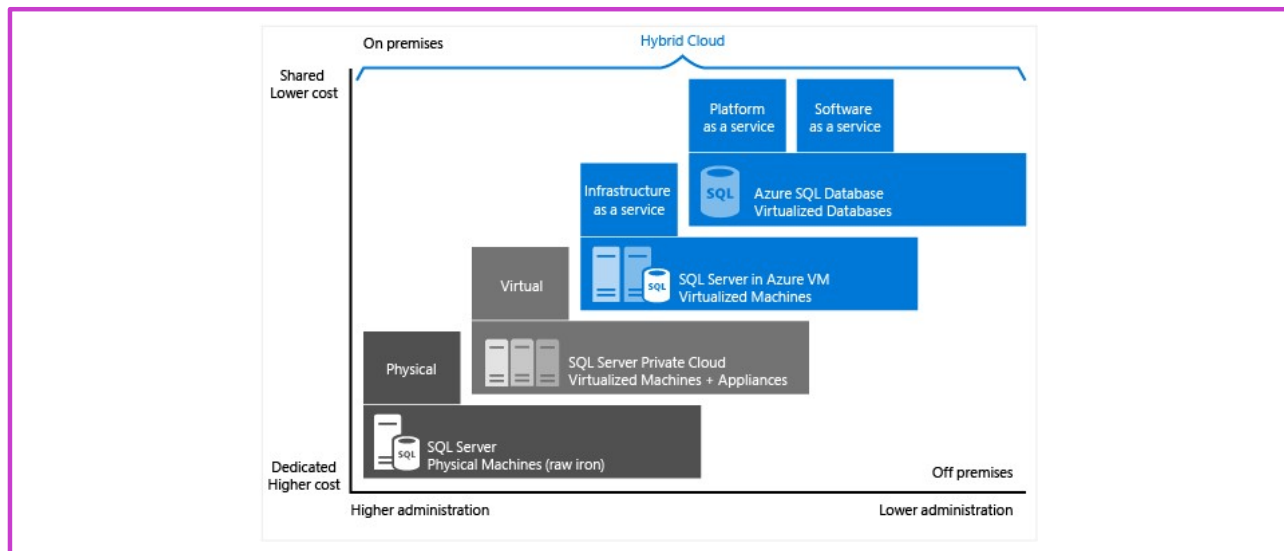
Azure Data Scientist:

Applies their knowledge of data science and machine learning to implement and run machine learning workloads on Azure

Azure Artificial Intelligence Engineer:

Use Cognitive Services, Machine Learning, and Knowledge Mining to build AI solutions

Azure SQL platform



© Copyright Microsoft Corporation. All rights reserved.

Bilden visar Azure SQL-plattformen som en skala från traditionell drift **on-premises** till fullt hanterade molntjänster i Azure. Skillnaden mellan nivåerna handlar främst om hur mycket ansvar kunden har jämfört med Microsoft.

Längst ner finns **SQL Server på fysiska servrar (raw iron)**. Här ansvarar organisationen för allt: hårdvara, operativsystem, patchning, backup och drift. Detta är vanligt i egna datacenter.

Nästa steg är **SQL Server Private Cloud**, där databasen körs i virtualiserade miljöer lokalt, exempelvis via Hyper-V eller Azure Stack. Man får flexibilitet men måste fortfarande administrera nästan allt själv.

I Azure kan man sedan använda **SQL Server i Azure VM (IaaS)**. Här körs SQL Server i en virtuell maskin i molnet. Microsoft hanterar infrastrukturen, men du ansvarar fortfarande för SQL Server-installation, uppdateringar och optimering. Exempel: en lift-and-shift-migrering av en befintlig databas.

Högst upp finns **Azure SQL Database och Managed Instance (PaaS)**. Här tar Microsoft hand om patchning, hög tillgänglighet och backup automatiskt. Du fokuserar på data och applikationer istället för drift.

Slutligen finns även SaaS-liknande lösningar där databaser är en del av större tjänster, till exempel Dataverse eller inbyggda databaser i affärssystem.

Ju högre upp man går, desto mindre administration och ofta lägre driftkostnad, men också mindre kontroll över infrastrukturen.

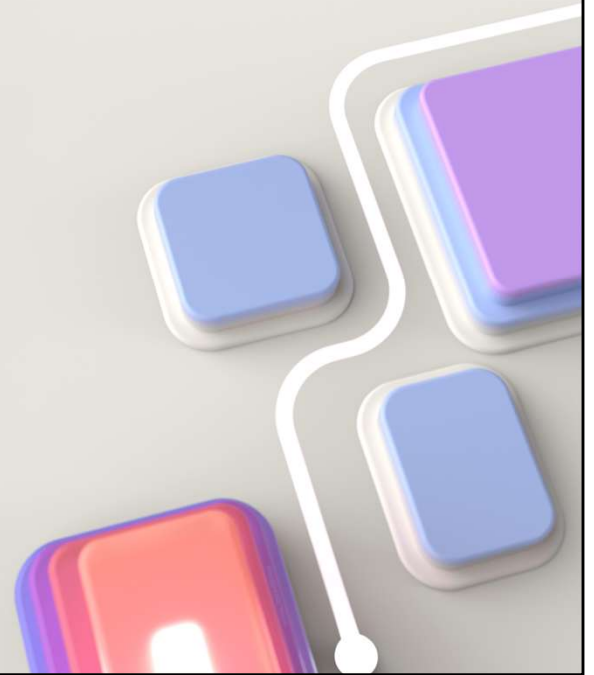
Dataverse är en molnbaserad dataplattform inom Microsoft Power Platform och Dynamics 365. Den fungerar som en färdig databas med inbyggd säkerhet, affärslogik och integration med Power Apps, Power Automate och Power BI. Skillnaden mellan PaaS och SaaS handlar om ansvar. Platform as a Service (PaaS) ger en hanterad plattform där Microsoft sköter servrar, patchning och backup, medan du bygger och administrerar databasen och applikationen själv, exempelvis med Azure SQL Database. Software as a Service (SaaS) är färdiga program som du bara använder, som Microsoft 365, Dynamics 365 och ofta Dataverse.

As seen in the diagram, each service offering can be characterized by the level of administration you have over the infrastructure, and by the degree of cost efficiency.

You can run your SQL Server workloads as either a hosted service (PaaS) or a hosted infrastructure (IaaS). PaaS offers multiple product options and service tiers within each option, while SaaS provides Microsoft Fabric. The key question to consider when choosing between PaaS, IaaS, or SaaS is whether you want to manage your database, apply patches, and take backups yourself, or delegate these operations to Azure.

Understand SQL Server running on Azure Virtual Machine

© Copyright Microsoft Corporation. All rights reserved.



A SQL Server running in an Azure virtual machine (IaaS) is equivalent to an on-premises SQL Server. You will notice that several features described for SQL Server on Azure virtual machine are applicable to all your on-premises SQL Servers.

SQL Server on Azure Virtual Machine

Gives you a wide range of computing power and RAM

Provides flexibility for applications that may have dependency on specific versions of SQL Server

Allows you to run additional SQL Server services like:

- Analysis Services
- Reporting Services
- Machine Learning Services
- Integration Services alongside database engine

© Copyright Microsoft Corporation. All rights reserved.

While platform as a service (PaaS) offerings such as Azure SQL Database and Azure SQL Database Managed Instance offer several benefits, many applications still require a VM running SQL Server. Some reasons for this include:

- **Older Versions of SQL Server** – If an application requires an older version of SQL Server for vendor support, running inside a VM is the best option for them, because it allows for the application to be supported by that vendor. While Microsoft has recommended supporting applications based on database compatibility level, many vendors have yet to adopt these measures.
- **Use of other SQL Server Services** – While Analysis Services and to an extent Integration Services (with Azure Data Factory) are available as PaaS offerings, many customers maximize their licensing by running these services on the same machine as the database engine.
- **General Application Incompatibility** – This is a little bit of a catch-all, but for example, Azure SQL Database does not support cross-database querying, while Managed Instance does. Some applications may require additional services to be co-located with the database instance in a manner that is not compatible with a PaaS offering.

In addition to the above reasons, many organizations find moving into VMs to be an easier migration path for their initial foray into the cloud. The networking and storage configurations are perceived to be easier to implement than a PaaS solution. With this in mind, SQL Server offers several options to maximize your productivity using VMs in Azure

Backup solutions:

Automated Backup

Allows you to schedule regular backups for all databases on a SQL Server VM

Azure Backup

Complete enterprise backup solution that automatically handles your backups across your infrastructure

Back up to URL

Allows you to use standard backup syntax to back up your databases to Azure Blob Storage service

Deployment options:

Azure Resource Manager (ARM)

Azure Resource Manager templates are a declarative deployment that describes the desired structure and state of the resources to be deployed

PowerShell/Azure CLI

Uses an imperative procedural model – To explicitly specify the process to be executed

Azure Bicep

Azure Bicep is a declarative language that allows you to deploy Azure resources

© Copyright Microsoft Corporation. All rights reserved.

Bilden visar två viktiga områden för hantering av SQL Server i Microsoft Azure: **säkerhetskopiering (backup)** och **driftsättning (deployment)** av resurser.

Under rubriken **Backup solutions** presenteras tre olika alternativ för säkerhetskopiering. **Automated Backup** gör det möjligt att schemalägga regelbundna säkerhetskopieringar för alla databaser på en SQL Server Virtual Machine (VM). Detta är en enkel och automatiserad lösning som minskar risken för mänskliga fel.

Azure Backup är en mer omfattande företagslösning som hanterar säkerhetskopiering över hela infrastrukturen. Den erbjuder centraliserad administration och är lämplig för större organisationer med höga krav på återställning och dataskydd. **Azure Backup** är Microsofts molnbaserade lösning för säkerhetskopiering och återställning av data. Tanken är att företag inte själva ska behöva bygga och underhålla avancerade backup-system. Azure Backup kan automatiskt skapa säkerhetskopior av servrar, virtuella datorer, databaser och filer enligt ett schema som administratören bestämmer.

En stor fördel är att säkerhetskopiorna lagras i Azure, vilket innebär att de finns kvar även om den lokala servern går sönder eller utsätts för exempelvis stöld, brand eller ransomware. Om något händer kan data återställas från en tidigare säkerhetskopia.

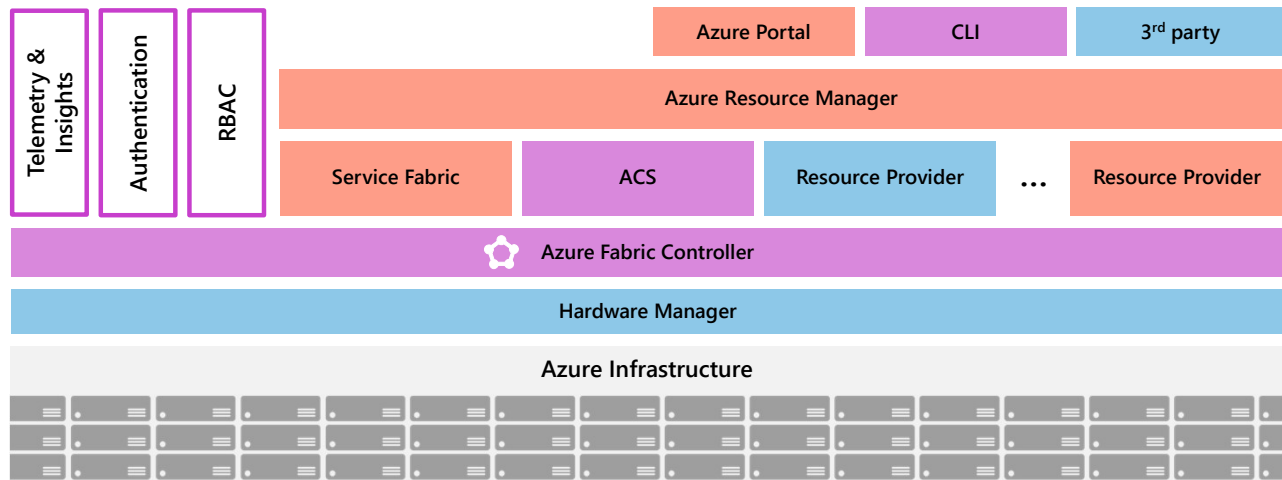
Azure Backup hanterar också lagring, kryptering och säkerhet automatiskt, vilket minskar administrationen. För användaren innebär det att man får ett pålitligt skydd för sina data utan att behöva tänka på den tekniska infrastrukturen bakom. Lösningen passar både små och stora organisationer som vill ha en enkel, säker och skalbar metod för att skydda viktig information.

Det tredje alternativet, **Back up to URL**, använder SQL Servers vanliga backup-kommandon för att säkerhetskopiera databaser direkt till Azure Blob Storage, vilket ger flexibilitet och kontroll över backup-processen.

Den nedre delen av bilden beskriver olika **Deployment options**, alltså metoder för att distribuera och konfigurera Azure-resurser. **Azure Resource Manager (ARM)** använder deklarativa mallar där man beskriver det önskade slutläget för resurserna. **PowerShell/Azure CLI** bygger istället på ett imperativt arbetssätt där varje steg i processen specificeras och körs i ordning. Slutligen visas **Azure Bicep**, ett modernt deklarativt språk som förenklar skapandet och hanteringen av Azure-resurser jämfört med traditionella ARM-mallar.

Sammanfattningsvis illustrerar bilden både hur data kan skyddas genom olika backupstrategier och hur Azure-resurser kan distribueras med olika verktyg och metoder beroende på organisationens behov och kompetens.

Azure Resource Manager templates



© Copyright Microsoft Corporation. All rights reserved.

Bilden visar en förenklad arkitekturöversikt av **Azure Resource Manager (ARM)** och hur resurser i Azure hanteras, från användargränssnitt ner till fysisk infrastruktur.

Längst upp ser du olika sätt att interagera med Azure: **Azure Portal**, **CLI** och **tredjepartsverktyg**. Oavsett vilket verktyg du använder går alla anrop via **Azure Resource Manager**, som är den centrala kontrollpunkten i Azure. ARM ansvarar för att ta emot begäran, validera dem och se till att rätt åtgärder utförs.

Till vänster visas tvärgående funktioner som **Authentication**, **RBAC (Role-Based Access Control)** samt **Telemetry & Insights**. Dessa gäller för alla resurser och säkerställer identitet, behörigheter och övervakning.

Under ARM finns olika interna tjänster, till exempel **Resource Providers** (som hanterar specifika resurstyper som VM, Storage eller SQL), **Service Fabric** och **ACS**. Det är här den faktiska logiken för att skapa, uppdatera och ta bort resurser finns.

Ännu längre ner ligger **Azure Fabric Controller**, som fungerar som Azure's "operativsystem" och koordinerar resurser över datacentren. Under den finns **Hardware Manager** och slutligen den fysiska **Azure Infrastructure** – servrar, nätverk och lagring.

ARM-templates passar in genom att de beskriver *vad* du vill ha (deklarativt), medan hela denna stack ser till att det faktiskt blir gjort – korrekt, säkert och reproducerbart.

All resources in Azure share a common provider known as Azure Resource Manager which acts as a management and deployment service for Azure. While there are numerous ways to deploy Azure resources, ultimately, they all end up going into JSON documents known as Azure Resource Manager (ARM) templates, which is itself is one of the deployment options for Azure Resources. The main difference between the processes is that ARM Templates are explicitly a declarative deployment approach which describe the desired structure and state of the resources to be deployed, whereas the other methods can all be described as imperative which uses procedural models to explicitly specify a process to be executed. In large scale

deployment the declarative approach is better and should be followed

Azure storage



The Azure Storage platform provides several storage services to store files, tables, blobs, queues and VM disks



SQL Server uses two types of Azure storage:

- Managed Disks
- Blob Storage



Heavy workloads typically use Premium SSD or Ultra Disk for database and transaction log files. Use Standard storage for your database backups

© Copyright Microsoft Corporation. All rights reserved.

Bilden ger en översikt av **Azure Storage** och hur plattformen används, särskilt i relation till **SQL Server i Azure**. Azure Storage är en samling lagringstjänster som är byggda för att vara skalbara, hållbara och högtillgängliga. Plattformen kan lagra flera olika typer av data: **filer**, **tabeller**, **blobbar**, **köer** samt **virtuella hårddiskar (VM-diskar)**.

För SQL Server i Azure används i huvudsak två typer av Azure Storage. Den första är **Managed Disks**, som är blocklagring kopplad till virtuella maskiner. Här ligger vanligtvis databaser och transaktionsloggar, alltså den mest prestandakritiska delen av SQL Server. Den andra är **Blob Storage**, som är objektlagring och ofta används för till exempel säkerhetskopior, arkivdata eller export/import av data.

Bilden betonar också vikten av att välja rätt lagringstyp beroende på arbetsbelastning. **Tunga och I/O-intensiva workloads** bör använda **Premium SSD** eller **Ultra Disk**, eftersom dessa erbjuder låg latens och högt antal IOPS – något som är avgörande för databaser och transaktionsloggar. För **backuper** är prestandakraven lägre, och därför rekommenderas **Standard Storage**, som är betydligt billigare men fullt tillräcklig för detta ändamål.

Sammanfattningsvis handlar bilden om att matcha rätt Azure-lagring med rätt användningsområde för att få både bra prestanda och kostnadseffektiv drift.

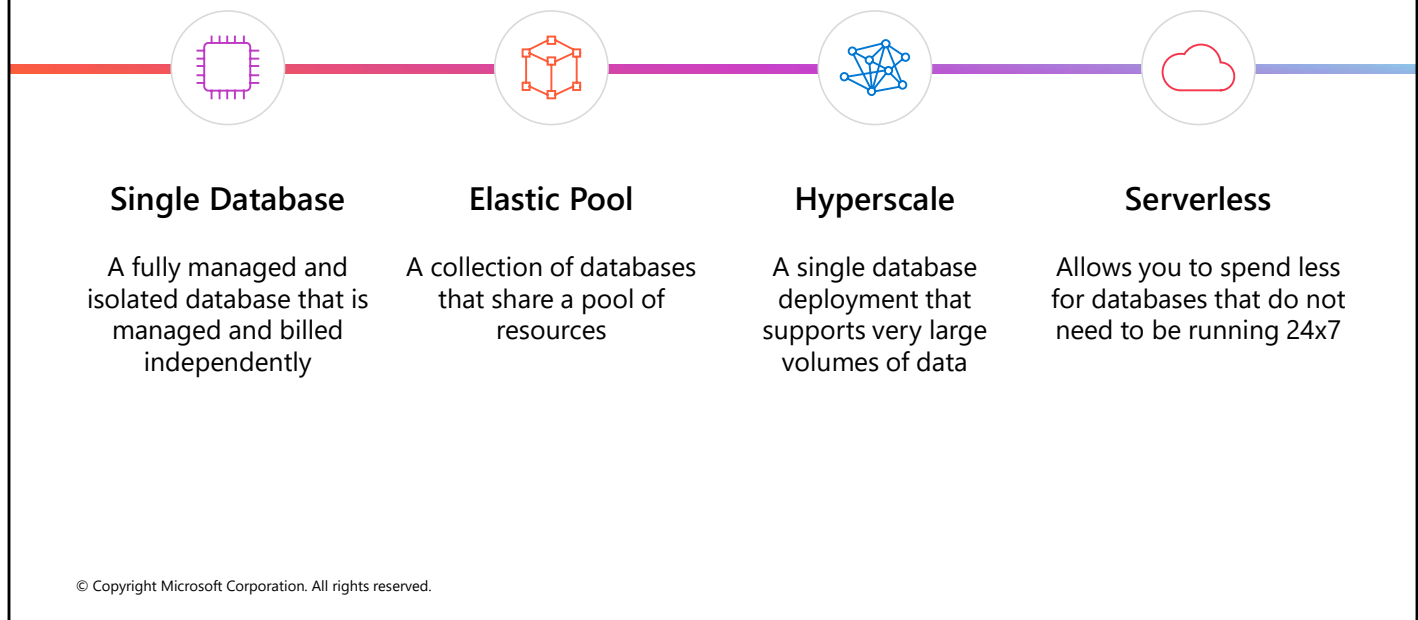
For production SQL Server data and transaction log files, you should only use Premium Storage and Ultra Disk. With premium storage you will see latencies in the range of 5-10 ms on a properly configured system, and with Ultra Disk you may have sub millisecond latency but will likely see 1-2ms workloads in the real world. You can use standard storage is for your database backups. The performance is adequate for most backup and restore workloads.

Design SQL Database for cloud-native applications

© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Database is a Platform as a Service (PaaS) that provides high scalability capabilities, and it can be a great solution for certain workloads, and requires minimal maintenance efforts.

Azure SQL Database deployment models



Bilden visar de **fyra huvudsakliga driftsmodellerna för Azure SQL Database**, som alla är fullt hanterade PaaS-tjänster men anpassade för olika behov vad gäller belastning, skalbarhet och kostnad.

Single Database är den enklaste modellen. Varje databas är helt isolerad och har egna resurser som hanteras och debiteras separat. Den passar bra för de flesta vanliga applikationer, särskilt när belastningen är förutsägbar och databasen inte behöver dela resurser med andra.

Elastic Pool är utformad för scenarier där många databaser har varierande och oförutsägbar belastning. Flera databaser delar på en gemensam pool av resurser (DTU eller vCore), vilket gör det mer kostnadseffektivt än att dimensionera varje databas för maxlast. Det är vanligt i SaaS-lösningar med många kunddatabaser.

Hyperscale är en specialiserad modell för mycket stora databaser och höga I/O-krav. Den använder en distribuerad arkitektur som gör det möjligt att hantera databaser på tiotals eller hundratals terabyte, med snabb skalning och snabba backuper. Den är särskilt lämpad för affärskritiska system med stora datamängder.

Serverless är ett kostnadsoptimerat alternativ där beräkningsresurser automatiskt skalas upp och ner, och till och med pausas vid inaktivitet. Den passar bra för utveckling, test, analys eller applikationer som inte behöver vara aktiva dygnet runt.

Valet av modell handlar i grunden om att balansera prestanda, skalbarhet och kostnad utifrån applikationens beteende.

- **Single Database** – A fully managed and isolated database that is managed and billed independently. A single database has its own dedicated resources
- **Elastic Pool** – A collection of databases that share a pool of resources. These resources are managed collectively and this architecture can reduce your costs and your management overhead
- **Hyperscale** – A single database deployment that supports very large volumes of data (up to 100 TB)
- **Serverless** – Allows you to spend less for databases that do not need to be running 24x7. Best suited for irregular workloads (up to 4 TB)

Single database



Simplest approach to deploying Azure SQL Database



Each database has its own full set of resources



All databases are isolated from each other and are portable



Service level and costs are configured at the individual database level

© Copyright Microsoft Corporation. All rights reserved.

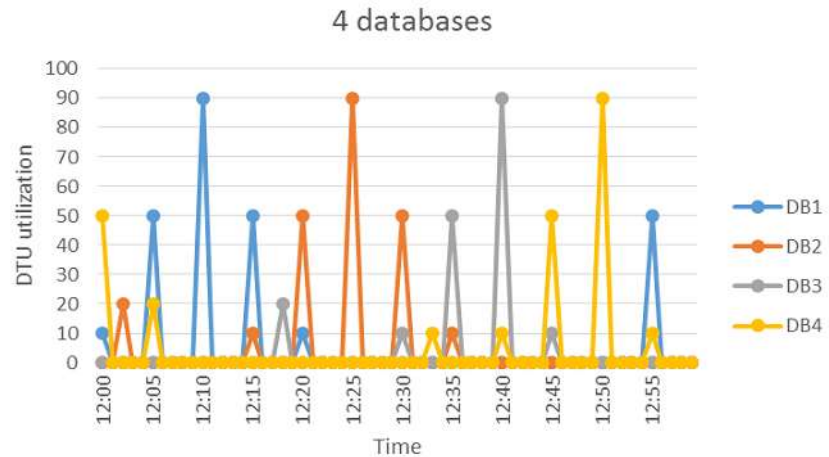
This is simplest and original deployment model of Azure SQL Database. Once your server is deployed, you add a database to it, and can then connect your application to that database. You manage each of your databases individually from scale and data size perspective. Each database deployed in this model (even if to the same logical server) has its own dedicated resources.

Elastic pools

Designed for multi-tenant applications

Pay one price for multiple databases that can be managed together

Rescaling an elastic pool has minimal downtime



Best suited for databases that have similar performance requirements and non-concurrent spikes in utilization

© Copyright Microsoft Corporation. All rights reserved.

Den här bilden förklarar hur **Elastic Pools** fungerar i **Azure SQL Database** och varför de är särskilt lämpade för **multi-tenant-lösningar**. Grundidén är att flera databaser delar på en gemensam uppsättning resurser, i stället för att varje databas måste dimensioneras för sin egen maxbelastning.

Till vänster betonas tre viktiga egenskaper. Elastic pools är **designade för multi-tenant-applikationer**, till exempel SaaS-lösningar där varje kund har sin egen databas. Man **betalar ett pris för en pool** av resurser som flera databaser kan använda gemensamt, vilket ofta ger betydligt bättre kostnadseffektivitet. Dessutom kan poolen **skalas upp eller ner med minimal nedtid**, vilket gör den flexibel när belastningen förändras.

Diagrammet till höger visar fyra databaser (DB1–DB4) och deras **DTU-utnyttjande över tid**. Varje databas har korta belastningstoppar, men topparna inträffar vid olika tidpunkter. Det är precis detta scenario som elastic pools är optimerade för: databaser med **liknande prestandakrav men icke-samtidiga toppar**. När en databas är inaktiv kan en annan tillfälligt använda mer resurser ur poolen.

Sammanfattningsvis passar elastic pools bäst när du har många databaser med ojämn och oförutsägbar belastning. I stället för att överdimensionera varje databas individuellt får du ett mer effektivt resursutnyttjande, enklare hantering och lägre kostnader – utan att offra prestanda där den faktiskt behövs.

NOTE: Explain resource utilization and implications when multiple databases share the same elastic pool versus single database

Elastic Pools

Elastic pools are the Azure SQL Database equivalent of putting multiple databases being stored in the same SQL Server instance. From a resource perspective, you deploy to an elastic pool, and resources are shared between multiple databases in the pool. This can dramatically lower the cost for a software as a service application model, since resources are shared between databases in the pool. **Single databases can be added or removed from the elastic pool with only brief downtime at the end of the operation.** Additionally, you can **rescale a**

pool itself with minimal downtime. Elastic pools are best suited for workloads that have a low average utilization in with non-concurrent spikes in workload.

Scaling Elastic Pools

- Resources in an elastic pool can be shared across all the databases in the pool
- You can assign storage and compute resources to each of the databases in the pool or set default values across the pool
- Scaling up and down will incur a small amount of downtime, and a brief interruption in the connections
- Available in DTU-based purchasing model or the vCore-based purchasing model

Hyperscale



Supports up to 100 TB of database size



Nearly instantaneous backups using snapshot technologies



Fast database restores



Higher overall throughput because of distributed log writes



Horizontal scaling model

© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver **Hyperscale**, som är den mest skalbara distributionsmodellen för **Azure SQL Database**. Den är framtagen för scenarier där traditionella databaser når sina gränser, både när det gäller storlek, prestanda och återställningstider.

En av de mest utmärkande egenskaperna är att Hyperscale **stödjer databaser upp till 100 TB**. Detta möjliggörs genom en distribuerad arkitektur där data inte längre är bundet till en enda lagringsenhet. I stället delas databasen upp över flera lagringsnoder som kan växa oberoende av beräkningsresurserna.

Hyperscale använder **snapshot-baserade backuper**, vilket gör att säkerhetskopiering sker nästan omedelbart – oavsett databasens storlek. Detta leder i sin tur till **mycket snabba återställningar**, eftersom Azure i praktiken bara pekar på existerande snapshots i stället för att kopiera stora datamängder.

En annan viktig fördel är **högre genomströmning** tack vare **distribuerade loggskrivningar**. Transaktionsloggen hanteras parallellt, vilket minskar flaskhalsar som annars kan uppstå vid hög belastning.

Slutligen bygger Hyperscale på en **horisontell skalningsmodell**. Det innebär att du kan skala ut systemet genom att lägga till resurser, snarare än att skala upp en enskild server. Detta gör Hyperscale särskilt lämpligt för mycket stora, affärskritiska databaser med höga krav på tillgänglighet, skalbarhet och snabb återhämtning vid incidenter.

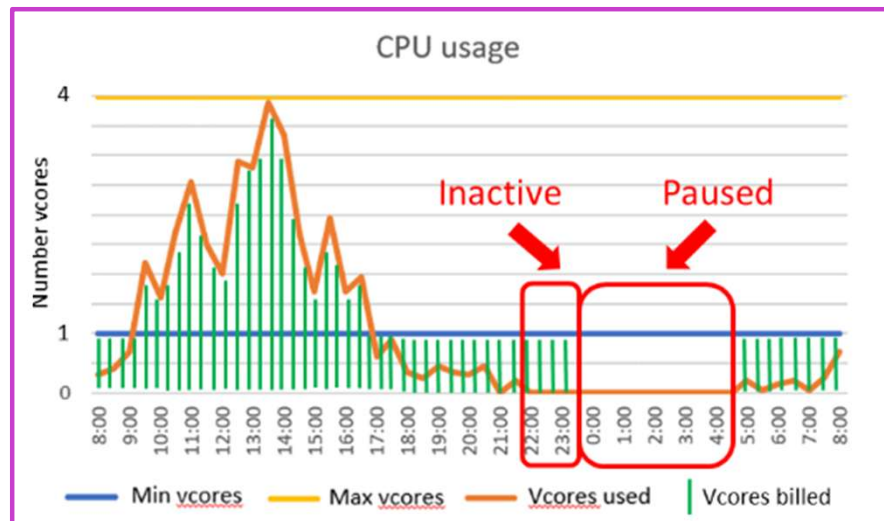
Horizontal scaling is a technique that uses advanced model to add compute nodes as the data sizes grow.

Databases originally created in Hyperscale tier cannot be changed to other tiers.

Azure SQL Database has been limited to 4 TB of storage per database for many years. This is due to a physical limitation of the Azure infrastructure. Azure SQL Database Hyperscale changes the paradigm and allows for databases to be 100 TB and beyond. Hyperscale introduces new horizontal scaling techniques that uses advanced techniques to add compute nodes as the data sizes grow. The cost of Hyperscale is the same as the cost of Azure SQL Database, however there is a per terabyte cost for storage. You should note that that once an Azure SQL Database is converted to Hyperscale, you cannot convert it back to a "regular" Azure SQL Database.

Serverless

- Allows you to spend less for databases that do not need to be running 24x7
- Best suited for irregular workloads
- Only available in vCore model



© Copyright Microsoft Corporation. All rights reserved.

Serverless är en **compute-tier för Azure SQL Database** som är utformad för att minimera kostnader för databaser som inte behöver vara aktiva dygnet runt. Den passar särskilt bra för **ojämna eller oförutsägbara arbetsbelastningar**, till exempel utvecklings- och testmiljöer, interna applikationer eller analyslösningar med periodisk användning. Serverless finns **endast i vCore-modellen**.

I serverless konfigurerar du ett **spann av beräkningskapacitet**, bestående av **minimum vCores** och **maximum vCores**. Databasen skalar automatiskt upp och ner inom detta intervall beroende på aktuell belastning.

Minne och I/O-gränser är proportionella mot det vCore-intervall som valts. När belastningen är låg kan databasen köra på miniminivå, och vid högre aktivitet skala upp mot maxnivån.

En central funktion är **autopause**. Med hjälp av en **autopause delay** (angiven i dagar, timmar eller minuter) definierar man hur länge databasen måste vara helt inaktiv innan den automatiskt pausas. När databasen är pausad används ingen beräkningskapacitet alls. Nästa inloggning eller aktivitet gör att databasen **automatiskt startas igen**. Applikationer bör därför implementera **retry-logik**, eftersom första anropet kan ta några sekunder längre. Autopause kan även **stängas av** om man vill behålla kontinuerlig tillgänglighet.

Kostnadsmodellen är en kombination av **compute-kostnad och lagringskostnad**. När databasen körs mellan min- och maxgränserna debiteras kostnaden baserat på faktiskt använda vCores och minne. Om användningen ligger under min-nivån debiteras ändå

kostnad för min vCores. När databasen är pausad är **compute-kostnaden noll**, och endast lagring debiteras. Lagringskostnaden fungerar på samma sätt som i provisioned-läget.

Det finns dock begränsningar. Serverless stöder **inte geo-replikering, long-term backup retention** eller användning som **job database i elastic jobs**. Dessa faktorer bör vägas in vid arkitekturval.

Sammanfattningsvis är serverless ett mycket kostnadseffektivt alternativ – rätt använt – men inte en universallösning för alla scenarier.

Autopause delay

Time in days/hours/minutes that the database must be idle in order to be paused – applications should implement retry logic

The **minimum vCores** and **maximum vCores** are configurable parameters that define the range of compute capacity available for the database. Memory and IO limits are proportional to the vCore range specified.

The **autopause delay** is a configurable parameter that defines the period of time the database must be inactive before it is automatically paused. The database is automatically resumed when the next login or other activity occurs. Alternatively, autopause can be disabled.

Serverless incompatibility:

- Geo-replication
- Long-term backup retention
- A job database in elastic jobs

Cost

The cost for a serverless database is the summation of the compute cost and storage cost.

When compute usage is between the min and max limits configured, the compute cost is based on vCore and memory used.

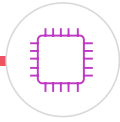
When compute usage is below the min limits configured, the compute cost is based on the min vCores and min memory configured.

When the database is paused, the compute cost is zero and only storage costs are incurred.

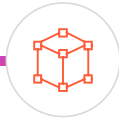
The storage cost is determined in the same way as in the provisioned compute tier.

For more cost details, see [Billing](#).

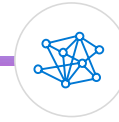
Azure SQL Database backups and restores



Automatically by Azure



**Stored in geo-redundant
storage accounts**



**Backup retention is 7 days by
default, up to 35 days**

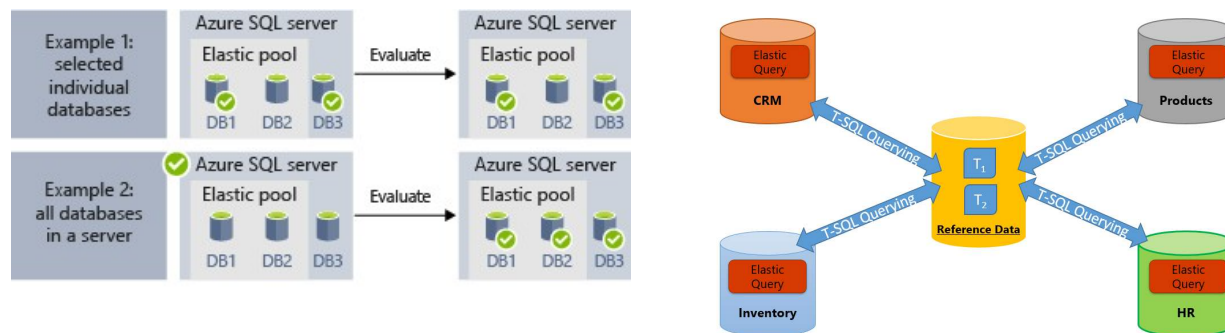
© Copyright Microsoft Corporation. All rights reserved.

You can also set a long-term backup retention (LTR) policy to retain backups for up to 10 years.

Point-in-time restore (PITR) is only supported if you are restoring a database in the same server the backup was originated.

Geo-restore allows you to recover from a geographic disaster when you cannot access your database or backups in the primary region

Elastic features



Elastic Jobs

Executed against one or many Azure SQL databases to run Transact-SQL (T-SQL) queries

Elastic query

A single database deployment that supports very large volumes of data

© Copyright Microsoft Corporation. All rights reserved.

Bilden visar två centrala funktioner i Azure SQL Database som är särskilt användbara i **skalbara och distribuerade databaslösningar**: **Elastic Jobs** och **Elastic Query**. Båda är tänkta att förenkla drift, underhåll och åtkomst när du arbetar med många databaser samtidigt.

Elastic Jobs används för att köra **Transact-SQL-skript mot en eller flera Azure SQL-databaser**, ofta schemalagt och återkommande. Typiska användningsområden är underhållsjobb, indexhantering, statistikuppdateringar, schemaändringar eller datarensning. Ett elastic job körs mot en **target group**, som kan bestå av en hel SQL-server, enskilda databaser, elastic pools eller databaser definierade via shard maps.

Databaserna kan ligga i **olika Azure-subscriptioner och regioner**, och exekveringen sker **parallellt**, vilket ger god skalbarhet.

Elastic Jobs kräver en **dedikerad SQL-databas** som lagrar metadata om jobben, till exempel definitioner, scheman och körhistorik. Varje jobb **loggar status**, och vid fel sker **automatiska retries**, vilket gör lösningen robust. Det är viktigt att notera att **Azure SQL Managed Instance inte stöder Elastic Jobs**.

Elastic Query möjliggör **T-SQL-frågor över flera databaser**, som om de vore en och samma. Det är särskilt användbart för applikationer som använder **tre- och fyrdelade namn** och där koden inte enkelt kan ändras. Därmed ökar även **portabiliteten vid migrering** till Azure SQL Database.

Elastic Query stöder två partitioneringsstrategier. **Vertikal partitionering** innebär att olika typer av data ligger i olika databaser, till exempel kunddata i en databas och betalningsinformation i en annan, med olika scheman. Med elastic query kan dessa kombineras i samma fråga. **Horisontell partitionering (sharding)** innebär att samma schema används i flera databaser, men att raderna fördelas mellan dem, till exempel per kund eller tenant. Detta stödjer både single-tenant- och multi-tenant-modeller.

Tillsammans ger Elastic Jobs och Elastic Query kraftfulla verktyg för att hantera och fråga stora mängder databaser på ett kontrollerat och skalbart sätt.

För en **Elastic Query** krävs i princip:

En databas som fungerar som startpunkt

Det är den databas där du kör frågan.

En eller flera externa databaser

Det är databaserna som innehåller data du vill läsa från.

External data source

Du måste definiera var den externa databasen finns, till exempel vilken Azure SQL Server den ligger på.

External table

I startdatabasen skapar du externa tabeller som pekar på tabeller i de andra databaserna.

Behörighet och inloggning

Startdatabasen måste kunna autentisera sig mot de andra databaserna.

Mycket enkelt: Elastic Query gör att du kan skriva en T-SQL-fråga i en databas, men hämta data från andra Azure SQL-databaser som om tabellerna fanns lokalt.

Azure SQL Databases motsvarighet till Linked Servers, men byggd på External Tables och optimerad för molnmiljön.

Elastic Jobs (more scenarios: <https://learn.microsoft.com/azure/azure-sql/database/elastic-jobs-overview>)

You can create and schedule elastic jobs that could be periodically executed against one or many Azure SQL databases to run Transact-SQL (T-SQL) queries and perform maintenance tasks.

- You can define target database or groups of databases where the job will be executed, and also define schedules for running a job. A job handles the task of logging in to the target database.
- You also define, maintain, and persist Transact-SQL scripts to be executed across a group of databases.
- Require a dedicated SQL database to hold the metadata for your jobs.
- Define a target group, a SQL Database server, one or more single databases or elastic pools as job targets or SQL Databases in shard maps.
- Database resources can run on different Azure subscriptions, and/or regions. The execution runs in parallel.
- Azure SQL Managed Instance doesn't support elastic jobs.

Every job logs the status of execution and also automatically retries the operations if any failure occurs.

Elastic Query

Useful for applications using three- and four-part names that cannot be changed Increases portability for migration

Elastic query allows you to run T-SQL queries that bridge multiple databases in SQL Database. This feature is particularly useful for applications using three- and four-part names that cannot be changed. It also increases portability as it allows for migration.

Elastic queries support the following partitioning strategies:

- Vertical partitioning - **Also called cross-database queries**. The data is partitioned vertically between many databases. Schemas are different for each database. For example, you may have a database used for customer data, and a different one used for payment information. With the help of vertical partitioning, you can now run a cross-database query between both databases.
- Horizontal Partitioning - **Also called sharding**. The data is partitioned horizontally to distribute rows across several scaled-out databases. In this topology, the schema remains the same among all sharding databases. It supports either single-tenant model or multiple tenant models.

Azure SQL Managed Instance

© Copyright Microsoft Corporation. All rights reserved.

While a lot of customers initially migrate to Azure using IaaS offerings, the platform as a service (PaaS) service offering allows for a number of additional benefits. One key benefit is that you no longer have to have to install or patch SQL Server as that is performed by the service. Additionally, consistency checking, and backups are also part of the managed service, and there are additional security and performance tools that are included in the PaaS offerings.

Azure SQL Managed Instance

Managed Instance is a PaaS offering that offers 99% of the functionality of SQL Server

Includes SQL Server Agent, Service Broker, and Common Language Runtime options

Allows for cross database queries

The Azure platform manages backups, patching, and high availability, embedded auditing, security and performance tools

© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Managed Instance är en **PaaS-tjänst** som är utformad för att ligga mycket nära en traditionell **SQL Server-installation**, men utan att du själv behöver hantera infrastrukturen. Tjänsten erbjuder omkring **99 % av funktionaliteten i SQL Server**, vilket gör den särskilt attraktiv vid migrering av befintliga applikationer från on-premises eller virtuella maskiner till Azure.

Till skillnad från Azure SQL Database (Single Database / Elastic Pool) innehåller Managed Instance flera klassiska SQL Server-komponenter. Den stöder bland annat **SQL Server Agent** för schemalagda jobb, **Service Broker** för meddelandebaserad kommunikation samt **Common Language Runtime (CLR)** för körning av .NET-kod i databasen. Dessutom tillåts **cross-database queries**, vilket innebär att T-SQL-frågor kan köras över flera databaser inom samma instans – ett vanligt krav i äldre system.

En stor fördel är att **Azure-plattformen sköter driften**. Microsoft ansvarar för **automatiska säkerhetskopior, patchning, hög tillgänglighet**, samt inbyggda funktioner för **säkerhet, auditing och prestandaövervakning**. Detta minskar det administrativa arbetet avsevärt jämfört med att köra SQL Server på virtuella maskiner.

Managed Instance körs i ett **virtuellt nätverk (VNet)**, vilket ger hög nätverksisolering och gör det enklare att integrera med befintliga system och säkerhetslösningar. Sammantaget är Azure SQL Managed Instance ett bra val när man vill ha maximal SQL Server-kompatibilitet kombinerat med fördelarna av en fullt hanterad molntjänst.

- Easy migration for existing applications through restores from on-premises backups
- Provides access to the system databases
- Integrated with Microsoft Entra ID
- A nice balance between a singleton SQL Database and SQL Server on a Virtual Machine
- Most of the features available on Azure SQL Database will also work for Azure SQL Managed Instance as they share the same base code.

Azure SQL Managed Instance allows for easy migration paths for existing applications by allowing restores from on-premises backups. Unlike Azure SQL Database, which is designed around single data- base structures,

Managed Instance provides an entire SQL Server instance, allowing up to 100 databases, as well as providing access to the system databases. Managed Instance provides other features that are not available in Azure SQL Database, including cross-database queries, common language runtime (CLR) and along with the msdb system database, it allows the use of SQL Agent.

<https://learn.microsoft.com/azure/azure-sql/database/features-comparison?view=azuresql>

Azure SQL Managed Instance service offerings

General purpose

- Uses Azure Premium storage
- Supports up to 16 TB of data
- Designed for applications with typical performance and I/O latency requirements

Business critical

- Supports readable secondary replicas of your database
- Direct attached storage (lower latency)
- Supports up to 4 TB of data

Azure SQL Database and Managed Instance have similar high availability architectures, which guarantee 99.99% percent uptime

© Copyright Microsoft Corporation. All rights reserved.

Bilden visar de två **service tiers** som finns för **Azure SQL Managed Instance: General Purpose** och **Business Critical**.

General Purpose använder Azure Premium Storage och stödjer databaser upp till **16 TB**. Den är anpassad för applikationer med normala prestanda- och I/O-krav och erbjuder en kostnadseffektiv balans mellan pris och kapacitet.

Business Critical är avsedd för affärskritiska system med höga krav på prestanda och tillgänglighet. Den använder **direktanslutet lagring**, vilket ger lägre latens, samt stöd för **läsbara sekundära repliker**. Den maximala databasstorleken är **4 TB**.

Båda nivåerna bygger på en **hög tillgänglighetsarkitektur** som garanterar **99,99 % upptid**.

Business critical

- In-memory OLTP
- More memory per core

[Resource limits - Azure SQL Managed Instance | Microsoft Docs](https://learn.microsoft.com/azure/azure-sql/managed-instance/resource-limits) <https://learn.microsoft.com/azure/azure-sql/managed-instance/resource-limits>

Managed Instance comes in two tiers of service – General Purpose and Business Critical. Both service tiers support the same feature set, and the main differences between the two tiers relate to performance and availability. The only feature differences between the two tiers are that Business Critical supports In-Memory OLTP and readable secondary replicas. Business critical also includes more memory per core, and uses direct attached storage (as opposed to networked)

[What is Azure SQL Managed Instance? - Azure SQL Managed Instance | Microsoft Docs:](https://learn.microsoft.com/azure/azure-sql/managed-instance/sql-managed-instance-paas-overview) <https://learn.microsoft.com/azure/azure-sql/managed-instance/sql-managed-instance-paas-overview>

High availability architecture in Azure PaaS

- Both SQL Database and SQL Managed Instance have similar high availability architectures which guarantees 99.99% uptime
- **General Purpose:** Relies on Azure Storage for high availability
- **Business Critical:** Uses an architecture similar to Always On Availability Groups with multiple copies of the service and database. Also, allows for readable secondary functionality
- High Availability depends on your service tier - is automatic and built-in

Azure SQL Database and Managed Instance have similar high availability architectures, which guarantee 99.99% percent uptime. Windows and SQL Server updates are handled by the backend infrastructure, generally without any impact to your application, though it is important to place retry logic into your application. The high availability solution is automatic and built-in to the platform, and is designed so that committed data is never lost to failure, and your database(s) have no single point of failure. Additionally, Azure SQL Database offers the option to deploy across Availability Zone deployments.

The availability model for general purpose is implemented by separating compute and storage. Azure Storage has built-in high availability and keeps three committed copies of your data (a write acknowledgment is not sent back to the operating system until the write is completed to all three copies). In practice this design is somewhat analogous to an Failover Cluster Instance, as if there was a host failure, the compute tier would come back online in another node, using the same storage the service was previously using.

The business critical tier uses an availability model that is based on a cluster of database engine processes, with multiple copies of the database. The model is built with the intent that a quorum of available engine nodes is always maintained. This also allows for the secondary replicas to be readable, since it shares technology with Always On Availability Groups. There are four replicas in the business critical design.

To achieve comprehensive business continuity on Azure, build your application architecture using the combination of Availability Zones with Azure region pairs. You can synchronously replicate your applications and data using Availability Zones within an Azure region for high-availability and asynchronously replicate across Azure regions for disaster recovery protection.

Hybrid licensing for PaaS services

Both Azure SQL Database and Managed Instance support the hybrid licensing benefit:

Enterprise Edition

Each core entitles you to 8 cores of General Purpose, or 1 core of Business Critical

Standard Edition

Each core of entitles you to 1 core of General Purpose

Network connectivity architecture

Deployed within its own subnet in a virtual network

The service has a publicly accessible name, but it's primarily accessible over a private IP address

Customers have to opt-in to having an open public IP endpoint

© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver två viktiga aspekter av **Azure SQL Database** och **Azure SQL Managed Instance: hybrid licensing** och **nätverksanslutning**.

M

ed **Hybrid licensing** (Azure Hybrid Benefit) kan organisationer återanvända sina befintliga **SQL Server-licenser med Software Assurance** för att minska kostnaden i Azure. Både Azure SQL Database och Managed Instance stöder detta. För **Enterprise Edition** gäller att **en licensierad kärna** ger rätt till antingen **8 vCores i General Purpose** eller **1 vCore i Business Critical**. För **Standard Edition** ger **en licensierad kärna** rätt till **1 vCore i General Purpose**. Detta kan ge betydande kostnadsbesparingar jämfört med att betala full licenskostnad i molnet.

Den andra delen av bilden handlar om **nätverksarkitekturen**, särskilt för **Azure SQL Managed Instance**. Tjänsten distribueras **i ett eget subnet i ett virtuellt nätverk (VNet)**, vilket ger hög grad av isolering och kontroll. Managed Instance har ett **publikt DNS-namn**, men trafiken går i första hand via en **privat IP-adress** inom VNet. Extern åtkomst via publik IP är **inte aktiverad som standard**, utan kräver att kunden aktivt väljer att öppna en publik endpoint.

Denna arkitektur gör Managed Instance väl lämpad för företag med höga krav på **säkerhet, nätverkskontroll och integration** med befintliga on-premises-miljöer eller andra resurser i Azure. Kombinationen av hybrid licensing och privat nätverksåtkomst gör lösningen både **kostnadseffektiv och säker** för affärskritiska system.

Microsoft offers a number of benefits to SQL Server licensees. With both Azure SQL Database and Azure SQL Database Managed Instances to reduce the cost of running the PaaS offering.

For each core of Enterprise Edition with Active Software Assurance, you are eligible for one vCore of Azure SQL Database or Managed Instance Business Critical, and eight vCores of general purpose. For each core of Standard Edition with Software Assurance that you own, you are eligible for one vCore of General Purpose. This can reduce the total license costs by up to 40%. Effectively, the customer is only paying for the compute and storage costs, and not the software licensing costs.

One major difference Azure SQL Database and Managed Instance is the network architecture, as in its default

settings Azure SQL Database is accessible from a firewalled public endpoint. (You'll learn more about this in the next unit). Managed Instance, by default is only available from within a virtual network (and it deploys into its own subnet within that virtual network). Managed instance relies on this because its architecture is inherently private and all network traffic will come through either peered or connected (via VPN) networks.

The managed instance management services (run by Microsoft) connect over endpoints that have a public IP address, outside of the virtual network. The managed instance is protected from other traffic using this IP through a combination of firewalls, network security groups, and a user defined route table. Additionally, a network intent policy is created to block changes to the existing networking configuration. Management traffic is only routed to nodes only if it's received over a pre-defined set of ports, from a specific Microsoft set of IP addresses, and is authenticated via certificates.

Automatic Tuning



Provides the following features configured at the service level:

- Query Store/Query Performance Insight
- Identifying Expensive Queries
- Force Last Good Execution Plan
- Adding Indexes
- Removing Unused Indexes

© Copyright Microsoft Corporation. All rights reserved.

Running in a PaaS service allows for you to take advantage of additional compute resources on Azure to allow value added services to run. One of the best of these features, is automatic database tuning. This feature is built on top of the query store and is partially implemented in SQL Server, but it is has four key features:

- Identify Expensive Queries
- Forcing Last Good Execution Plan
- Adding Indexes
- Removing Indexes

The query store acts as a flight data recorder, capturing execution plan history, as well as runtime details for most query executions. This allows a data repository to build intelligence from, which is how the plan forcing feature works. The service looks for query plan regression, where estimated CPU time for a given query increases by a certain amount or the number of errors in the current execution plan is higher than the recommended plan, the database engine will force use of the prior, faster plan. This allows for automatic execution plan correction if performance starts to go awry.

Azure SQL builds on this feature and includes automatic index management which can identify indexes which can improve the performance of your queries and deploy them. It can also remove unnecessary indexes which will reduce storage consumption as well improve insert and update performance. The Azure services uses a combination of built-in intelligence and advanced heuristics to determine the best indexes for your query patterns. These indexes are tested on a shadow copy of your database and then ultimately implemented into your database.

All of the aforementioned tuning features have the ability to be turned off in the event you would like to have more control over your environment

Machine Learning Services



Provides machine learning operations within your relational database structure, by using Python and R packages



Provides high-performance capabilities:

- Train machine learning models based on either sampled dataset or population dataset
- Reduce complexity in security and compliance
- Deploy machine learning models using T-SQL stored procedures
- Use of open-source libraries like scikit-learn, PyTorch, and TensorFlow.

```
-- Enable Machine Learning Services feature
EXEC sp_configure 'external scripts enabled', 1;
RECONFIGURE WITH OVERRIDE;
```

© Copyright Microsoft Corporation. All rights reserved.

Absolut – här är både en **förklaring av bilden** och en **bedömning av hur aktuell den är i dag**, i ljuset av modern AI-utveckling.

Bilden beskriver **Machine Learning Services i SQL Server / Azure SQL Managed Instance**, ofta känt som **SQL Server Machine Learning Services** (tidigare R Services). Funktionen gör det möjligt att **köra Python- och R-kod direkt i databasen**, nära datat. Det innebär att man kan träna och köra ML-modeller utan att först flytta data till ett externt ML-system, vilket minskar latens och förenklar säkerhet och compliance.

Tjänsten stödjer **öppen källkods-bibliotek** som *scikit-learn*, *TensorFlow* och *PyTorch* och låter dig träna modeller på hela datamängder eller samplingsar. Modeller kan sedan **exponeras via T-SQL stored procedures**, vilket gör dem enkla att integrera i befintliga applikationer och ETL-flöden. Kodexemplet i bilden visar hur funktionen aktiveras genom `external scripts enabled`.

Hur aktuell är detta i dag?

Tekniskt sett är funktionen **fortfarande aktuell och stöds**, särskilt i **SQL Server (on-prem)** och **Azure SQL Managed Instance**. Den används främst för:

In-database analytics
 Prediktioner nära transaktionella system
 Reglerade miljöer där data inte får lämna databasen

Däremot är den **inte längre i framkant av AI-utvecklingen**. Moderna AI-lösningar bygger oftare på:

Azure Machine Learning

Azure OpenAI

Separata ML-pipelines och feature stores
 GPU-accelererad träning och inferens

Sammanfattningsvis:

Machine Learning Services är **stabil, mogen och relevant för klassisk ML och prediktiv analys**, men för **generativ AI, stora modeller och snabb innovation** är externa AI-plattformar i Azure i dag det naturliga

förstahandsvalet.

Machine Learning Services provides machine learning operations within your relational database structure. This feature supports Python and R packages, ideal for high-intensive predictive capabilities. This option is available on SQL Managed Instance, SQL Server on Azure virtual machine, and on-premises SQL Server.

Applications can use relational database on Azure combined with machine learning high-performance capabilities, where you can:

- Train machine learning models based on either sampled dataset or population dataset.
- Reduce complexity in security and compliance, where you don't need to relocate your data to build and train your machine learning models.
- Deploy machine learning models using T-SQL stored procedures that support Python or R programming language.
- Use of open-source libraries like scikit-learn, PyTorch, and TensorFlow.

For busy environments, you can use the [T-SQL PREDICT function](#), which allows you to accelerate predictions based on your stored model.

Knowledge check



An Azure SQL Database Managed Instance represents what kind of cloud service?

- ☐ Software as a Service (SAAS)
- ☐ Infrastructure as a Service (IAAS)
- ☒ Platform as a Service (PAAS)



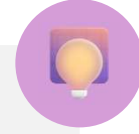
Which of the following is NOT a valid reason for migrating your database into an IaaS environment?

- ☐ Your applications need to run older versions of SQL Server, such as SQL Server 2016
- ☒ You want Azure to manage all the upgrades, patching and server configuration
- ☐ You need to use other SQL Server services with your application, such as SQL Server Analysis Services (SSAS), Integration Services (SSIS) and Reporting Services (SSRS)



Which of the following is NOT an Azure SQL Database deployment option?

- ☒ Availability Zones
- ☐ Serverless
- ☐ Elastic Pools



© Copyright Microsoft Corporation. All rights reserved.

1. An Azure SQL Database Managed Instance represents what kind of cloud service?

Software as a Service (SaaS) – ✗ Fel

→ Du ansvarar fortfarande för databaser, användare och schema.

Infrastructure as a Service (IaaS) – ✗ Fel

→ Du hanterar inte operativsystem eller SQL-installation själv.

Platform as a Service (PaaS) – ✓ Rätt

→ Azure hanterar drift, patchning, backup och hög tillgänglighet.

2. Which of the following is NOT a valid reason for migrating your database into an IaaS environment?

Applications need older SQL Server versions (e.g. SQL Server 2016) – ✗ Fel

→ IaaS behövs för full versionskontroll och äldre SQL-versioner.

You want Azure to manage upgrades, patching and configuration – ✓ Rätt

→ Detta är ett PaaS-skäl, inte ett IaaS-skäl.

You need SSAS, SSIS or SSRS with your application – ✗ Fel

→ Dessa kräver ofta IaaS för full funktionalitet.

3. Which of the following is NOT an Azure SQL Database deployment option?

Availability Zones – ✓ Rätt

→ Det är en tillgänglighetsfunktion, inte en deployment-modell.

Serverless – ✗ Fel

→ Serverless är en giltig Azure SQL Database compute-tier.

Elastic Pools – ✗ Fel

→ Elastic pools är en etablerad deployment-modell i Azure SQL.

Vill du kan jag även formulera detta som **facit-slides, quiz-feedback** eller **examensförklaringar**.

Deploy IaaS solutions with Azure SQL

<https://learn.microsoft.com/training/modules/deploy-iaas-solutions-with-azure-sql/>

© Copyright Microsoft Corporation. All rights reserved.

The most common reason for deploying SQL Server in an Azure Virtual Machine (VM) is because you want an easy, straightforward method to migrate an existing on-premises SQL Server into the cloud. Understanding the options and methods for deploying SQL Server in an Azure VM is critical to ensure a successful migration. Infrastructure as a Service (IaaS) allows for greater flexibility in configuration; however this means that you, as a database administrator, must plan carefully to properly configure VM sizing, storage, and networking options to ensure adequate performance for your workloads.

Objectives

-  Explore the basics of SQL Server in an Infrastructure as a Service (IaaS) Offering
-  Learn the available options for provisioning and deployment
-  Explore performance and security options available
-  Understand the high availability and disaster recovery options

© Copyright Microsoft Corporation. All rights reserved.

Azure SQL platform offerings



SQL Server on Azure Virtual Machines

Best for lift and shift and/or workloads requiring OS-level access

Infrastructure-as-a-Service



Azure SQL Managed Instance

Best for modernizing existing apps



Azure SQL Database

Best for supporting modern cloud apps

Platform-as-a-Service

Azure SQL enabled by Azure Arc
Run Azure SQL on premises and in multicloud environments

Elimination of
Hardware and Software

Operational Efficiency
& Savings

Improved
Business Agility

© Copyright Microsoft Corporation. All rights reserved.

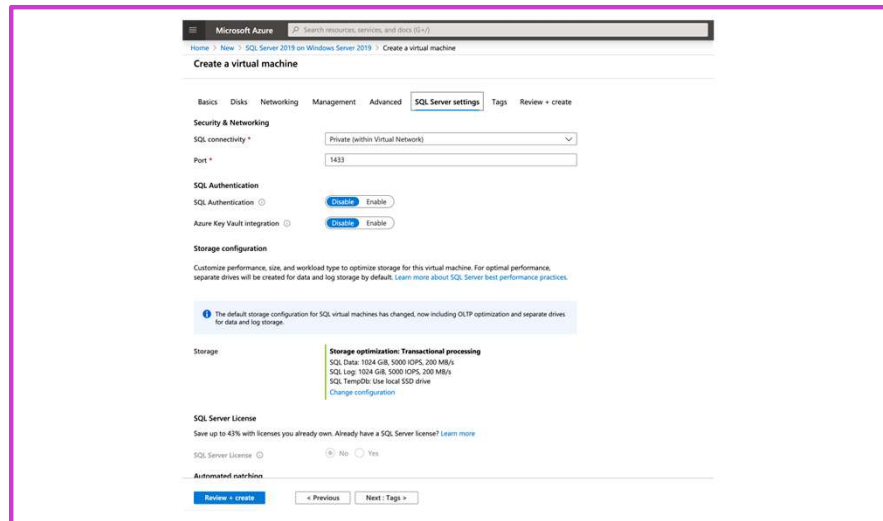
Azure SQL is a family of managed, secure, and intelligent products that use the SQL Server database engine in the Azure cloud.

Azure SQL is built upon the familiar SQL Server engine, so you can migrate applications with ease and continue to use the tools, languages, and resources you're familiar with. Your skills and experience transfer to the cloud, so you can do even more with what you already have.

Azure deployment options

You can deploy Azure resources using the following methods:

- Using the Azure Portal
- Running PowerShell or Azure CLI scripts
- Deploying Azure Resource Manager templates



© Copyright Microsoft Corporation. All rights reserved.

Resources can be deployed using any of the methods shown here. The Azure Portal is typically used for one-off deployments or when you are getting started. PowerShell and Azure CLI can also be used to deploy resources, but the most powerful and comprehensive method for automated deployment is using Azure ARM templates. You will learn more details around deployment in Learning Path 6.

Deploying from the Azure portal

- This method is useful for single VM deployments
- The portal provides step by step instructions for deployment
- This is a manual process
- Not easily repeatable in a consistent matter

Dashboard > Virtual machines > Create a virtual machine

Create a virtual machine

Basics Disks Networking Management Advanced Tags Review + create

Create a virtual machine that runs Linux or Windows. Select an image from Azure marketplace or use your own customized image. Complete the Basics tab then Review + create to provision a virtual machine with default parameters or review each tab for full customization.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group *

[Create new](#)

Instance details

Virtual machine name *

Region *

Availability options

Image *

[Browse all public and private images](#)

Azure Spot instance ☐ Yes ☒ No

Size *

2 vcpus, 8 GiB memory (\$80.30/month)

[Change size](#)

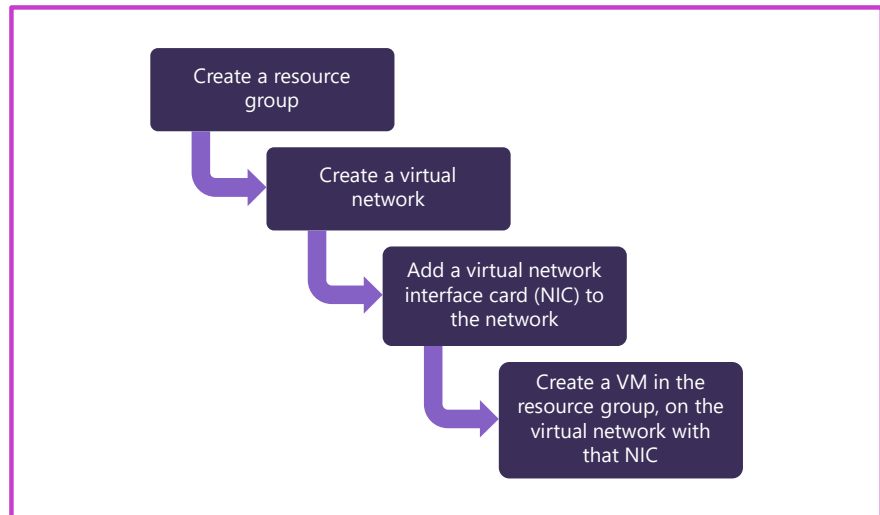
© Copyright Microsoft Corporation. All rights reserved.

Utilizing the portal is quick and easy for a single deployment of virtual machines in Azure.

It isn't easily repeatable from the portal and you must use ARM templates or scripting languages (PowerShell/Azure CLI) to automate.

Deployment using PowerShell/Azure CLI

- While not as flexible as using ARM templates, PowerShell and the Azure CLI provide a repeatable methodology for deploying resources
- Using PowerShell or the Azure CLI are an imperative framework, which means they indicate a specific order of operations to execute
- This approach can offer flexibility and ease of use for some situations compared to the relative complexity of ARM templates



© Copyright Microsoft Corporation. All rights reserved.

Bilden förklarar hur resurser i Azure kan **deployas med PowerShell eller Azure CLI**. Dessa verktyg erbjuder ett **imperativt arbetssätt**, vilket innebär att man steg för steg anger *hur* resurser ska skapas och i vilken ordning. Exemplet visar en typisk sekvens: först skapas en **resource group**, därefter ett **virtuellt nätverk**, sedan ett **nätverkskort (NIC)** och till sist en **virtuell maskin** som kopplas till nätverket.

Jämfört med **ARM-templates** är PowerShell och Azure CLI **mindre deklarativa** och därmed inte lika flexibla för komplexa, återanvändbara miljöer. Däremot är de ofta **enklare att förstå och snabbare att använda**, särskilt vid mindre deploymenter, felsökning eller automatisering i skript. Verktynen passar bra när man vill ha **kontroll över exekveringsordningen** och snabbt kunna anpassa processen efter behov.

Using PowerShell or the Azure CLI are an imperative framework, which means they indicate a specific order of operations to execute. For example:

1. Create a resource group
2. Create a virtual network
3. Add a virtual network interface card (NIC) to the network
4. Create a VM in the resource group, on the virtual network with that NIC

The process of creating a VM using PowerShell or the CLI is complex because of the dependency relationships in for each of the resources that compose the VM.

Azure Resource Manager templates

- ARM Templates provide a repeatable, declarative process for deploying Azure resources at scale
- Allow for integration with popular continuous integration and deployment CI/CD tools
- Allow you to create nearly any Azure resource or option

JSON

```
"resources": [
  {
    "type": "Microsoft.Storage/storageAccounts",
    "apiVersion": "2016-01-01",
    "name": "mystorageaccount",
    "location": "westus",
    "sku": {
      "name": "Standard_LRS"
    },
    "kind": "Storage",
    "properties": {}
  }
]
```

© Copyright Microsoft Corporation. All rights reserved.

Skapa resursgrupp:

```
az group create \
  --name Stockholm \
  --location northeurope
```

Arm template är sparad i c:\dp300

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2023-01-01",
      "name": "stockholmdemostore01",
      "location": "northeurope",
      "sku": {
        "name": "Standard_LRS"
      },
      "kind": "StorageV2"
    }
  ]
}
```

```
}
```

Deploy med:

```
az deployment group create \
  --resource-group Stockholm \
  --template-file storage.json
```

The template has the following sections:

[Parameters](#) - Provide values during deployment that allow the same template to be used with different environments.

[Variables](#) - Define values that are reused in your templates. They can be constructed from parameter values.

[User-defined functions](#) - Create customized functions that simplify your template.

[Resources](#) - Specify the resources to deploy.

[Outputs](#) - Return values from the deployed resources.

Declarative syntax: Resource Manager templates allow you to create and deploy an entire Azure infrastructure declaratively. For example, you can deploy not only virtual machines, but also the network infrastructure, storage systems and any other resources you may need.

Repeatable results: Repeatedly deploy your infrastructure throughout the development lifecycle and have confidence your resources are deployed in a consistent manner. Templates are idempotent, which means you can deploy the same template many times and get the same resource types in the same state. You can develop one template that represents the desired state, rather than developing lots of separate templates to represent updates.

Orchestration: You don't have to worry about the complexities of ordering operations. Resource Manager orchestrates the deployment of interdependent resources, so they're created in the correct order. When possible, Resource Manager deploys resources in parallel so your deployments finish faster than serial deployments. You deploy the template through one command, rather than through multiple imperative commands.

Built-in validation: Your template is deployed only after passing validation. Resource Manager checks the template before starting the deployment to make sure the deployment will succeed. Your deployment is less likely to stop in a half-finished state.

Modular files: You can break your templates into smaller, reusable components and link them together at deployment time. You can also nest one template inside another templates.

Create any Azure resource: You can immediately use new Azure services and features in templates. As soon as a resource provider introduces new resources, you can deploy those resources through templates. You don't have to wait for tools or modules to be updated before using the new services.

Tracked deployments: In the Azure portal, you can review the deployment history and get information about the template deployment. You can see the template that was deployed, the parameter values passed in, and any output values. Other infrastructure as code services aren't tracked through the portal.

Policy as code: [Azure Policy](#) is a policy as code framework to automate governance. If you're using Azure policies, policy remediation is done on non-compliant resources when deployed through templates.

Deployment Blueprints: You can take advantage of [Blueprints](#) provided by Microsoft to meet regulatory and compliance standards. These blueprints include pre-built templates for various architectures.

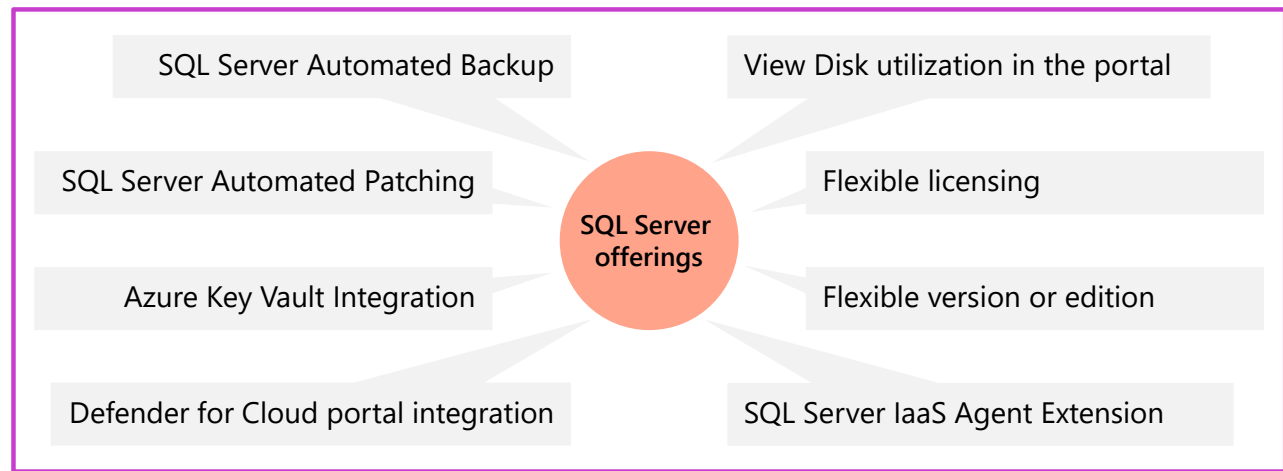
CI/CD integration: You can integrate templates into your continuous integration and continuous deployment (CI/CD) tools, which can automate your release pipelines for fast and reliable application and infrastructure updates. By using Azure DevOps and Resource Manager template task, you can use Azure Pipelines to continuously build and deploy Azure Resource Manager template projects. To learn more, see [VS project with pipelines](#) and [Continuous integration with Azure Pipelines](#).

Exportable code: You can get a template for an existing resource group by either exporting the current state of the resource group, or viewing the template used for a particular deployment. Viewing the [exported template](#) is a helpful way to learn about the template syntax.

Authoring tools: You can author templates with [Visual Studio Code](#) and the template tool extension. You get

intellisense, syntax highlighting, in-line help, and many other language functions. In addition to Visual Studio code, you can also use [Visual Studio](#).

SQL Server IaaS Agent Extension



© Copyright Microsoft Corporation. All rights reserved.

Den här bilden visar vad **SQL Server IaaS Agent Extension** ger dig när du kör **SQL Server på en virtuell maskin i Azure (IaaS)**.

I grunden är det en **Azure-agent** som installeras på VM:en och gör att Azure kan **förstå och hantera SQL Server** bättre via portalen. Utan extensionen är VM:en "bara" en vanlig virtuell maskin med SQL Server installerat manuellt.

Med SQL Server IaaS Agent Extension får du bland annat:

- Automatiska backuper** av SQL Server till Azure Storage
- Automatisk patchning** av SQL Server, schemalagd och kontrollerad
- Integration med Azure Key Vault** för säker hantering av lösenord och certifikat
- Överblick av disk- och lagringsanvändning** direkt i Azure Portal
- Flexibel licenshantering**, inklusive Azure Hybrid Benefit
- Möjlighet att ändra **SQL-version eller edition** på ett kontrollerat sätt
- Integration med Microsoft Defender for Cloud** för säkerhetsrekommendationer

Kort sagt:

SQL Server IaaS Agent Extension gör att **SQL Server på VM i Azure beter sig mer som en PaaS-tjänst**, utan att du tappar den fulla kontroll och kompatibilitet som IaaS ger. Den är därför **starkt rekommenderad** för alla SQL Server-installationer på Azure VM.

- Reduce Administrative overhead.
- Allows you to view information about your SQL Server's configuration and storage utilization.
- Code that is executed on your VM post-deployment - for deploying configurations, anti-virus, or a Windows feature.

While platform as a service (PaaS) offerings such as Azure SQL Database and Azure SQL Managed Instance offer a number of benefits, many applications still require a VM running SQL Server. Some reasons for this include:

Older Versions of SQL Server – If an application requires an older version of SQL Server for vendor support, running inside a VM is the best option for them, because it allows for the application to be supported by that vendor. While Microsoft has recommended supporting applications based on database compatibility level, many vendors have yet to adopt these measures.

Use of other SQL Server Services – While Analysis Services and to an extent Integration Services (through the use of Azure Data Factory) are available as PaaS offerings, many customers maximize their licensing by running these services on the same machine as the database engine.

General Application Incompatibility – This is a little bit of a catch-all, but for example, Azure SQL Database does not support cross-database querying, while Managed Instance does. Some applications may require additional services to be co-located with the database instance in a manner that is not compatible with a PaaS offering.

In addition to the above reasons, many organizations find moving into VMs to be an easier migration path for their initial foray into the cloud. The networking and storage configurations are perceived to be easier to implement than a PaaS solution. With this in mind, SQL Server offers a number of offerings to maximize your productivity using VMs in Azure

SQL Server IaaS Agent Extension

Extensions are code that are executed on your VM post-deployment, typically to perform post deployment configurations, like installing anti-virus, or installing a Windows feature. The SQL Server IaaS Agent Extension provides three key features that can reduce your administrative overhead.

- **SQL Server Automated Backup** – This automates the scheduling of your backups on the VM. The backups are taken to Azure Blob Storage
- **SQL Server Automated Patching** – This allows you to configure a patching window where Windows Updates to your VM can take place. Only SQL Server updates that are pushed down through the Windows Update process will be applied. At the time of this writing, that is limited to SQL Server GDR updates, which means in order to keep your SQL Server VM fully patched, the DBA would need to install cumulative updates (CUs) to SQL Server.
- **Azure Key Vault Integration** – This enables you to use Azure Key Vault as secured storage location for SQL Server certificates, backup encryption keys, and any other secrets such as service account passwords.

Other main features:

- **Defender for Cloud portal integration**
- **View Disk utilization in the portal**
- **Flexible licensing**
- **Flexible version or edition**
- **SQL Server IaaS Agent Extension**

In addition to these features the extension allows you to view information about your SQL Server's configuration and storage utilization.

SQL Server licensing models in Azure

Without Software Assurance

Use a SQL Server image from the marketplace and pay-per-minute. This is known as Pay as You Go licensing

With Azure Hybrid Licensing Benefit your options are

- Bring Your Own License (BYOL) – This is check box in the Azure Portal
- Manually installing SQL Server on a Windows or Linux VM
- Uploading a SQL Server VM image (VHD) to Azure and deploying that image as an Azure VM

© Copyright Microsoft Corporation. All rights reserved.

Bilden visar de två huvudsakliga **licensmodellerna för SQL Server i Azure (IaaS)** och skillnaden mellan dem.

Utan Software Assurance använder du färdiga **SQL Server-images från Azure Marketplace**. Licenskostnaden för SQL Server ingår då i VM-priset och debiteras **per minut** enligt modellen *Pay as You Go*. Det är enkelt att komma igång och kräver inga befintliga licenser, men blir ofta dyrare på sikt.

Med Azure Hybrid Benefit (Software Assurance) kan du **återanvända befintliga SQL Server-licenser**. Du kan antingen markera *Bring Your Own License* i portalen, installera SQL Server manuellt på en Windows- eller Linux-VM, eller ladda upp en egen SQL Server-image. Detta sänker licenskostnaden avsevärt men kräver att du redan äger licenser med Software Assurance.

Kort sagt: *Pay as You Go* ger enkelhet, medan *Hybrid Benefit* ger lägre kostnad.

If you aren't participating in the Microsoft Software Assurance (SA) program, you can deploy an image from the Azure Marketplace containing a preconfigured SQL Server, and pay-per-minute for the use of SQL Server. This is referred to as the Pay as you Go model and the cost of the SQL Server license is included with the cost of the virtual machine.

If you are participating in the Microsoft Software Assurance (SA) program you have more flexibility in how your SQL Server is licensed:

You can use the previous method and pay-per-minute by deploying a virtual machine image containing a SQL Server from the Azure Marketplace

You can Bring Your Own License (BYOL) when deploying the virtual machine that doesn't contain a preconfigured SQL Server instance. This means that you already have purchased a valid SQL Server license for your on-premises infrastructure. This license can be applied to the virtual machine to ensure that you are properly licensed. You must report the usage of licenses to Microsoft by using the License Mobility verification form within 10 days of implementing the virtual machine.

When choosing this method, you can manually install SQL Server through media you have obtained, or you can choose to upload a virtual machine image to Azure.

In addition to flexible licensing options for SQL Server, there is also Windows Server licensing options that can be taken advantage of known as the Azure Hybrid Use Benefit (AHUB). Similar to applying a SQL Server license you already have purchased, you are able to take advantage of Windows Server licenses you already own.

Azure Virtual Machine families

- **General purpose:**
Balanced CPU to memory ratio
- **Compute optimized:**
High CPU to memory ratio
- **Memory optimized:**
High memory to CPU ratio
- **Storage optimized:**
High disk throughput provided by local storage
- **High performance compute:**
Most powerful CPU

Type	Sizes
General purpose	B, Dsv3, Dv3, Dasv4, Dav4, DSv2, Dv2, Av2, DC, DCv2, DCadsv5
Compute optimized	Fsv2
Memory optimized	Esv3, Ev3, Easv4, Eav4, Mv2, M, DSv2, Dv2, Edv4
Storage optimized	Lsv2
GPU	NC, NCv2, NCv3, ND, NDv2, NV, NVv3, Nvv4, NVadsa10v5
High performance compute	HB, HBv2, HC, H

© Copyright Microsoft Corporation. All rights reserved.

The following six series each have various sizes available, the details of which are fully described in the Azure Portal when you choose the option to select your VM size.

[General purpose](#) – These VMs provide a balanced ration of CPU to memory. This VM class is ideal for testing and development, small to medium database servers, and low to medium traffic web servers.

[Compute optimized](#) – Compute optimized VMs have a high CPU-to-memory ratio and are good for medium traffic web servers, network appliances, batch processes, and application servers. These VMs can also support machine learning workloads that cannot benefit from GPU-based VMs.

[Memory optimized](#) – These VMs provide high memory-to-CPU ratio. These VMs cover a broad range of CPU and memory options (all the way up to 4 TB of RAM) and are well suited for most database workloads.

[Storage optimized](#) – Storage optimized VMs provide fast, local, NVMe storage that is ephemeral. This makes them a good candidate for scale-out data workloads such as Cassandra. It is possible to use them with SQL Server, however since the storage is ephemeral, you would need to ensure you configured data protection using a feature like Always On Availability Groups or Log Shipping.

[GPU](#) – Azure VMs with GPUs are targeted at two main types of workloads—naturally graphics processing operations like video rendering and processing, but also massively parallel machine learning workloads that can take advantage of GPUs.

[High performance compute](#) – High Performance Compute workloads support applications that can scale horizontally to thousands of CPU cores. This support is provided by high performance CPU and remote direct memory access (RDMA) networking that provides low latency communications between VMs.

Azure Virtual Machine sizing

A-Series – Entry-level for dev/test

H-Series – High performance computing

Bs-Series – Economical bursting

Ls-Series – Storage optimized

D-Series – General purpose compute

M-Series – Memory optimized

E-Series – In-memory hyper-threaded applications optimized

Mv2-Series – Largest memory optimized

F-Series – Compute optimized

N-Series – GPU enabled

G-Series – Memory and storage optimized

© Copyright Microsoft Corporation. All rights reserved.

Här är en **kort och pedagogisk beskrivning av 4 populära VM-storleksserier i Azure:**

B-series (Bs)

Bs-serien är populär för **lätta arbetslaster** som dev/test, små webbservrar och labbmiljöer. De är billiga och använder ett **burst-koncept**, där VM:en kan låna CPU-kapacitet vid tillfälliga toppar men annars går på låg nivå.

D-series

D-serien är den vanligaste **"standard-VM:en"** i Azure. Den passar bra för **affärsapplikationer, API:er och mindre databaser** och erbjuder en balanserad kombination av CPU, minne och I/O.

E-series

E-serien är **minnesoptimerad** och mycket populär för **SQL Server, SAP och andra minneskrävande applikationer**. Den ger mer RAM per vCore än D-serien och är ofta förstahandsval för databaser.

M-series

M-serien används när **extremt mycket minne** behövs, till exempel för mycket stora SQL Server- eller SAP HANA-installationer. Den är dyr men oersättlig i vissa enterprise-scenarier.

👉 Tumregel:

Bs = billigt, D = standard, E = databaser, M = riktigt stora system.

Sizes for virtual machines in Azure

<https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/overview>

SQL Server configuration

Configure specific SQL Server settings like Security and Networking, SQL Authentication preferences, SQL instance settings, and other options.

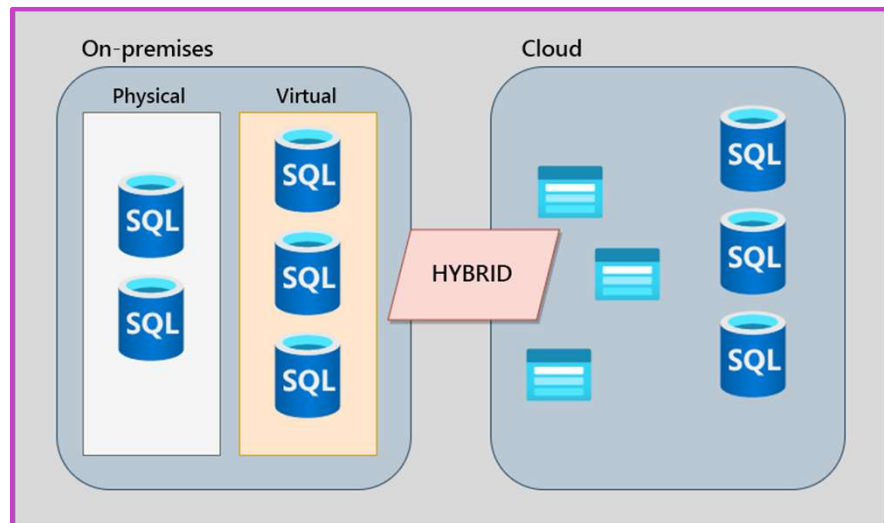
The image displays two screenshots of the Azure portal's 'Create a virtual machine' wizard, specifically the 'SQL Server settings' tab. The left screenshot shows the 'SQL Authentication' section with 'SQL Authentication' and 'Azure Key Vault integration' both set to 'Enable'. The 'SQL instance settings' section shows 'Default configuration' with 'SQL Server memory (max)' set to 2147483647 MB. The right screenshot shows the 'SQL Server License' section with 'SQL Server License' set to 'No' and 'Automated patching' set to 'Enabled' with a patching window of 'Sunday at 2:00'. The 'Automated backup' section is set to 'Disable'.

© Copyright Microsoft Corporation. All rights reserved.

[Provision SQL Server on Azure VM \(Azure portal\) - SQL Server on Azure VMs | Microsoft Docs](https://learn.microsoft.com/azure/azure-sql/virtual-machines/windows/create-sql-vm-portal)
<https://learn.microsoft.com/azure/azure-sql/virtual-machines/windows/create-sql-vm-portal>

Understand hybrid scenarios

- Organizations commonly have a mixture of physical and virtualized deployments of SQL Server
- Offers the benefits of both on-premises and cloud services – Extends on-prem solutions
- Cloud component is usually used for storage or SQL Server VMs



© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver ett **hybrid-scenario**, där en organisation använder både **on-premises** och **Azure-molnet** samtidigt. I praktiken innebär det att SQL Server körs både på **fysiska servrar och virtuella maskiner lokalt**, och parallellt i **Azure**.

Hybridlösningar används ofta när man vill **behålla befintliga system** men samtidigt dra nytta av molnets fördelar, till exempel skalbarhet, backup, hög tillgänglighet eller katastrofåterställning. Molndelen används ofta för **lagring, säkerhetskopior, sekundära repliker** eller **SQL Server på Azure-VM**.

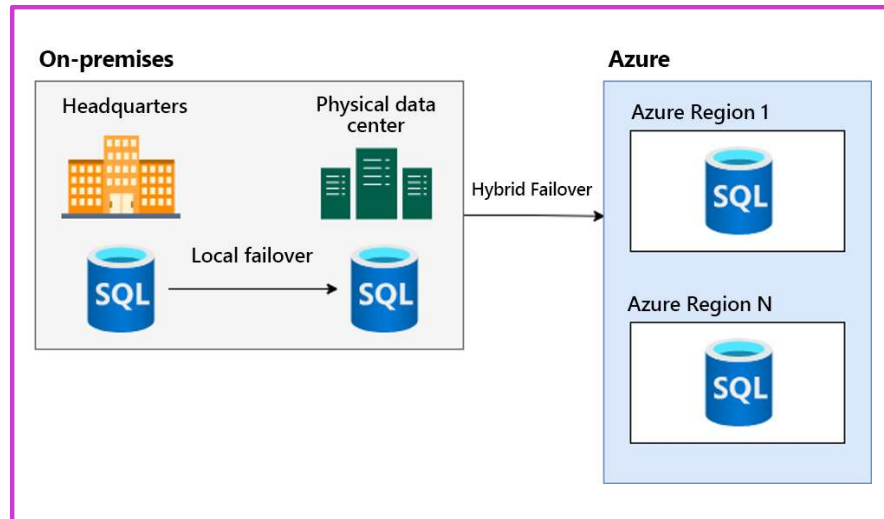
På så sätt kan organisationen stegvis modernisera sin miljö utan att göra en full migrering direkt. Hybridmodellen kombinerar **kontroll och närhet till data lokalt** med **flexibilitet och tjänster i molnet**, vilket gör den till ett vanligt och praktiskt val för många företag.

Suppose your organization has a significant investment in SQL Server infrastructure on-premises and in local data centers. In that case, it's crucial to be aware that using the cloud isn't an all-or-nothing proposition. There are ways to use existing on-premises infrastructure in a hybrid capacity with Azure to improve operational resiliency and reduce cost.

Implementing a hybrid infrastructure is also an excellent first step in evaluating cloud computing for organizations that have been traditionally on-premises and skeptical of the cloud. It's common for today's organizations to have a mixture of physical and virtualized deployments of SQL Server on-premises, both of which may extend to the cloud as part of a hybrid solution. The hybrid SQL Server platform offers the benefits of both on-premises and cloud services; it's a complimentary middle ground between them.

Hybrid scenarios for SQL Server – Disaster Recovery

- Most common scenario for a hybrid deployment of SQL Server.
- Organizations ensure business continuity during catastrophic events.
- Azure is used for DR failover (to one or more regions) while the regular day-to-day processing continues to use on-premises servers for local high availability.



© Copyright Microsoft Corporation. All rights reserved.

Bilden visar ett vanligt **hybrid-scenario för SQL Server med fokus på Disaster Recovery (DR)**. I vardagen körs SQL Server **on-premises**, ofta med lokal hög tillgänglighet genom till exempel failover-kluster eller Always On Availability Groups inom det egna datacentret. Detta hanterar normala driftstörningar lokalt.

Azure används som ett **sekundärt DR-mål** vid större incidenter, som strömbavbrott, brand eller total förlust av datacentret. Data replikeras till en eller flera **Azure-regioner**, och vid en katastrof kan systemen **faila över till Azure**. På så sätt säkerställs **affärskontinuitet** utan att man behöver drifta ett eget sekundärt datacenter.

Hybrid DR är ofta det **första steget till molnet**, eftersom det ger ökad säkerhet och flexibilitet utan att ändra den dagliga driften.

Disaster Recovery is the most common scenario for a hybrid deployment of SQL Server. Disaster recovery means that organizations ensure business continuity in the light of catastrophic events. Organizations may distribute deployments across multiple data centers for failover in an on-premises approach. These data centers typically reside within the same geographic region as the organization, thus susceptible to more significant regional disasters.

Physical data centers are also costly to deploy, monitor, and maintain. Arguably, the cost of spinning up Azure SQL Server virtual machines in various geographical regions is much less than establishing a new physical data center in another geography.

Hybrid scenarios for SQL Server – SQL Server backups

SQL Server backups

- Backups may go directly into Azure Storage via URL or Azure file share (SMB)
- Protects against data loss when on-site backup storage fails
- Can be restored to virtual machines in Azure and tested as part of Disaster Recovery procedures

Azure Storage

- Store on-premises SQL Server data files for user databases – User files and not system databases
- In the case of local storage failure, the user files are safely stored in the cloud, preventing data loss
- Built in reliability guarantees stored files in the cloud are more resilient

© Copyright Microsoft Corporation. All rights reserved.

Bilden visar ett **hybrid-scenario för SQL Server där Azure används för backup och dataskydd**. I stället för att lagra alla säkerhetskopior lokalt kan **SQL Server backupar skrivas direkt till Azure Storage**, antingen via URL (Blob Storage) eller via Azure File Share (SMB). Detta skyddar mot dataförlust om lokal backup-lagring skulle gå sönder eller bli otillgänglig.

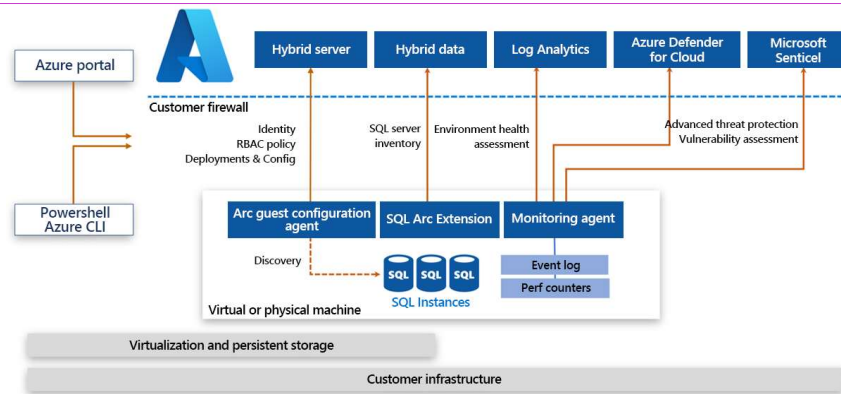
Backupar som lagras i Azure kan dessutom **återställas till virtuella maskiner i Azure**, vilket gör dem användbara som en del av **Disaster Recovery-rutiner** och för regelbundna återställningstester. Azure Storage kan även användas för att lagra **användardatabasfiler** (inte systemdatabaser), vilket ger extra skydd vid lokala lagringsfel.

Tack vare Azure's inbyggda **replikering och hållbarhetsgarantier** är data i molnet ofta mer robust än lokal lagring. Detta gör molnbaserade backupar till ett enkelt och kostnadseffektivt första steg mot hybrid drift.

SQL Server Backups is another common hybrid scenario. Backups may be done directly into Azure Storage via URL or Azure file share (SMB). This scenario protects against data loss when on-site backup storage fails. In addition, these backups may also be restored to virtual machines in Azure and tested as part of Disaster Recovery procedures.

Another scenario uses Azure Storage to store on-premises **SQL Server data files for user databases**. Note that these are user files and not system databases. In the case of local storage failure, the user files are safely stored in the cloud, preventing data loss. In addition, by using Azure storage, there are [built-in reliability guarantees](#) so storing these files in the cloud is more resilient. For this hybrid scenario, it's essential to keep the network communication secure, evaluate the network latency of the solution, and ensure the storage account is locked down using ACLs and Microsoft Entra ID.

Hybrid scenarios for SQL Server – Azure Arc enabled SQL Servers



Extends and centralizes

Enables the inventory

Security

© Copyright Microsoft Corporation. All rights reserved.

Den här bilden visar hur **Azure Arc-enabled SQL Servers** används i ett **hybrid-scenario**. Azure Arc gör det möjligt att **ansluta SQL Server-instanser som körs utanför Azure** – till exempel on-premises eller i andra moln – till **Azure Portal** som om de vore Azure-resurser.

Med **Azure Arc** installerar man en agent på den fysiska eller virtuella servern. Därefter kan Azure användas för **central inventering, konfigurationshantering, övervakning** och **säkerhet**. Alla SQL-instanser blir synliga på ett ställe, oavsett var de körs. Via **Log Analytics** samlas prestandadata och loggar in, och **Microsoft Defender for Cloud** och **Sentinel** kan användas för hotdetektering och säkerhetsanalys.

För SQL Server innebär Arc att man får:

En samlad **översikt över alla SQL-instanser**
Policy-styrning och efterlevnad via Azure
Säkerhetsrekommendationer och sårbarhetsanalys

Central hantering utan att flytta databaserna till Azure

Kort sagt:

Azure Arc är ett sätt att "förlänga Azure" till din befintliga infrastruktur, så att hybrid- och multi-cloud-miljöer kan hanteras, övervakas och säkras från ett gemensamt gränssnitt.

Extends and centralizes Azure management services to SQL Server instances hosted on-premises, in your data centers, on the edge, and in multi-cloud environments.

Enables the inventory of all registered SQL Server deployments and assesses their configurations, usage patterns, and security to provide actions and recommendations based on best practices.

Uses Azure Defender real-time security alerts and vulnerability reporting on both on-premises SQL Servers and their host operating systems and Azure Sentinel for security threat introspection.

Azure Virtual Machine storage

Each Azure VM has two or more disks:

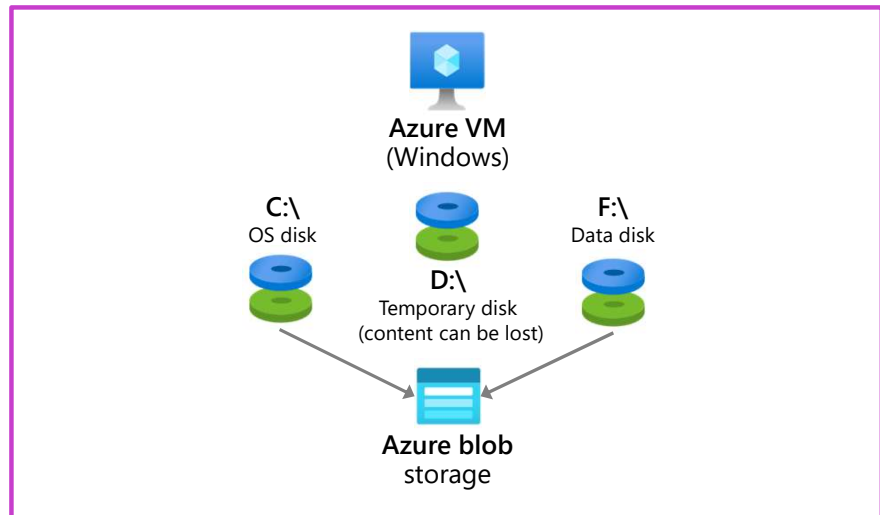
- OS disk
- Temporary disk
- Data disks (optional)

OS and data disks reside in Azure-based storage service:

- Standard (HDD, SSD)
- Premium (SSD)
- Ultra (SSD)

When creating an Azure VM, you can choose between:

- Managed disks (recommended)
- Unmanaged disks



© Copyright Microsoft Corporation. All rights reserved.

Bilden visar hur **lagring fungerar för en Azure Virtual Machine**. Varje Azure-VM har minst två diskar: en **OS-disk** (C:) som innehåller operativsystemet och en **temporär disk** (D:) som används för tillfälliga filer. Den temporära disken är snabb men **innehållet kan förloras** vid omstart eller underhåll och ska därför inte användas för viktiga data. Utöver detta kan man lägga till en eller flera **datadiskar** (t.ex. F:) för applikationer och databaser.

OS- och datadiskar lagras i **Azure Storage** och kan vara av typen **Standard**, **Premium** eller **Ultra SSD**, beroende på prestandakrav. Vid skapande av VM rekommenderas **Managed Disks**, där Azure automatiskt hanterar lagring, skalning och tillgänglighet, i stället för äldre unmanaged disks.

<https://learn.microsoft.com/azure/virtual-machines/disks-types>

Storage recommendations

- SQL Server workloads on Azure should use Premium SSD or Ultra SSD
- Separate transaction logs and data files onto their own volumes, with read-caching enabled on the data file volumes
- Stripe multiple disks using Storage Spaces or Logical Volume Management to get increased IOPs and storage volumes

© Copyright Microsoft Corporation. All rights reserved.

Bilden sammanfattar **rekommenderade lagringsprinciper för SQL Server i Azure** för att uppnå bra prestanda och stabilitet. För produktionslaster bör man använda **Premium SSD eller Ultra SSD**, eftersom de ger låg latens och höga IOPS jämfört med Standard-diskar.

En viktig best practice är att **separera datafiler och transaktionsloggar** på olika diskar eller volymer. Datafiler kan använda read-cache för bättre läsprestanda, medan loggfiler ska ha cache avstängd för att säkerställa korrekt skrivordning.

För krävande arbetslaster kan man **stripa flera diskar** med Storage Spaces (Windows) eller LVM (Linux). Det gör att IOPS och genomströmning kan skalas genom att lägga till fler diskar, snarare än att byta till en enskilt större disk.

The best practices for SQL Server on Azure recommend using Premium Disks pooled for increased IOPs and storage capacity. Data files should be stored in their own pool with read-caching on the Azure disks. Transaction log files will not benefit from this caching, so those files should go into their own pool without caching. TempDB can optionally go into its own pool, or using the VM's temporary disk, which offers low latency since it is physically attached to the physical server where the VMs are running. Properly configured Premium SSD will see latency in single digit milliseconds. For mission critical workloads that require latency lower than that, you should consider Ultra SSD.

Performance consideration – Table and index partitioning

- Can improve query performance of large tables, while improving performance and scalability
- Used when the table becomes large enough that starts compromising query performance
- Maintenance operations on a partitioned table will reduce maintenance duration

```
-- Partition function
CREATE PARTITION FUNCTION PartitionByMonth (datetime2)
AS RANGE RIGHT
-- The boundary values defined is the first day of each month
FOR VALUES ('20210101', '20210201', '20210301',
'20210401', '20210501', '20210601', '20210701',
'20210801', '20210901', '20211001', '20211101',
'20211201');

-- The partition scheme below will use the partition function created above
CREATE PARTITION SCHEME PartitionByMonthSch
AS PARTITION PartitionByMonth
TO (FILEGROUP1, FILEGROUP2, FILEGROUP3, FILEGROUP4,
FILEGROUP5, FILEGROUP6, FILEGROUP7, FILEGROUP8,
FILEGROUP9, FILEGROUP10, FILEGROUP11, FILEGROUP12);

-- Creates a partitioned table called Order that applies PartitionByMonthSch
CREATE TABLE Order ([Id] int PRIMARY KEY, OrderDate datetime2)
ON PartitionByMonthSch (OrderDate) ;
GO
```

© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver **tabell- och indexpartitionering i SQL Server**, en teknik som används för att förbättra prestanda och underhåll av **mycket stora tabeller**. I stället för att lagra all data i en enda struktur delas tabellen upp i **logiska partitioner**, ofta baserat på datum, till exempel per månad eller år.

När frågor filtrerar på partitioneringskolumnen kan SQL Server använda **partition elimination**, vilket innebär att bara relevanta partitioner läses. Det ger snabbare frågor och bättre skalbarhet. Partitionering gör också **underhåll enklare**, till exempel indexunderhåll, arkivering eller radering av gammal data, eftersom detta kan göras per partition i stället för på hela tabellen.

Viktigt att komma ihåg är att partitionering inte automatiskt gör allt snabbare – den ger störst nytta för **stora tabeller med tydliga åtkomstmönster**, ofta i datalager eller logg-/historiktabeller.

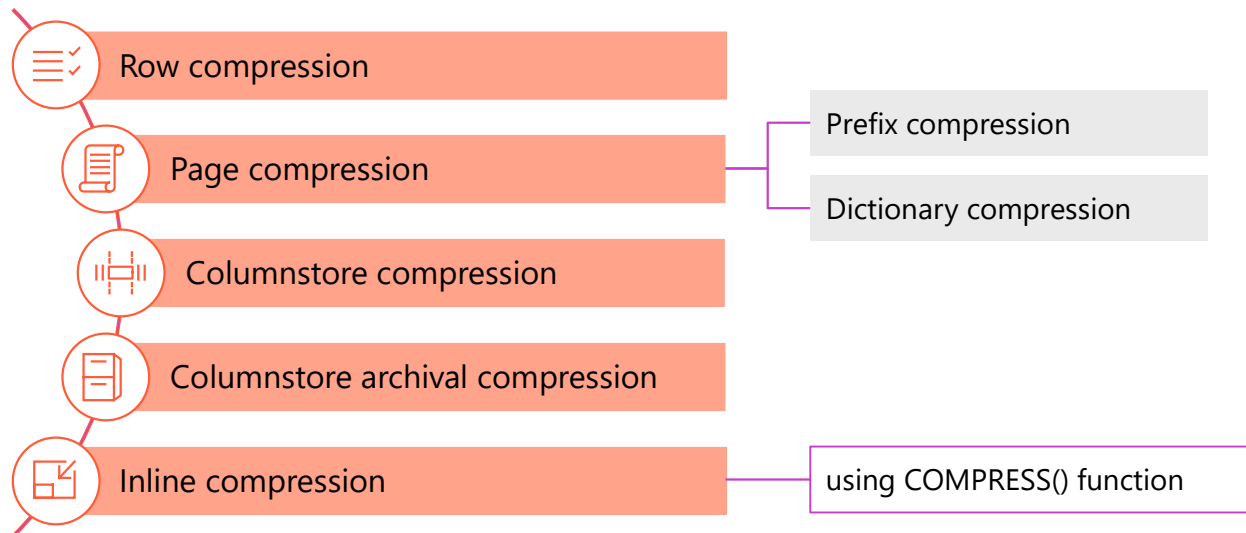
Table partitioning provides many benefits, but often times this strategy is only considered when the table becomes large enough that starts compromising query performance. Identifying which tables are candidates for table partitioning is a good practice that could lead to less disruptions and interventions. When you filter your data using your partition column, only a subset of the data is accessed, not the entire table. Similarly, maintenance operations on a partitioned table will reduce maintenance duration, for example, by compressing specific data in a particular partition or rebuilding specific partitions of an index.

There are four main steps required when defining a table partition:

- The filegroups creation, which defines the files involved when the partitions are created.
- The partition function creation, which defines the partition rules based on the specified column.
- The partition scheme creation, which defines the filegroup of each partition.
- The table to be partitioned.

The example above shows how to create a partition function for January 1, 2021 through December 1, 2021, and distribute the partitions across different filegroups.

Performance consideration – Data compression



© Copyright Microsoft Corporation. All rights reserved.

[Row compression](#) helps store data more efficiently in a row by storing fixed-length data types in variable-length storage format. A compressed row uses 4 bits per compressed column to store the length of the data in the column. NULL and 0 values across all data types take no additional space other than these 4 bits.

[Page compression](#) is a superset of row compression. In addition to storing data efficiently inside a row, page compression optimizes storage of multiple rows in a page, by minimizing the data redundancy. Page compression uses prefix compression and dictionary compression.

[Prefix compression](#) looks for common patterns in the beginning of the column values on a given column across all rows on each page.

[Dictionary compression](#) looks for exact value matches across all columns and rows on each page. Both dictionary and prefix are type-agnostic and see every column value as a bag of bytes.

The data compression feature is available in the Enterprise and Developer editions of SQL Server 2008. Databases with compressed tables or indexes cannot be restored, attached, or in any way used on other editions. To determine whether a database is using compression, query the dynamic management view (DMV) `sys.dm_db_persisted_sku_features`. To determine what is compressed, and how (row or page), query the `data_compression_desc` column in the catalog view `sys.partition`

Additional options for performance

For overall production workload:

- Enable backup compression
- Enable instant file initialization for data files
- Limit autogrowth of the database
- Disable autoshrink/autoclose for the databases
- Move all databases to data disks, including system databases
- Move SQL Server error log and trace file directories to data disks
- Set max SQL Server memory limit
- Enable lock pages in memory
- Enable optimize for adhoc workloads for OLTP heavy environments
- Enable Query Store
- Schedule SQL Server Agent maintenance jobs
- Monitor and manage the health and size of the transaction log files

© Copyright Microsoft Corporation. All rights reserved.

Instructor may highlight that any database change in production should be tested and validated, and that these are overall guidelines that not necessarily reflect specific workloads and scenarios.

[Checklist: Best practices & guidelines - SQL Server on Azure VMs | Microsoft Docs](https://learn.microsoft.com/azure/azure-sql/virtual-machines/windows/performance-guidelines-best-practices-checklist)

<https://learn.microsoft.com/azure/azure-sql/virtual-machines/windows/performance-guidelines-best-practices-checklist>

[SQL best practices assessment - SQL Server on Azure VMs | Microsoft Docs](https://learn.microsoft.com/azure/azure-sql/virtual-machines/windows/sql-assessment-for-sql-vm)

<https://learn.microsoft.com/azure/azure-sql/virtual-machines/windows/sql-assessment-for-sql-vm>

Azure platform High Availability (HA) and Disaster Recovery (DR)

Availability sets

Protection from planned or unplanned Azure maintenance events and local hardware outage

Availability zones

Protection from datacenter failures

Paired Azure regions

Protection from regional failures while preserving data residency and compliance boundaries

If a multi-region disaster happens, one region in each pair will be prioritized for recovery

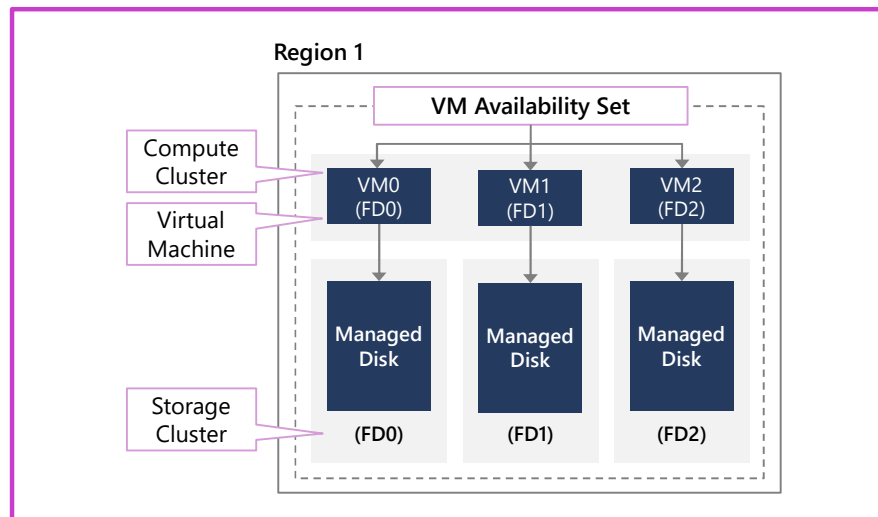
© Copyright Microsoft Corporation. All rights reserved.

One of the major benefits of cloud computing is that platform high availability is part of the architecture. Azure provides a high-level of built-in hardware, storage, and networking redundancy.

Beyond its built-in high availability, the Azure platform offers two options for providing higher levels of availability for VM and some PaaS workloads. *Availability Zones* and *Availability Sets* protect your workloads from planned maintenance activity and potential hardware failures.

Availability sets

- Provide increased availability for multi-VM deployments by ensuring the VM are deployed to different physical hosts
- Use **fault domains** and **update domains** to protect workloads against host update failure and single points of hardware failure



© Copyright Microsoft Corporation. All rights reserved.

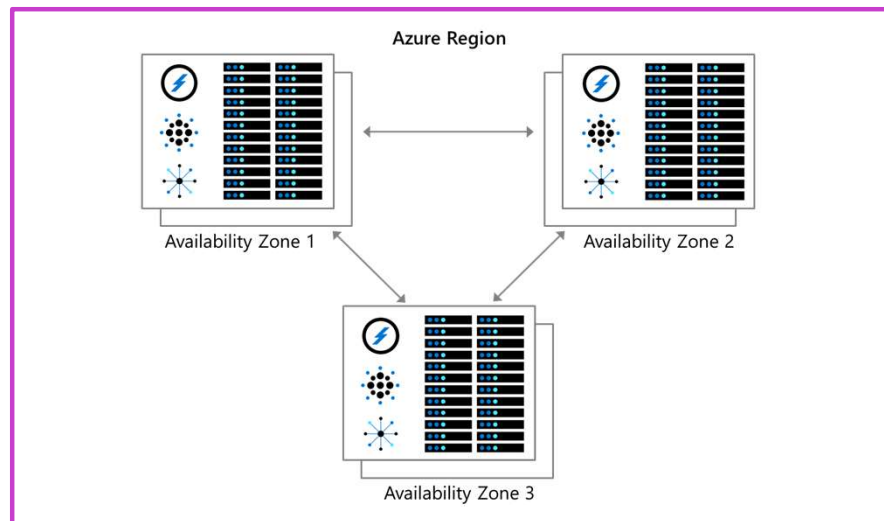
Bilden förklarar **Availability Sets** i Azure, som används för att öka **tillgängligheten** för applikationer som körs på flera virtuella maskiner. När VM:ar placeras i ett availability set ser Azure till att de körs på **olika fysiska servrar** och i olika **fault domains** (olika ström- och nätverkskällor). Dessutom sprids de över **update domains**, vilket innebär att inte alla VM:ar uppdateras eller startas om samtidigt vid planerat underhåll.

Resultatet är att om en fysisk server går ner eller Azure genomför uppdateringar, fortsätter minst en VM att vara tillgänglig. Availability Sets är därför en grundläggande och kostnadseffektiv lösning för hög tillgänglighet inom **en och samma Azure-region**, särskilt för klassiska IaaS-lösningar som SQL Server på VM.

Availability sets in Azure enable you to place the high-availability nodes into separate fault domains and update domains. The Azure platform assigns an update domain and a fault domain to each virtual machine in your availability set. This configuration within a datacenter ensures that during either a planned or unplanned maintenance event, at least one virtual machine is available and meets the Azure SLA of 99.95 percent.

Availability zones

- Allow workloads to be deployed to different data centers in the same Azure region
- The latency between availability zones is low, and generally allows for **synchronous** data replication



© Copyright Microsoft Corporation. All rights reserved.

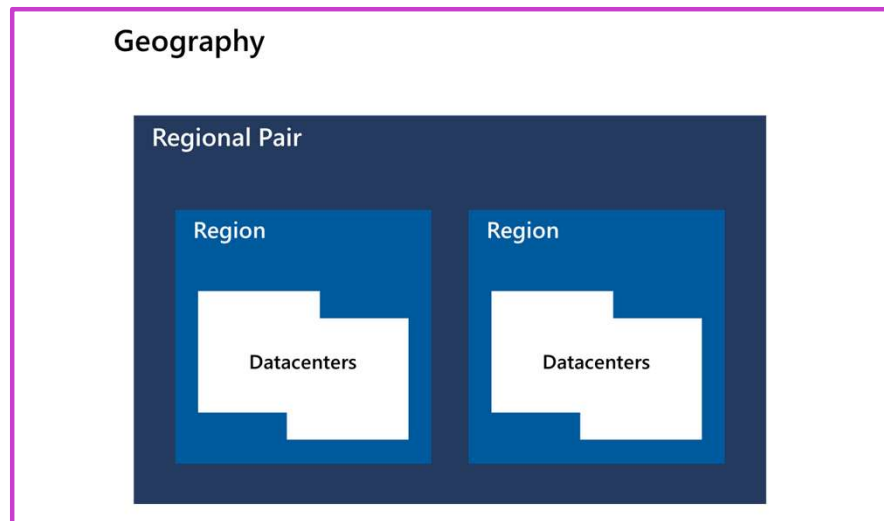
Bilden visar **Availability Zones** i Azure, som är ett mer avancerat sätt att bygga hög tillgänglighet än availability sets. En availability zone är ett **separat fysiskt datacenter** inom samma Azure-region, med egen ström, kylning och nätverk. Genom att placera resurser, till exempel virtuella maskiner eller databaser, i olika zoner skyddas lösningen mot **hela datacenterfel**, inte bara enskilda servrar.

Latensen mellan zonerna är mycket låg, vilket gör att **synkron replikering** ofta är möjlig, till exempel för SQL Server eller Azure SQL Database. Availability Zones används när kraven på tillgänglighet är höga och passar affärskritiska system som måste klara även större infrastrukturfel inom en region.

Availability zones are unique physical locations within an Azure region. Each zone consists of one or more datacenters equipped with independent power, cooling, and networking. The physical separation of availability zones within a region helps protect applications and data from datacenter failures by ensuring that at least one virtual machine is available and meets the Azure SLA of 99.99 percent. There will be additional charges for network-to-network transfers between availability zones.

Paired Azure regions

- All Azure regions have a "paired" region in which deployments and updates are applied
- If a multi-region disaster happens, one region in each pair will be prioritized for recovery
- Regional pairs are set and cannot be changed



© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver **Paired Azure Regions**, som är Azure's strategi för **regional katastrofåterställning**. Varje Azure-region är permanent kopplad till en annan region inom samma geografiska område, till exempel *North Europe* ↔ *West Europe*. Microsoft styr denna parning och den kan inte ändras av kunden.

Syftet är att skydda mot **storskaliga regionala avbrott**. Vid planerade uppdateringar uppdateras regionerna i paret **sekventiellt**, inte samtidigt, vilket minskar risken för driftstopp. Vid en större katastrof prioriteras alltid **minst en region per par** för återställning. Paired regions används ofta tillsammans med geo-replikering och är grunden för många Azure-tjänsters SLA och DR-strategier.

An Azure region consists of a set of data centers deployed within a latency-defined perimeter and connected through a dedicated low-latency network. This ensures that Azure services within an Azure region offer the best possible performance and security.

An Azure geography defines an area of the world containing at least one Azure region. Geographies define a discrete market, typically containing two or more regions, that preserve data residency and compliance boundaries.

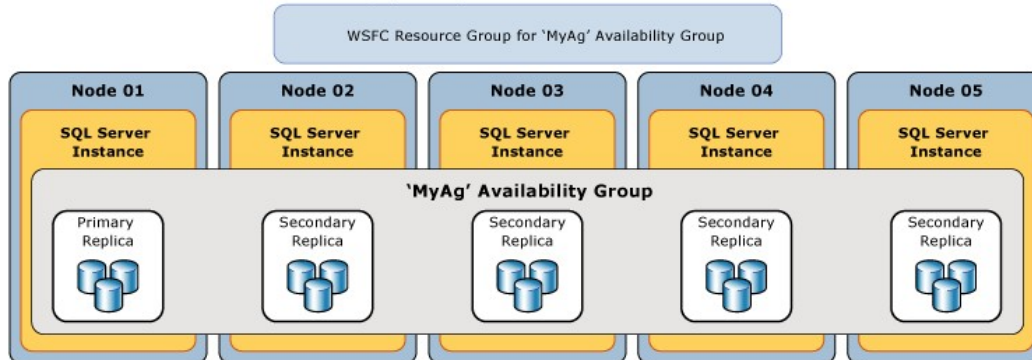
A regional pair consists of two regions within the same geography. Azure serializes platform updates (planned maintenance) across regional pairs, ensuring that only one region in each pair updates at a time. If an outage affects multiple regions, at least one region in each pair will be prioritized for recovery.

Some Azure services take further advantage of paired regions to ensure business continuity and to protect against data loss. Azure provides several storage solutions that take advantage of paired regions to ensure data availability.

Regional pairs are set and cannot be changed. For instance, East US is paired with West US, and UK West is paired with UK South.

Always On Availability Groups

Windows Server Failover Clustering (WSFC) Cluster



Database transactions are committed to the primary replica, and then the transactions are sent either synchronously or asynchronously to all secondary replicas

- **Asynchronous** mode – If the secondary replicas are geographically separate
- **Synchronous** mode – If the replicas are within the same Azure region

© Copyright Microsoft Corporation. All rights reserved.

Bilden visar **Always On Availability Groups**, SQL Servers lösning för **hög tillgänglighet och disaster recovery på databasenivå**. Flera SQL Server-instanser (noder) ingår i ett **Windows Server Failover Cluster (WSFC)**. En instans är **Primary Replica**, där alla skrivningar sker, och en eller flera är **Secondary Replicas**.

Transaktioner skickas från primär till sekundära repliker antingen **synkront** (vanligt inom samma Azure-region för automatisk failover utan dataförlust) eller **asynkront** (vanligt mellan regioner för DR, med viss risk för dataförlust). Sekundära repliker kan även användas för **läsning och backup**. Always On AG är kraftfullt men mer komplext än PaaS-alternativ och används ofta i affärskritiska IaaS-lösningar.

Can be implemented between two or more (up to a maximum of nine) SQL Server instances running on Azure virtual machines or hybrid (on-premises data center to Azure)

Database transactions are committed to the primary replica, and then the transactions are sent either synchronously or asynchronously to all secondary replicas.

The physical distance between the servers determines which availability mode you should choose.

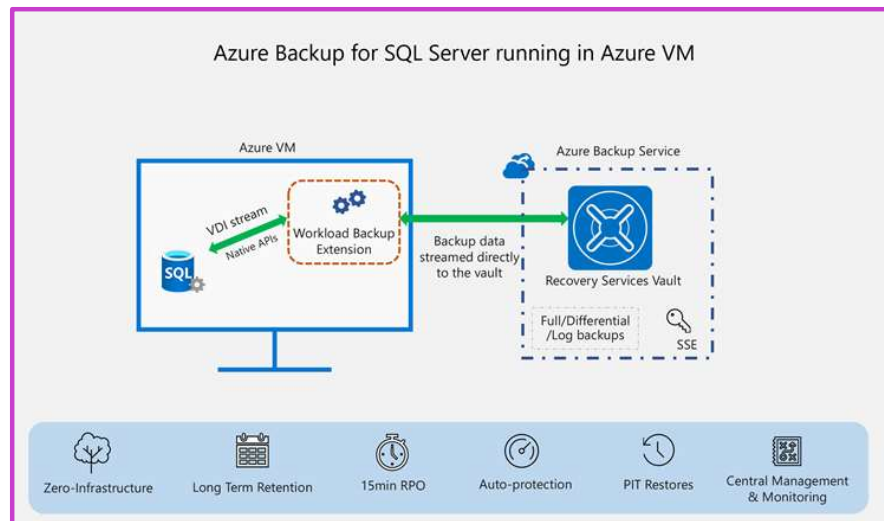
If the secondary replicas are geographically separate - asynchronous availability mode is recommended.

If the replicas are within the same Azure region - synchronous commit can be used.

Provides both high availability and disaster recovery to protect a group of databases, not the entire instance.

Azure Backup for SQL Server

- Provides long-term data retention, automated management and additional data protection
- Offers a more complete backup feature set
- Requires an agent to be installed on the virtual machine
- Central location to manage and monitor the backups



© Copyright Microsoft Corporation. All rights reserved.

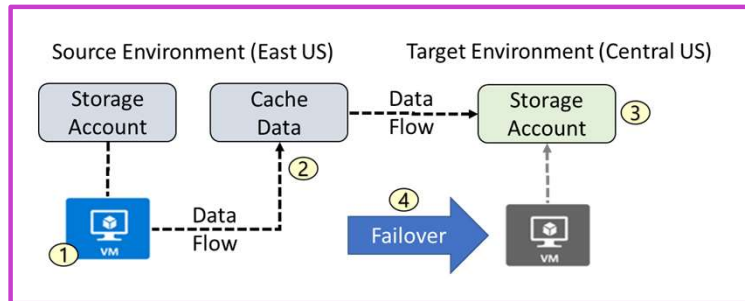
Bilden beskriver **Azure Backup for SQL Server**, som är en **hanterad backuptjänst** för SQL Server som körs på **Azure Virtual Machines**. Tjänsten använder en backup-agent (Workload Backup Extension) på VM:n och tar **fulla, differentiella och loggbakuper** direkt till ett **Recovery Services Vault** i Azure.

Fördelarna är **central hantering**, automatisering och **långtidslagring** utan att du själv behöver bygga backup-infrastruktur. Backupdata lagras säkert med kryptering och kan återställas till valfri tidpunkt (Point-in-Time Restore), med RPO ner till cirka 15 minuter. Azure Backup passar bra när du vill ersätta eller komplettera egna SQL-backupjobb och få bättre överblick, övervakning och efterlevnad i molnet.

- A comprehensive, enterprise backup solution that provides long-term data retention, automated management and additional data protection.
- Offers more than simply performing your own backups, or using the Azure resource provider for SQL Server, but does offer a more complete backup feature set.
- Requires an agent to be installed on the virtual machine to manage automatic backups of your SQL Server databases.
- Central location to manage and monitor the backups and to help meet any specified RPO/RTO metrics

Azure Site Recovery (ASR)

- Low-cost solution that performs block level replication of your Azure virtual machine
- Better suited for migrations with some allowed downtime
- Use with Availability Groups to provide a lower RPO



1. VM is registered with Azure Site Recovery
2. Data is continuously replicated to cache
3. Cache is replicated to the target storage account
4. During failover the virtual machine is added to the target environment

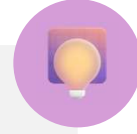
© Copyright Microsoft Corporation. All rights reserved.

Bilden visar **Azure Site Recovery (ASR)**, som är en tjänst för **katastrofåterställning och migrering** av virtuella maskiner. ASR replikerar data på **blocknivå** från en käll-VM till en annan Azure-region via ett cache-lager och ett lagringskonto. Replikeringen sker kontinuerligt och vid ett avbrott kan du starta VM:n i målregionen genom en **failover**.

ASR är relativt **kostnadseffektivt** och passar bra när viss **nedtid är acceptabel**, till exempel vid planerade migreringar eller som DR-lösning. Genom att kombinera ASR med Availability Groups kan man nå lägre RPO. ASR fokuserar på hela VM:n, inte enbart SQL Server.

- Low-cost solution that performs block level replication of your Azure virtual machine.
- Can test and verify your disaster recovery strategy.
- Best utilized for **stateless** environments (e.g. web servers) versus transactional database virtual machines due to its relatively high recovery point objective (up to an hour).
- Better suited for migrations with some allowed downtime.
- Use with Availability Groups to provide a lower RPO.

Knowledge check



What type of storage offers the lowest latency in Azure?

- ☒ Ultra Disk
- ☐ Premium Disk
- ☐ Standard SSD



Which option should you choose to spread workloads across data centers in a region?

- ☐ Availability sets
- ☒ Availability zones
- ☐ Availability units



To reduce the cost of an Azure SQL Server VM you intend to run full time for 3 years, which option should you choose?

- ☒ Azure Reserved VM Instances
- ☐ Availability Set
- ☐ Pay as Your Go Licensing

© Copyright Microsoft Corporation. All rights reserved.

1) What type of storage offers the lowest latency in Azure?

Ultra Disk (RÄTT): Lägst latency och högst IOPS; för krävande databaser.

Premium Disk (FEL): Snabbt SSD-alternativ, men inte lägst latency.

Standard SSD (FEL): Billigare, högre latency och lägre prestanda.

2) Which option should you choose to spread workloads across data centers in a region?

Availability sets (FEL): Sprider inom samma datacenter; skyddar mot host-fel.

Availability zones (RÄTT): Sprider över flera datacenter i samma region.

Availability units (FEL): Ingen vanlig Azure-tjänst/term för detta ändamål.

3) To reduce the cost of an Azure SQL Server VM full time for 3 years

Azure Reserved VM Instances (RÄTT): Förskotts-/bindningsrabatt för 1–3 år, sänker kostnad.

Availability Set (FEL): Handlar om tillgänglighet, inte prisrabatt.

Pay as You Go Licensing (FEL): Flexibelt men dyrast vid konstant drift länge.

Instructor led labs: Provision a SQL Server on an Azure Virtual Machine

Deploy a SQL Server on an Azure Virtual Machine
Connect to SQL Server on an Azure Virtual Machine

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/01-provision-sql-vm.html>

Deploy PaaS solutions with Azure SQL

<https://learn.microsoft.com/training/modules/deploy-paas-solutions-with-azure-sql/>

© Copyright Microsoft Corporation. All rights reserved.

The Platform as a Service (PaaS) offering for SQL Server can be a great solution for certain workloads. The PaaS offering provides less granular control over the infrastructure and relegates management of the underlying components (memory, CPU, storage, operating system, etc) to the cloud provider, namely Microsoft. This unit will focus on ways to provision and deploy both Azure SQL Database and SQL Server Managed Instances, as well as provide clear and definitive guidance on the various options when performing a migration to this platform.

Objectives

-  Gain an understanding of SQL Server in a Platform as a Service (PaaS) offering
-  Understand PaaS provisioning and deployment options
-  Understand elastic pools
-  Examine Managed Instances
-  Configure a template for PaaS deployment

© Copyright Microsoft Corporation. All rights reserved.

The Platform as a Service (PaaS) offering for SQL Server can be a great solution for certain workloads. The PaaS offering provides less granular control over the infrastructure and relegates management of the underlying components (memory, CPU, storage, operating system, etc) to the cloud provider, namely Microsoft.

This module will focus on ways to provision and deploy both Azure SQL Database and SQL Server Managed Instances, as well as provide clear and definitive guidance on the various options when performing a migration to this platform.

Azure SQL Database service tier options

Service tier options – DTU

The original model which uses a formula using memory, storage, and IO resources to assign a service tier

Three different service tiers:

- **Basic**
- **Standard**
- **Premium**

Service tier options – Vcore

Allows you to choose a number of virtual CPUs, which have a fixed relationship to memory and storage provided by the database

Three different service tiers:

- **General Purpose**
- **Hyperscale**
- **Business Critical**

© Copyright Microsoft Corporation. All rights reserved.

Här är en **enkel och sammanhängande förklaring på ca 200 ord**:

Azure SQL Database erbjuder två huvudsakliga modeller för att välja prestanda och kapacitet: **DTU-modellen** och **vCore-modellen**.

DTU (Database Transaction Unit) är den ursprungliga och mest förenklade modellen. Här paketeras CPU, minne, lagring och IO i ett gemensamt mått – DTU. Du väljer helt enkelt en nivå och Azure bestämmer hur resurserna fördelas bakom kulisserna. DTU-modellen passar bra för enklare applikationer där kraven är förutsägbara och där man vill ha ett snabbt och okomplicerat val. De tre nivåerna är **Basic** (enkla, små databaser),

Standard (vanliga affärsapplikationer) och **Premium** (krävande arbetslaster med låg latency).

vCore-modellen ger betydligt mer kontroll och transparens. Här väljer du själv antal virtuella CPU:er, mängd minne och lagring, vilket gör det lättare att dimensionera och jämföra med SQL Server on-premises eller i VM.

Den är också bättre anpassad för licensoptimering och kostnadskontroll. vCore-modellen har tre tjänstenivåer: **General Purpose** för de flesta arbetslaster, **Business Critical** för mycket höga prestanda och låg latency, samt **Hyperscale**, som är optimerad för extremt stora databaser med snabb och flexibel skalning.

Sammanfattningsvis: DTU är enkelt men grovt, vCore är flexibelt och mer professionellt.

Database Transaction Unit (DTU)

DTUs are a computed based on a formula over compute, storage and IO resources.

PaaS comes in several different service tiers. Each tier has varying capabilities which allow you to have a wide range of options when choosing this platform.

The DTU model is available in three different service tiers:

- **Basic** is best used for lightweight dev and test workloads
- **Standard** is best used for a wide variety of production applications
- **Premium** is best suited for workloads that need extremely low latency and have high transactional volume

Maximum storage size & DTUs

- Maximum storage size:
Basic: 2 GB
Standard: 1 TB
Premium: 4 TB
- Maximum DTUs:
Basic: 5
Standard: 3000
Premium: 4000

vCore

Allows customers to specified specific number of vCores based on their workload. It allows for better scalability and flexibility. vCore is the new default purchasing model based out of DTU vs vCore.

General purpose, which is designed for budget-oriented workloads.

- Offers balanced compute and storage suited for most workloads
- Similar feature set to business critical

Hyperscale, which is designed for most business workloads, providing highly scalable storage, read scale-out, and fast database restore capabilities.

- Highly scalable storage
- Read scale-out
- Single SQL Database

Business critical, which is designed for low-latency workloads with high resiliency to failures and fast failovers.

- Local SSD storage provides very low storage Low-latency workloads
- Allows for readable secondary databases and in-memory OLTP

GP: 5-10ms latency

Hyperscale – High IOPS, data 1-2ms latency; log 5-10ms latency

Business Critical – High IOPS, 1-2ms latency

Once you switch to Hyperscale you can't switch out of it.

Every 100 DTUs in the standard tier require at least 1 vCore in the General Purpose service tier.

Every 125 DTUs in the premium tier require at least 1 vCore in the Business Critical service tier.

Deploy single database via Azure portal

Create SQL Database

Microsoft

Create a SQL database with your preferred configurations. Complete the Basics tab then go to Review + Create to provision with smart defaults, or visit each tab to customize. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group * [Create new](#)

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name * ✓

Server [Create new](#)

Want to use SQL elastic pool? * ☐ Yes ☒ No

Compute + storage * [Configure database](#)

Gen5, 2 vCores, 32 GB storage

[Review + create](#) [Next: Networking >](#)

© Copyright Microsoft Corporation. All rights reserved.

The image below shows the portal blade in which you can configure the database options. Here you will notice that the service tier is General Purpose and compute tier is Provisioned. Provisioned implies that the compute resources are pre-allocated and billed per hour based on the number of configured vCores.

Deploy single database via PowerShell/Azure CLI

PowerShell:

```
# Create a server firewall rule that allows access from the specified IP range

$serverFirewallRule = New-AzSqlServerFirewallRule -ResourceGroupName $resourceGroupName `
  -ServerName $serverName `
  -FirewallRuleName "AllowedIPs" -StartIpAddress $startIp -EndIpAddress $endIp

# Create a blank database with an S0 performance level

$database = New-AzSqlDatabase -ResourceGroupName $resourceGroupName `
  -ServerName $serverName `
  -DatabaseName $databaseName `
  -RequestedServiceObjectiveName "S0" `
  -SampleName "AdventureWorksLT"
```

Azure CLI:

```
# Configure a firewall rule for the server

az sql server firewall-rule create \
  --resource-group $resourceGroupName \
  --server $serverName \
  --n AllowYourIp \
  --start-ip-address $startIp \
  --end-ip-address $endip

# Create a database in the server

az sql db create \
  --resource-group $resourceGroupName \
  --server $serverName \
  --name mySampleDatabase \
  --sample-name AdventureWorksLT \
  --edition GeneralPurpose \
  --family Gen4 \
  --capacity 1 \
```

© Copyright Microsoft Corporation. All rights reserved.

You can also deploy your database using Azure PowerShell or the Azure CLI.

Deploy single database via Azure Resource Manager templates

```
#Define Variables for parameters to pass to template
$projectName = Read-Host -Prompt "Enter a project name"
$location = Read-Host -Prompt "Enter an Azure location (i.e. centralus)"
$adminUser = Read-Host -Prompt "Enter the SQL server administrator username"
$adminPassword = Read-Host -Prompt "Enter the SQL server administrator password" -AsSecureString
$resourceGroupName = "${projectName}rg"

#Create Resource Group and Deploy Template to Resource Group
New-AzResourceGroup -Name $resourceGroupName -Location $location

New-AzResourceGroupDeployment -ResourceGroupName $resourceGroupName `
-TemplateUri "https://raw.githubusercontent.com/Azure/azure-quickstart-templates/master/101-sql-logical-server/azuredeploy.json"
-administratorLogin $adminUser -administratorLoginPassword $adminPassword

Read-Host -Prompt "Press [ENTER] to continue ..."
```

<https://raw.githubusercontent.com/Azure/azure-quickstart-templates/master/101-sql-logical-server/azuredeploy.json>

© Copyright Microsoft Corporation. All rights reserved.

Another method for deploying resources is as mentioned earlier using an Azure Resource Manager template. A Resource Manager template gives you the most granular control over your resources, and Microsoft provides a GitHub repository called "Azure-Quickstart-Templates", which hosts Azure Resource Manager templates that you can reference in your deployments. A PowerShell example of deploying a GitHub based template is shown above.

Demo: Explore single SQL Database and elastic pool

Explore single SQL Database
and elastic pool

© Copyright Microsoft Corporation. All rights reserved.

Navigate through Azure portal and the steps on how to create a single SQL database versus SQL Database elastic pool

Azure SQL Database backups

Backup schedule:

- A weekly full backup
- A differential backup every 12 hours
- A transaction log backup every 5-10 minutes based on log utilization

Retention:

- **vCore and Basic Databases:** 7-day default retention, that can extend to 35 days
- **Standard and Premium Databases:** 35-day retention period
- **Long Term Retention (LTR):** Keep backups up to 10 years on Azure blob storage

© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Database har **inbyggd och automatiserad backup**, vilket innebär att du inte behöver hantera backupjobb själv. Systemet tar en **fullständig backup varje vecka**, kompletterat med **differentiella backupar var 12:e timme**. Dessutom tas **transaktionsloggbackupar var 5–10 minut**, vilket möjliggör återställning till valfri tidpunkt inom retentionstiden (point-in-time restore).

Retentionen beror på tjänstemodell. **vCore- och Basic-databaser** har som standard **7 dagars retention**, som kan utökas till **35 dagar**. **Standard- och Premium-databaser** har alltid **35 dagars retention**. För långsiktiga krav finns **Long Term Retention (LTR)**, där backupar kan sparas upp till **10 år i Azure Blob Storage**, exempelvis för regelefterlevnad och revision.

Backups are performed automatically by the service

Backups are stored in geo-redundant storage accounts

Restore an existing database to a point-in-time in the past within the retention period using the Azure portal, Azure PowerShell, Azure CLI, or REST API. In Single database and Elastic pools, this operation will create a new database in the same server as the original database. In Managed Instance, this operation can create a copy of the database or same or different Managed Instance under the same subscription.

Restore a deleted database to the time it was deleted or anytime within the retention period. The deleted database can only be restored in the same logical server or Managed Instance where the original database was created.

Restore a database to another geographical region. Geo-restore allows you to recover from a geographic disaster when you cannot access your server and database. It creates a new database in any existing server anywhere in the world.

Restore a database from a specific long-term backup on Single Database or Elastic Pool if the database has been configured with a long-term retention policy (LTR). LTR allows you to restore an old version of the database using [the Azure portal](#) or [Azure PowerShell](#) to satisfy a compliance request or to run an old version of the application. For more information, see [Long-term retention](#).

Backups are specified and managed from the virtual SQL Server level and NOT at the database level.

Restoring an Azure SQL Database

- Cannot manually restore a database utilizing the T-SQL command RESTORE DATABASE.
- Not possible to restore over an existing database.
- Existing database must be dropped or renamed prior to initiating the restore.

Restore options:

Azure portal

PowerShell or Azure CLI

© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Database har ett **annorlunda återställningssätt än traditionell SQL Server**. Du kan **inte använda T-SQL-kommandot RESTORE DATABASE**, eftersom Microsoft hanterar lagringen och backuperna bakom kulisserna. En återställning kan **inte ske ovanpå en befintlig databas**. I stället skapas alltid en **ny databas** vid restore. Vill du återställa till samma namn måste den befintliga databasen först **tas bort eller döpas om**.

Återställning görs via **Azure Portal**, **PowerShell** eller **Azure CLI**. Vanligast är så kallad **point-in-time restore**, där du väljer ett exakt klockslag inom retentionstiden. Detta gör Azure SQL Database mycket säkert och enkelt, men med mindre manuell kontroll än klassisk SQL Server.

Azure portal – Can restore a database to the same Azure SQL Database server or create a new database on a new server in any Azure region.

Restore an existing database to a point-in-time in the past within the retention period using the Azure portal, Azure PowerShell, Azure CLI, or REST API. In Single database and Elastic pools, this operation will create a new database in the same server as the original database. In Managed Instance, this operation can create a copy of the database or same or different Managed Instance under the same subscription.

Restore a deleted database to the time it was deleted or anytime within the retention period. The deleted database can only be restored in the same logical server or Managed Instance where the original database was created.

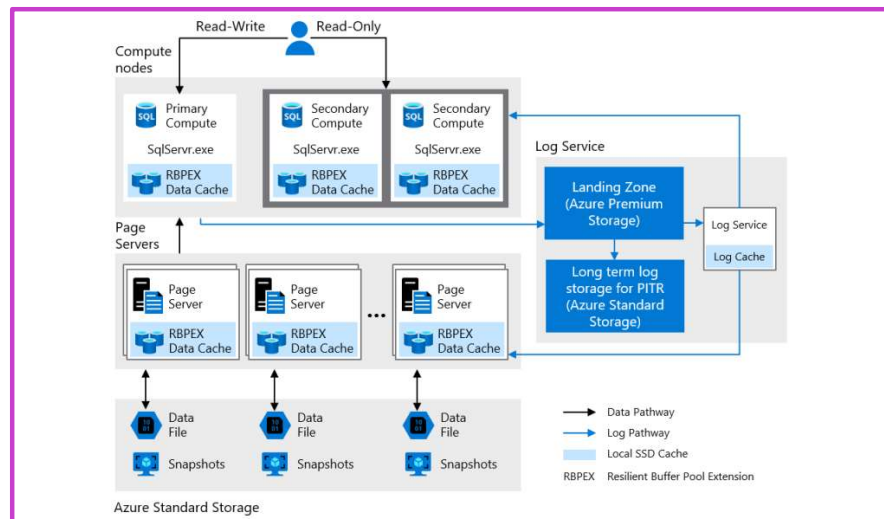
Restore a database to another geographical region. Geo-restore allows you to recover from a geographic disaster when you cannot access your server and database. It creates a new database in any existing server anywhere in the world.

Restore a database from a specific long-term backup on Single Database or Elastic Pool if the database has been configured with a long-term retention policy (LTR). LTR allows you to restore an old version of the database using [the Azure portal](#) or [Azure PowerShell](#) to satisfy a compliance request or to run an old version of the application. For more information, see [Long-term retention](#).

Backups are specified and managed from the virtual SQL Server level and NOT at the database level.

Hyperscale SQL Database

- Designed for very large OLTP databases
- Able to auto-scale independently storage and compute with no limits
- Restores in minutes rather than hours and days
- Scale up or down in real time to accommodate workload changes



© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Database **Hyperscale** är byggd för **mycket stora OLTP-databaser** och höga krav på skalbarhet. Till skillnad från traditionella tjänstnivåer separerar Hyperscale **compute och storage helt**, vilket gör att lagring kan växa till **tiotals terabyte** utan att påverka prestandan nämnvärt. Compute-noder kan skalas upp eller ned **i realtid**, och flera **read-only repliker** kan användas för rapportering och läsintensiva arbetslaster.

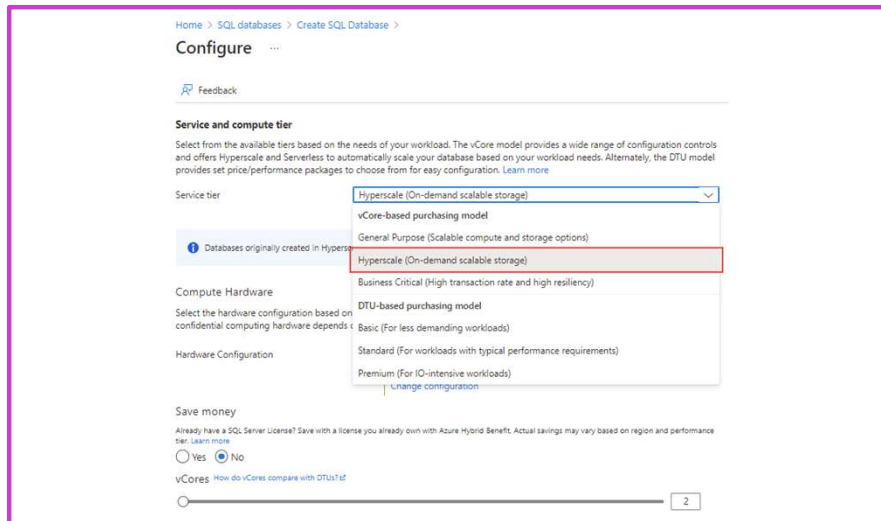
Data lagras i distribuerade **page servers** medan loggar hanteras i en separat loggtjänst. Tack vare denna arkitektur kan **backup och restore ske på minuter**, även för mycket stora databaser. Hyperscale passar särskilt bra för applikationer med snabbt växande datamängder, ojämn belastning och krav på snabba återställningar.

- Designed for very large OLTP databases – as large as 100 TB - billed only for the capacity you use
- Other databases are limited by the resources available in a single node, databases in the Hyperscale service tier have no such limits
- For read-intensive workloads, Hyperscale provides rapid scale-out by provisioning extra replicas as needed for offloading read workloads.
- Time required to create database backups or to scale up or down is no longer tied to the volume of data in the database.
 - Hyperscale databases can be backed up instantaneously.
 - Scale a database in the tens of terabytes up or down in minutes
- Rapid scalability based on your workload demand.
 - Scaling Up/Down
 - Scaling In/Out – compute replicas as read-only replicas to offload your read workload from the primary compute and serve as hot-standbys to failover from the primary.

- Provisioning of each of these extra compute replicas can be done in constant time and is an online operation.

Hyperscale use cases

- Modernize by moving to the cloud
- Expand the potential for database growth
- Workloads that need both high performance and high scalability



© Copyright Microsoft Corporation. All rights reserved.

Hyperscale är särskilt lämpad för **use cases där traditionella databasserver-begränsningar blir ett hinder**. Ett vanligt scenario är **modernisering av befintliga SQL Server-lösningar** genom flytt till Azure, där man vill behålla SQL-funktionalitet men få molnets skalbarhet.

Hyperscale passar också när databasen förväntas **växa kraftigt över tid**, till exempel i SaaS-lösningar, logg- och händelsedata eller snabbt växande affärssystem.

Tjänsten är idealisk för **arbetslaster som kräver både hög prestanda och mycket hög skalbarhet**, exempelvis transaktionsintensiva system med varierande belastning. Möjligheten att skala compute och storage oberoende, samt använda read-only repliker, gör Hyperscale väl lämpad för applikationer med kombinerade OLTP- och rapporteringsbehov.

Hyperscale performance considerations



Nearly instantaneous database backups (based on file snapshots stored in Azure Blob storage) regardless of size with no IO effect on compute resources



Fast database restores – Minutes rather than hours or days (not a size of data operation)



Rapid scale out – Provision one or more read-only replicas for offloading read workload and for use as hot-standbys



Rapid scale up – Scale up your compute resources to accommodate heavy workloads when needed, and then scale the compute resources back down when not needed

© Copyright Microsoft Corporation. All rights reserved.

Hyperscale är optimerat för **extrem prestanda och snabb skalning**, även för mycket stora databaser. Backup tas nästan **omedelbart** med hjälp av **filbaserade snapshots i Azure Blob Storage**, utan märkbar IO-belastning på compute-resurserna. Det innebär att backupfönster och påverkan på drift i praktiken försvinner.

Återställningar är också mycket snabba – oftast **minuter istället för timmar eller dagar**, oberoende av databasen storlek. Hyperscale stödjer **snabb scale-out** genom att lägga till en eller flera **read-only repliker**, som kan användas för rapportering, läsintensiva frågor eller som hot-standby.

Samtidigt kan compute-resurser **skalas upp och ned snabbt** för att möta tillfälliga toppar i belastning och därefter minska kostnader när belastningen sjunker.

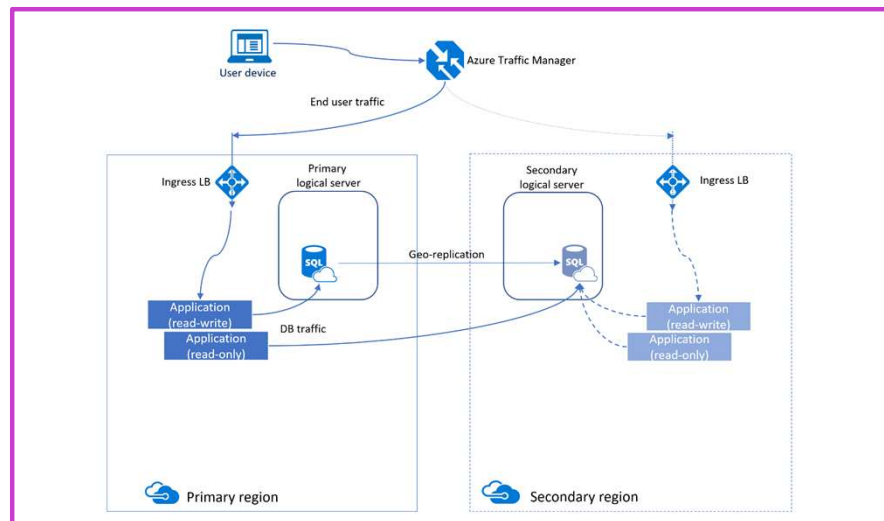
The Hyperscale service tier is intended for customers who have large on-premises SQL Server databases and want to modernize their applications by moving to the cloud, or for customers who are already using Azure SQL Database and want to significantly expand the potential for database growth. Hyperscale is also intended for customers who seek both high performance and high scalability.

SQL Database Hyperscale does not support the following features:

- SQL Managed Instance
- Elastic Pools
- Geo-replication
- Query Performance Insights

Active geo-replication

- Provides automatic **asynchronous** replication of committed transactions on primary database to secondary database in the Azure using **snapshot isolation**
- Provides a **readable secondary copy** of your database, no matter your service tier



© Copyright Microsoft Corporation. All rights reserved.

Active geo-replication i Azure SQL Database används för att skapa **geografiskt separerade kopior av en databas** för hög tillgänglighet och katastrofåterställning. Lösningen bygger på **asynkron replikering**, där alla bekräftade transaktioner på primärdatabasen skickas vidare till en eller flera sekundära databaser i andra Azure-regioner. Replikeringen sker med **snapshot isolation**, vilket ger konsekvent läsning utan att blockera transaktioner.

De sekundära databaserna är **läsbara**, vilket gör att de kan användas för rapportering, analyser eller andra läsintensiva arbetslaster. Vid ett större fel eller regionavbrott kan en sekundär databas snabbt **promoterats till primär**, antingen manuellt eller automatiserat via till exempel Azure Traffic Manager.

Active geo-replication fungerar oberoende av tjänstenivå och ger ett effektivt sätt att kombinera **disaster recovery, lastavlastning och global åtkomst**, med låg administrationsbörda och hög flexibilitet.

- You can have up to 4 secondary copies.
- Failover requires connection string updates.
- Failover can be controlled by the application or a manual process via a user.
- Utilizes availability group technologies under the hood.

Grace period is utilized to ensure transactions are transmitted to the secondary prior to a failover.

Active geo-replication leverages the [Always On](#) technology of SQL Server to asynchronously replicate committed transactions on the primary database to a secondary database using snapshot isolation. Auto-failover groups provide the group semantics on top of active geo-replication but the same asynchronous replication mechanism is used. While at any given point, the secondary database might be slightly behind the primary database, the secondary data is guaranteed to never have partial transactions. Cross-region redundancy enables applications to quickly recover from a permanent loss of an entire datacenter or parts of a datacenter caused by natural disasters, catastrophic human errors, or malicious acts.

To guarantee that a critical change on the primary database is replicated to a secondary before you failover, you

can force synchronization to ensure the replication of critical changes (for example, password updates). Forced synchronization impacts performance because it blocks the calling thread until all committed transactions are replicated. For details, see [sp_wait_for_database_copy_sync](#). To monitor the replication lag between the primary database and geo-secondary, see [sys.dm_geo_replication_link_status](#).

Planned failover

Planned failover switches the roles of primary and secondary databases after the full synchronization is completed. It is an online operation that does not result in data loss. The time of the operation depends on the size of the transaction log on the primary that needs to be synchronized. Planned failover is designed for following scenarios: (a) to perform DR drills in production when the data loss is not acceptable; (b) to relocate the database to a different region; and (c) to return the database to the primary region after the outage has been mitigated (failback).

Unplanned failover

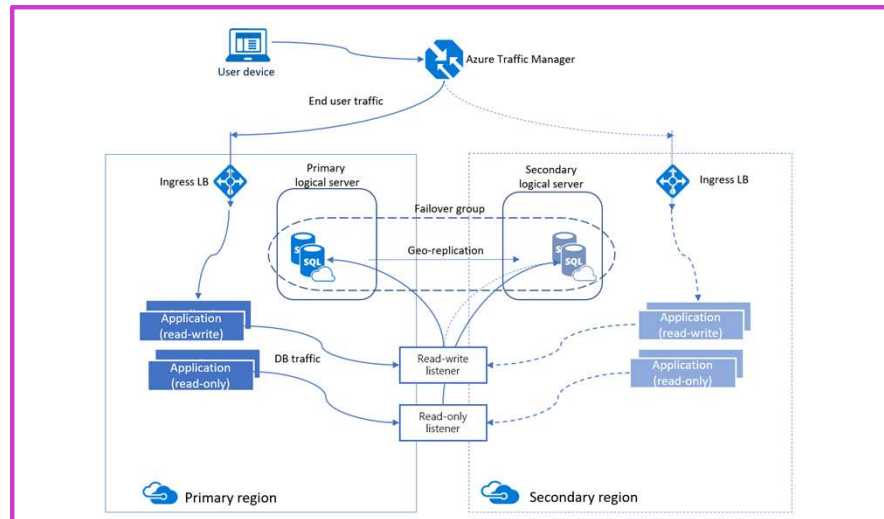
Unplanned or forced failover immediately switches the secondary to the primary role without any synchronization with the primary. Any transactions committed to the primary but not replicated to the secondary will be lost. This operation is designed as a recovery method during outages when the primary is not accessible, but the database availability must be quickly restored. When the original primary is back online it will automatically re-connect and become a new secondary. All unsynchronized transactions before the failover will be preserved in the backup file but will not be synchronized with the new primary to avoid conflicts. These transactions will have to be manually merged with the most recent version of the primary database.

Multiple readable secondaries

Up to 4 secondary databases can be created for each primary. If there is only one secondary database, and it fails, the application is exposed to higher risk until a new secondary database is created. If multiple secondary databases exist, the application remains protected even if one of the secondary databases fails. The additional secondaries can also be used to scale out the read-only workloads.

Failover groups

- Builds on the geo-replication feature
- Provides for automatic failover
- Allows a single connection string to connect to the group
- Multiple databases can be in the same failover group



© Copyright Microsoft Corporation. All rights reserved.

Failover Groups i Azure SQL Database bygger vidare på **Active Geo-replication**, men tillför **automation, enkelhet och applikationsvänlighet**. Med Active Geo-replication skapar du manuellt en eller flera sekundära databaser i andra regioner. Du ansvarar själv för **failover**, för att uppdatera **connection strings** och för att samordna flera databaser. Det fungerar bra tekniskt, men kräver mer logik i applikationen och mer driftarbete.

Failover Groups löser detta genom att **gruppera flera databaser** (och även elastic pools) i en logisk enhet med **automatisk failover**. Gruppen tillhandahåller två stabila DNS-namn: ett för **read-write** och ett för **read-only** trafik. Vid ett regionavbrott flyttas primärrollen automatiskt till den sekundära regionen, och applikationen fortsätter fungera utan att connection strings behöver ändras.

En annan viktig skillnad är **samordning**: i en Failover Group sker failover för alla databaser i gruppen samtidigt, vilket är avgörande för applikationer som använder flera databaser med beroenden mellan sig.

Kort sagt:

Active Geo-replication = byggblock, manuell kontroll

Failover Groups = färdig DR-lösning med automation, enklare drift och bättre stöd för verkliga applikationer

Allows a single connection string to connect to the group (no connection string changes needed)

Disaster recovery for SQL MI is slightly different, allowing only one replica in the paired Azure region to be primary

Like geo-replication, failover groups continues to utilize the availability group technology under the hood.

Utilizes sister regions and you are unable to select which sister region is applicable to your primary region.

Failover group (FOG)

A failover group is a named group of databases managed by a single SQL Database server or within a single managed instance that can fail over as a unit to another region in case all or some primary databases become unavailable due to an outage in the primary region. When created for managed instances, a failover group contains all user databases in the instance and therefore only one failover group can be configured on an instance.

Important

The name of the failover group must be globally unique within the .database.windows.net domain.

SQL Database servers

With SQL Database servers, some or all of the user databases on a single SQL Database server can be placed in a failover group. Also, a SQL Database server supports multiple failover groups on a single SQL Database server.

Primary

The SQL Database server or managed instance that hosts the primary databases in the failover group.

Secondary

The SQL Database server or managed instance that hosts the secondary databases in the failover group. The secondary cannot be in the same region as the primary.

Adding single databases to failover group

You can put several single databases on the same SQL Database server into the same failover group. If you add a single database to the failover group, it automatically creates a secondary database using the same edition and compute size on secondary server. You specified that server when the failover group was created. If you add a database that already has a secondary database in the secondary server, that geo-replication link is inherited by the group. When you add a database that already has a secondary database in a server that is not part of the failover group, a new secondary is created in the secondary server.

Important

Make sure that the secondary server doesn't have a database with the same name unless it is an existing secondary database. In failover groups for managed instance all user databases are replicated. You cannot pick a subset of user databases for replication in the failover group.

Adding databases in elastic pool to failover group

You can put all or several databases within an elastic pool into the same failover group. If the primary database is in an elastic pool, the secondary is automatically created in the elastic pool with the same name (secondary pool). You must ensure that the secondary server contains an elastic pool with the same exact name and enough free capacity to host the secondary databases that will be created by the failover group. If you add a database in the pool that already has a secondary database in the secondary pool, that geo-replication link is inherited by the group. When you add a database that already has a secondary database in a server that is not part of the failover group, a new secondary is created in the secondary pool.

Initial Seeding

When adding databases, elastic pools, or managed instances to a failover group, there is an initial seeding phase before data replication starts. The initial seeding phase is the longest and most expensive operation. Once initial seeding completes, data is synchronized, and then only subsequent data changes are replicated. The time it takes for the initial seed to complete depends on the size of your data, number of replicated databases, and the speed of the link between the entities in the failover group. Under normal circumstances, typical seeding speed is 50-500 GB an hour for a single database or elastic pool, and 18-35 GB an hour for a managed instance. Seeding is performed for all databases in parallel. You can use the stated seeding speed, along with the number of databases and the total size of data to estimate how long the initial seeding phase will take before data replication starts.

For managed instances, the speed of the Express Route link between the two instances also needs to be considered when estimating the time of the initial seeding phase. If the speed of the link between the two instances is slower than what is necessary, the time to seed is likely to be notably impacted. You can use the stated seeding speed, number of databases, total size of data, and the link speed to estimate how long the initial seeding phase will take before data replication starts. For example, for a single 100 GB database, the initial seed phase would take anywhere from 2.8 - 5.5 hours if the link is capable of pushing 35 GB per hour. If the link can

only transfer 10 GB per hour, then seeding a 100 GB database will take about 10 hours. If there are multiple databases to replicate, seeding will be executed in parallel, and, when combined with a slow link speed, the initial seeding phase may take considerably longer, especially if the parallel seeding of data from all databases exceeds the available link bandwidth. If the network bandwidth between two instances is limited and you are adding multiple managed instances to a failover group, consider adding multiple managed instances to the failover group sequentially, one by one.

DNS zone

A unique ID that is automatically generated when a new instance is created. A multi-domain (SAN) certificate for this instance is provisioned to authenticate the client connections to any instance in the same DNS zone. The two managed instances in the same failover group must share the DNS zone.

Note

A DNS zone ID is not required for failover groups created for SQL Database servers.

Failover group read-write listener

A DNS CNAME record that points to the current primary's URL. It is created automatically when the failover group is created and allows the read-write SQL workload to transparently reconnect to the primary database when the primary changes after failover. When the failover group is created on a SQL Database server, the DNS CNAME record for the listener URL is formed as <fog-name>.database.windows.net. When the failover group is created on a managed instance, the DNS CNAME record for the listener URL is formed as <fog-name>.zone_id.database.windows.net.

Failover group read-only listener

A DNS CNAME record formed that points to the read-only listener that points to the secondary's URL. It is created automatically when the failover group is created and allows the read-only SQL workload to transparently connect to the secondary using the specified load-balancing rules. When the failover group is created on a SQL Database server, the DNS CNAME record for the listener URL is formed as <fog-name>.secondary.database.windows.net. When the failover group is created on a managed instance, the DNS CNAME record for the listener URL is formed as <fog-name>.zone_id.secondary.database.windows.net.

Automatic failover policy

By default, a failover group is configured with an automatic failover policy. The SQL Database service triggers failover after the failure is detected and the grace period has expired. The system must verify that the outage cannot be mitigated by the built-in [high availability infrastructure of the SQL Database service](#) due to the scale of the impact. If you want to control the failover workflow from the application, you can turn off automatic failover.

Note

Because verification of the scale of the outage and how quickly it can be mitigated involves human actions by the operations team, the grace period cannot be set below one hour. This limitation applies to all databases in the failover group regardless of their data synchronization state.

Read-only failover policy

By default, the failover of the read-only listener is disabled. It ensures that the performance of the primary is not impacted when the secondary is offline. However, it also means the read-only sessions will not be able to connect until the secondary is recovered. If you cannot tolerate downtime for the read-only sessions and are OK to temporarily use the primary for both read-only and read-write traffic at the expense of the potential performance degradation of the primary, you can enable failover for the read-only listener by configuring the AllowReadOnlyFailoverToPrimary property. In that case, the read-only traffic will be automatically redirected to the primary if the secondary is not available.

Planned failover

Planned failover performs full synchronization between primary and secondary databases before the secondary switches to the primary role. This guarantees no data loss. Planned failover is used in the following scenarios:

- Perform disaster recovery (DR) drills in production when the data loss is not acceptable
- Relocate the databases to a different region
- Return the databases to the primary region after the outage has been mitigated (failback).

Unplanned failover

Unplanned or forced failover immediately switches the secondary to the primary role without any synchronization with the primary. This operation will result in data loss. Unplanned failover is used as a recovery method during outages when the primary is not accessible. When the original primary is back online, it will automatically reconnect without synchronization and become a new secondary.

Manual failover

You can initiate failover manually at any time regardless of the automatic failover configuration. If automatic failover policy is not configured, manual failover is required to recover databases in the failover group to the secondary. You can initiate forced or friendly failover (with full data synchronization). The latter could be used to relocate the primary to the secondary region. When failover is completed, the DNS records are automatically updated to ensure connectivity to the new primary.

Grace period with data loss

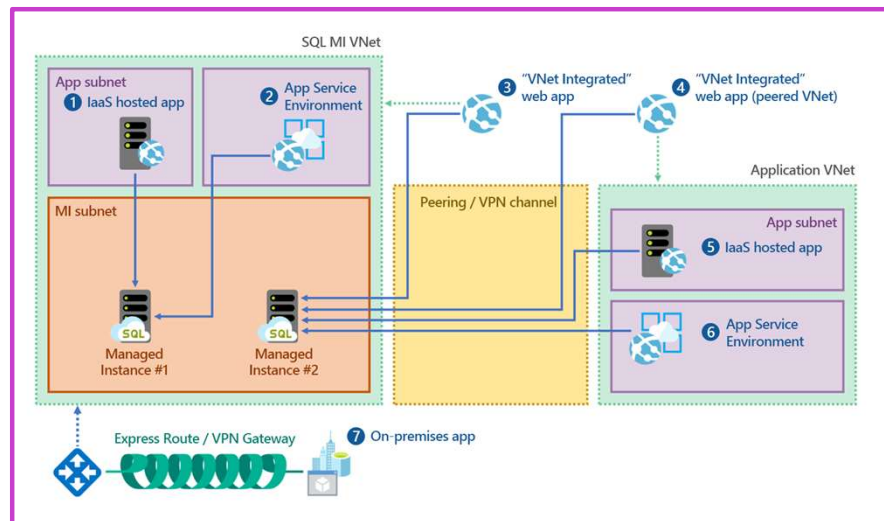
Because the primary and secondary databases are synchronized using asynchronous replication, the failover may result in data loss. You can customize the automatic failover policy to reflect your application's tolerance to data loss. By configuring `GracePeriodWithDataLossHours`, you can control how long the system waits before initiating the failover that is likely to result data loss.

Multiple failover groups

You can configure multiple failover groups for the same pair of servers to control the scale of failovers. Each group fails over independently. If your multi-tenant application uses elastic pools, you can use this capability to mix primary and secondary databases in each pool. This way you can reduce the impact of an outage to only half of the tenants.

Azure managed networking

- SQL Managed Instance is deployed into a virtual cluster in a Virtual Network
- Connections must come from its virtual network or a peered network



© Copyright Microsoft Corporation. All rights reserved.

Azure Managed Networking för **SQL Managed Instance (SQL MI)** innebär att databasen körs **helt inuti ett Azure Virtual Network (VNet)** som Microsoft hanterar. SQL MI placeras i ett **virtuellt kluster** och exponeras **inte publikt mot internet**. All nätverkstrafik måste komma från samma VNet, ett **peered VNet**, eller via **VPN/ExpressRoute** från on-premises. Det ger hög säkerhet och gör SQL MI mer lik en traditionell on-prem SQL Server-miljö.

I bilden visas hur olika typer av applikationer kan ansluta: IaaS-VM:ar och App Services som är **VNet-integrerade**, både i samma VNet och i peered VNets. Även on-premises-applikationer kan nå SQL MI via VPN eller ExpressRoute. Flera Managed Instances kan ligga i samma MI-subnet och dela nätverksinfrastruktur.

Sammanfattningsvis: Azure Managed Networking ger **isolerad, privat och kontrollerad åtkomst**, med full nätverkskontroll, hög säkerhet och stöd för hybrida scenarier – utan att du själv behöver bygga eller underhålla SQL-servers nätverksarkitektur.

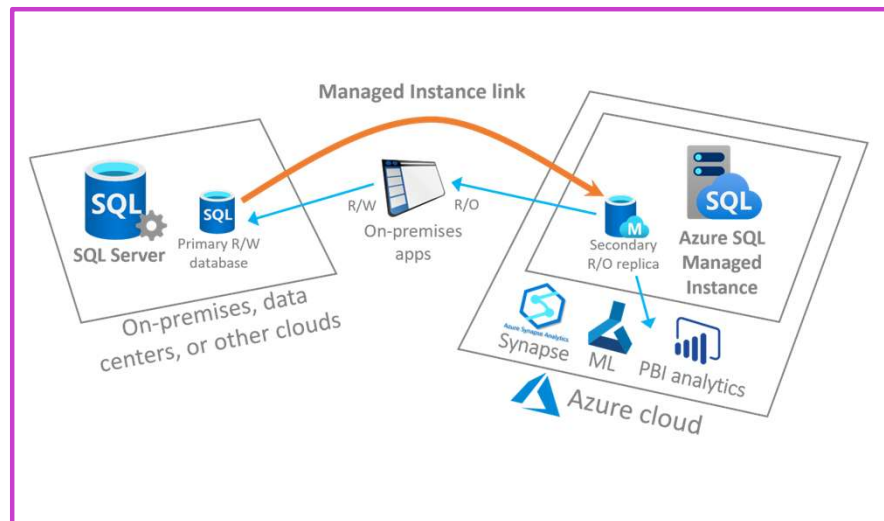
By default, it does not have a public endpoint.

Azure SQL Managed Instance link

Connects your SQL Servers hosted anywhere to SQL Managed Instance, providing hybrid flexibility and database mobility

Supported scenarios:

- Use Azure services without migrating to the cloud
- Offload read-only workloads to Azure
- Migrate to Azure



© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Managed Instance link gör det möjligt att koppla samman en befintlig **SQL Server** (on-premises, i annan molnleverantör eller i Azure VM) med **Azure SQL Managed Instance** i ett **hybridskenario**. Tekniken bygger på **Always On Availability Groups**, där SQL Server är **primär (read/write)** och Managed Instance fungerar som **sekundär replika (read-only)**.

Detta ger flera viktiga användningsfall. Du kan börja använda **Azure-tjänster** som Power BI, Synapse och Machine Learning **utan att flytta hela databasen till molnet**. Läsintensiva arbetslaster kan **offloadas till Azure**, vilket minskar belastningen på den lokala SQL Servern. Samtidigt får du ett mycket kontrollerat och stegvis sätt att **migrera till Azure**, där Managed Instance redan är synkroniserad och redo att ta över.

Till skillnad från klassisk migrering sker ingen "big bang". Applikationer kan fortsätta skriva till den lokala SQL Servern medan Azure används för analys, rapportering och framtida övergång. När du är redo kan rollerna bytas, och Managed Instance blir primär.

Sammanfattningsvis ger SQL Managed Instance link **hybrid flexibilitet, låg risk och hög datamobilitet**, samtidigt som du drar nytta av Azure's ekosystem och Managed Instance-funktionalitet.

- Supports Azure SQL Managed Instance provisioned on any service tier.
- After a disastrous event, you can continue running your read-only workloads on SQL Managed Instance in Azure.
- Replicate databases from SQL Server instances to Azure SQL Managed Instance. Uses distributed Always On Availability groups. Transaction log records are replicated as part of distributed availability groups.
- Supports Azure SQL Managed Instance provisioned on any service tier.

- For hybrid disaster, fail over SQL Server databases to a database running on SQL Managed Instance.
- For a read-only secondary database to offload intensive read-only operations.

Link feature provides hybrid capability of replicating databases from SQL Server instances to Azure SQL Managed Instance. The link feature replicates data using distributed availability groups available on Always On availability group technology. Transaction log records are replicated as part of distributed availability groups.

The transaction log records on the primary instance can't be truncated until they've been replicated to the secondary instance. Regularly transaction log backups reduce the risk of running out of space on your primary instance.

Link feature can also be used as a hybrid disaster recover solution, where you can fail over your SQL Server databases hosted anywhere to a database running on SQL Managed Instance. Likewise, you can use link feature to provide a read-only secondary database in SQL Database SQL Managed Instance to offload intensive read-only operations.

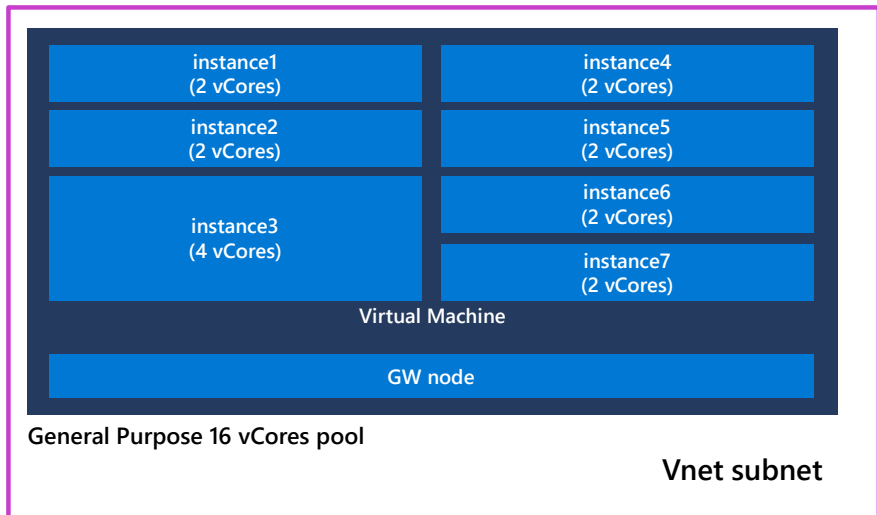
<https://learn.microsoft.com/azure/azure-sql/managed-instance/managed-instance-link-preparation>

Azure SQL Managed Instance – Instance Pools

This feature supports multiple managed instances on the same virtual machine

Benefits

- Ability to host 2-vCore instances.
- Only for instances in the instance pools
- Predictable and fast instance deployment time.
- Minimal IP address allocation



© Copyright Microsoft Corporation. All rights reserved.

flera Managed Instances kan dela på samma underliggande VM

Det är exakt vad **Instance Pools** är till för:

Azure skapar **en kraftigare VM** (t.ex. 16, 32 eller 64 vCores)

Den VM:en **partitioneras logiskt** i flera Managed Instances

Varje Managed Instance får:

- egna vCores
- eget minne
- egna databaser

Men de **delar nätverk, IP-adresser och gateway node**

Tänk så här 🙋

🏠 **VM = hyreshus**

🏠 **Managed Instances = lägenheter**

Azure SQL Managed Instance – Instance Pools gör det möjligt att distribuera **flera Managed Instances på samma virtuella maskin**, där den underliggande VM:ens storlek baseras på det **totala antalet vCores som tilldelas poolen**. Arkitekturen innebär att den virtuella maskinen **partitioneras i flera logiska instanser**, som kan ha olika storlek. Ett viktigt exempel är **2-vCore-instanser**, som **endast är tillgängliga i instance pools** och inte som fristående Managed Instances.

Instance Pools stöds **endast i service tier General Purpose** och är tillgängliga **endast på Gen5-hårdvara**. När poolen väl är skapad går **hanteringsoperationer mycket snabbare**, till exempel skapande eller skalning av instanser. Detta beror på att den underliggande **virtuella klustermiljön redan finns på plats**, vilket eliminerar tidskrävande infrastrukturprovisionering per instans.

Eftersom **alla instanser delar samma virtuella maskin** påverkas inte IP-adressbehovet av hur många instanser som skapas. Varje pool använder **endast nio IP-adresser** (utöver fem reserverade i subnetet), vilket gör Instance Pools särskilt attraktiva i nätverk med **begränsat IP-utrymme**.

Det finns dock tydliga resursbegränsningar. Det totala antalet vCores som används av instanserna får **inte överstiga poolens vCores**. Poolen har också ett tak på **8 TB total lagring** och **100 databaser totalt**, oavsett hur dessa är fördelade mellan instanserna. Pooler kan skapas med **8 till 80 vCores**, och instanser i pooler kan ha mellan **2 och 80 vCores**, med särskilda lagringsgränser för mindre instanser.

Alla instanser i en pool **måste använda samma service tier**, som är knuten till poolresursen. Sammantaget passar Instance Pools mycket bra för **konsolidering av många mindre SQL-miljöer**, SaaS-lösningar och utvecklings- eller testscenarier där **kostnad, snabb provisionering och effektiv resursanvändning** är centrala krav.

En **GW node (Gateway node)** är en **infrastrukturkomponent i Azure SQL Managed Instance** som fungerar som **nätverks- och anslutningsgateway** mellan klienter/applikationer och de Managed Instances som körs i en **VNet-subnet**.

Instance pools enable deployment of multiple instances on the same virtual machine where the virtual machine's compute size is based on the total number of vCores allocated for the pool. This architecture allows *partitioning* of the virtual machine into multiple instances, which can be any supported size, including 2 vCores (2 vCore instances are only available for instances in pools).

Supported by general purpose service tier only

Management operations on instances in a pool are much faster once the pool is initially deployed. These operations are faster because deployment or extension of a [virtual cluster](#) (dedicated set of virtual machines) is not part of provisioning the managed instance.

Because all instances in a pool share the same virtual machine, the total IP allocation does not depend on the number of instances deployed, which is convenient for deployment in subnets with a narrow IP range.

Each pool has a fixed IP allocation of only nine IP addresses (not including the five IP addresses in the subnet that are reserved for its own needs). For details, see [subnet size requirements for single instances](#).

Instance pools resource limitations

There are several resource limitations regarding instance pools and instances inside pools:

Instance pools are available only on Gen5 hardware.

Instances within a pool have dedicated CPU and RAM, so the aggregated number of vCores across all instances must be less than or equal to the number of vCores allocated to the pool.

All [instance level limits](#) apply to instances created within a pool.

In addition to instance-level limits there are also two limits imposed *at the instance pool level*:

- Total storage size per pool (8 TB).
- Total number of databases per pool (100).

Total storage allocation and number of databases across all instances must be lower or equal to the limits exposed by instance pools.

Instance pools support 8, 16, 24, 32, 40, 64, and 80 vCores.

Managed instances inside pools support 2, 4, 8, 16, 24, 32, 40, 64 and 80 vCores.

Managed instances inside pools support storage sizes between 32 GB and 8 TB, except:

- 2 vCore instances support sizes between 32 GB and 640 GB
- 4 vCore instances support sizes between 32 GB and 2 TB

The [service tier property](#) is associated with the instance pool resource so all instances in a pool must be the same service tier as the service tier of the pool. At this time, only the General Purpose service tier is available (see the following section on limitations in the current preview).

Azure SQL Managed Instance backups

Nearly the same as Azure SQL Database.

Backup schedule:

- A weekly full backup
- A differential backup every 12 hours
- A transaction log backup every 5-10 minutes based on log utilization

- **Retention:** 7-day default retention, that can extend to 35 days
- Backups are stored in geo-redundant storage accounts
- Ability to perform manual COPY_ONLY backups to Azure blob storage

© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Managed Instance-backuper fungerar **nästan likadant som för Azure SQL Database**, men med några viktiga skillnader. Managed Instance tar automatiskt **full backup varje vecka, differential backup var 12:e timme och transaktionsloggbackuper var 5–10 minut**, vilket möjliggör point-in-time restore. Standardretentionen är **7 dagar**, men kan utökas till **35 dagar**, och backuper lagras i **geo-redundant Azure-lagring**.

Skillnaden mot Azure SQL Database är att **Managed Instance även stödjer manuella COPY_ONLY-backuper till Azure Blob Storage**, vilket ger större flexibilitet för exempelvis migration, arkivering eller integration med befintliga backupflöden. Dessutom ligger Managed Instance närmare klassisk SQL Server i funktionalitet, medan Azure SQL Database är mer strikt plattformsstyrd.

Backups are performed automatically by the service.

You can restore your own backups from Azure blob storage.

Restore an existing database to a point-in-time in the past within the retention period using the Azure portal, Azure PowerShell, Azure CLI, or REST API. In Single database and Elastic pools, this operation will create a new database in the same server as the original database. In Managed Instance, this operation can create a copy of the database or same or different Managed Instance under the same subscription.

Restore a deleted database to the time it was deleted or anytime within the retention period. The deleted database can only be restored in the same logical server or Managed Instance where the original database was created.

Restore a database to another geographical region. Geo-restore allows you to recover from a geographic disaster when you cannot access your server and database. It creates a new database in any existing server anywhere in the world.

Restore a database from a specific long-term backup on Single Database or Elastic Pool if the database has been configured with a long-term retention policy (LTR). LTR allows you to restore an old version of the database using [the Azure portal](#) or [Azure PowerShell](#) to satisfy a compliance request or to run an old version of the application. For more information, see [Long-term retention](#).

Backups are specified and managed from the virtual SQL Server level and NOT at the database level.

Knowledge check



You need to migrate a set of databases that use distributed transactions from on-premises SQL Server. Which of the following options should you choose?

- ☒ Azure SQL Managed Instance
- ☐ Azure SQL Database
- ☐ Azure SQL Database Hyperscale



You are building a new cloud database that you expect to grow to 50 TB. Which is the best option for your database?

- ☐ Azure SQL Managed Instance
- ☐ Azure SQL Database
- ☒ Azure SQL Database Hyperscale



You are building a database for testing purposes that will be used less than 8 hours a day and is expected to be 20 GB in size. What is your most cost-effective option?

- ☒ Azure SQL Database Serverless
- ☐ Azure SQL Database Hyperscale
- ☐ Azure SQL Managed Instance

© Copyright Microsoft Corporation. All rights reserved.

Här är **rätta och felaktiga svar**, ca **10 ord per alternativ**:

1. Migrera databaser med distribuerade transaktioner

☒ Azure SQL Managed Instance (rätt)

Stöder MSDTC och SQL Server-funktioner nära on-prem.

☒ Azure SQL Database (fel)

Stöder inte distribuerade transaktioner eller cross-database-queries.

☒ Azure SQL Database Hyperscale (fel)

Fokuserar på skalbarhet, inte distribuerade transaktioner.

2. Ny databas som förväntas växa till 50 TB

☒ Azure SQL Managed Instance (fel)

Begränsad skalning, inte optimal för extremt stora databaser.

☒ Azure SQL Database (fel)

Maxstorlek räcker inte för 50 TB.

☒ Azure SQL Database Hyperscale (rätt)

Designad för mycket stora databaser med nästan obegränsad storlek.

3. Testdatabas <8 timmar/dag, ca 20 GB

☒ Azure SQL Database Serverless (rätt)

Auto-pausar och debiterar bara vid användning.

☒ Azure SQL Database Hyperscale (fel)

Onödigt kraftfull och dyr för små testdatabaser.

✖ Azure SQL Managed Instance (fel)

För hög grundkostnad för tillfälliga och små arbetslaster.

Instructor led labs: Provision an Azure SQL Database

Create a Virtual Network

Provision an Azure SQL Database in the Azure portal

Enable access to an Azure SQL Database

Connect to an Azure SQL Database in SQL Server Management Studio

Query an Azure SQL Database with SQL Server Management Studio

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/02-provision-sql-database.html>

Migrate SQL Server workloads to Azure SQL Database

<https://learn.microsoft.com/training/modules/migrate-sql-workloads-azure-sql-databases/>

© Copyright Microsoft Corporation. All rights reserved.

IMPORTANT: Share with the learners specific migration hands-on labs available on <https://microsoftlearning.github.io/mslearn-sql-migration/> This is an **optional** and additional material so they can try at their own pace and their own subscription.

Objectives



Explore options for migrating to Azure SQL Database



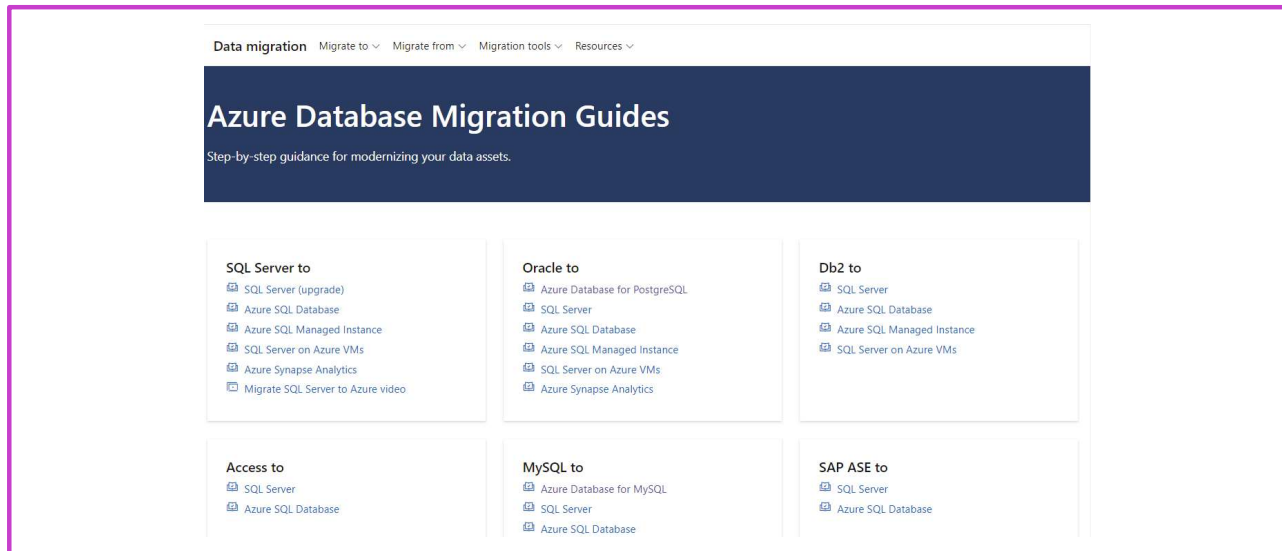
Learn how to migrate to an Azure SQL Database



Understand service continuity to Azure SQL Database

© Copyright Microsoft Corporation. All rights reserved.

Exploring Azure database migration guides



© Copyright Microsoft Corporation. All rights reserved.

Azure Database Migration Guides är Microsofts samlade, strukturerade vägledning för att hjälpa organisationer att flytta databaser till Azure på ett kontrollerat och säkert sätt. Guiderna är uppbyggda per källsystem (t.ex. SQL Server, Oracle, DB2, MySQL, SAP ASE) och per målplattform i Azure, som Azure SQL Database, Azure SQL Managed Instance, SQL Server på Azure VM eller Azure Synapse Analytics.

Varje guide följer en tydlig metodik: **Assess** → **Migrate** → **Optimize**. Först analyseras befintliga databaser med verktyg som Azure Data Migration Assistant för att identifiera kompatibilitetsproblem och funktionsskillnader. Därefter ges konkreta steg-för-steg-instruktioner för själva migreringen, inklusive rekommenderade verktyg som Azure Database Migration Service. Slutligen får man råd om hur lösningen optimeras efter flytten, exempelvis genom rätt tjänstnivå, prestandajustering och kostnadsoptimering.

Guiderna hjälper också till att välja rätt Azure-databasplattform baserat på krav som skalbarhet, tillgänglighet, funktionalitet och driftsmodell. De är därför ett viktigt stöd både för tekniska specialister och arkitekter vid modernisering av databasmiljöer.

The Azure Database Migration Guides (<https://learn.microsoft.com/en-us/data-migration/>) website provides the latest information about migrations between different systems. You should run through the recommendation wizard that appears near the top of the website to see the migration scenario and the migration learning and best practices that are returned after this course.

Benefits of migrating to Azure SQL Database



© Copyright Microsoft Corporation. All rights reserved.

1.Backup and recovery: Azure SQL Database provides built-in backup and restore capabilities, reducing operational overhead compared to manual backup processes. You can set a long-term backup retention (LTR) policy to retain backups for up to 10 years

2.High availability: Azure SQL Database offers high availability by service tier, including general purpose, business critical, and hyperscale. It ensures resiliency and high availability with Azure availability zones, supported by a 99.99% uptime SLA for virtual machines.

3.Disaster recovery: Azure SQL Database provides automated backup for all databases. Geo-restore and Geo-replication ensure that your data is always available, even in extreme cases like natural disasters.

4.Service scalability: Azure SQL Database offers flexible options for application migration, modernization, and development, allowing you to scale your database workloads efficiently.

5.Security: Azure's built-in security features help protect against infrastructure disruptions and ensure compliance with regulatory needs for critical applications.

6.Licensing: When you migrate SQL Server to Azure SQL Database, you can take advantage of the Azure Hybrid Benefit. This benefit allows you to use your existing SQL Server licenses with Software Assurance or qualifying subscription licenses to pay a reduced rate on Azure services.

Migration options supported

Azure Database Migration Service (recommended)	Offline
Azure Migrate	Offline
Transactional replication	Online
Import Export Wizard/BACPAC	Offline
Bulk copy (bcp utility)	Offline
Azure Data Factory	Offline

© Copyright Microsoft Corporation. All rights reserved.

There are two modes of migration to Azure SQL Database: **Online** and **Offline**. The online mode has minimal or no downtime, while the offline mode experiences downtime during the migration process.

Reinforce that [Azure Database Migration Service](#) is the recommended approach **for large migrations and enhanced overall experience.**

Requirements for using Azure Database Migration Service



Ensure the TCP/IP protocol is enabled in SQL Server



Configure your Windows Firewall for database engine access



When using a firewall appliance in front of your source databases, add firewall rules



Create an Azure Virtual Network (VNet) for Azure Database Migration Service by using the Azure Resource Manager deployment model

© Copyright Microsoft Corporation. All rights reserved.

Create Database Migration Service

Home > Azure Database Migration Services >

Select migration scenario and Database Migration Service ...

Migration scenario

Tell us about your migration scenario, the source database type and target database type, and based on that we will recommend the best Database Migration Service.

Source server type * ⓘ
SQL Server

Target server type * ⓘ
Azure SQL Database

Database Migration Service

Database Migration Service * ⓘ
Database Migration Service

Migrate your databases using Azure Database Migration Service

Azure Database Migration Service is a fully-managed Azure service that helps you easily migrate various databases to their corresponding Azure data services. [Learn more](#) ⓘ

The following source to target migration scenarios are supported:

Source	Target
SQL Server	Azure SQL MI, Azure SQL VM, Azure SQL DB

Select

Create Data Migration Service ...

Basics Tags Review + create

Azure Database Migration Service is a fully managed service designed to streamline your migration process of SQL Server databases, whether running on-premises or in other cloud environments, to Azure SQL targets. [Learn more](#) ⓘ

Select the subscription to manage deployed resources and costs. Use resource groups as you would use folders, to organize and manage all of your resources.

Subscription * ⓘ WWL-JuPadrao

Resource group * ⓘ
Select existing
[Create new](#)

Instance details

Location * ⓘ East US

Migration service name * ⓘ Enter the service name

Review + create Next : Tags >>

© Copyright Microsoft Corporation. All rights reserved.

Så här **startar du Azure Database Migration Service (DMS)** och kommer fram till bilden du visar, steg för steg i **Azure Portal**:

Logga in i Azure Portal

Gå till <https://portal.azure.com> och logga in.

Sök efter tjänsten

Klicka i sökrutan högst upp och skriv
"Azure Database Migration Services"

Välj tjänsten i listan.

Skapa ny instans

Klicka på **+ Create** (eller *Create database migration service*).

Välj migreringsscenario

Du hamnar nu på sidan *Select migration scenario and Database Migration Service*:

- **Source server type**: t.ex. *SQL Server*
- **Target server type**: t.ex. *Azure SQL Database*, *Azure SQL Managed Instance* eller *SQL Server on Azure VM*
- **Database Migration Service**: lämnas som *Database Migration Service*
Klicka **Select** → detta motsvarar vänstra bilden.

Create Data Migration Service – Basics

Nu visas sidan *Create Data Migration Service*:

- **Subscription**: välj rätt subscription
- **Resource group**: välj befintlig eller skapa ny
- **Location**: välj region (helst samma som målservern)

- **Migration service name:** ange ett unikt namn

Slutför skapandet

Klicka **Review + create** → **Create**.

Efter några minuter är DMS skapad. Därefter kan du:

Skapa ett **migration project**

Ange käll- och målserver

Välja **online** eller **offline migration**

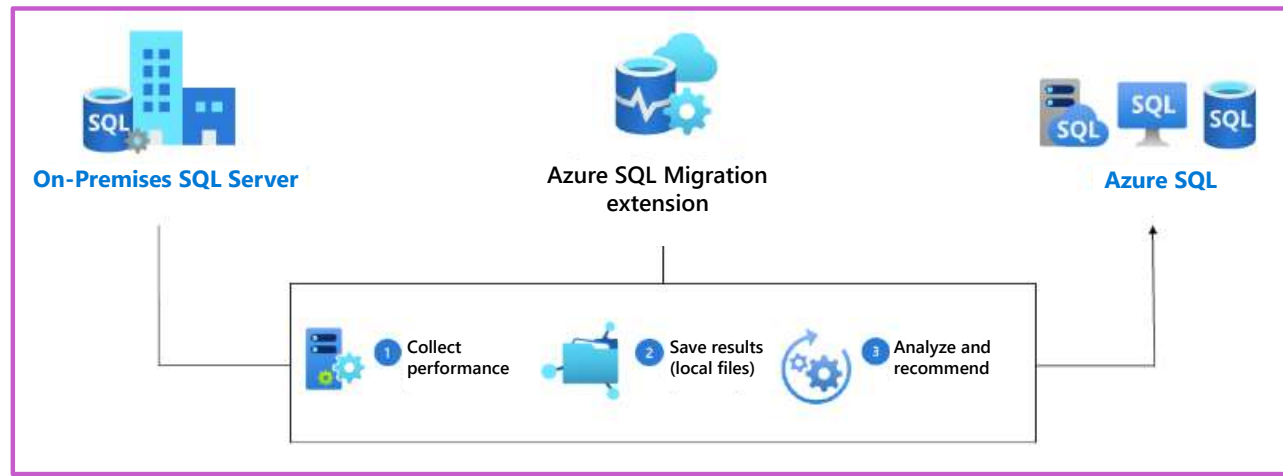
Starta själva databasmigreringen

👉 Kort sagt:

Azure Portal → **Azure Database Migration Services** → **Create** → **välj scenario** → **Create Data Migration Service**.

This is a build slide that shows how to create the Azure Database Migration Service.

Azure SQL Migration extension process



© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Migration extension används i **planeringsfasen**. Den analyserar din befintliga SQL Server: prestanda, belastning och kompatibilitet, och ger **rekommendationer** om **vart** och **hur** du bör migrera (t.ex. Azure SQL Database vs Managed Instance, storlek, kostnad).

Azure Database Migration Service (DMS) används i **genomförandefasen**. Den gör själva **flytten av data** – schema och/eller data – från källan till Azure, antingen offline eller med minimal nedtid.

Enkelt uttryckt

Migration extension = "Vad ska jag flytta till, och hur stort?"

Database Migration Service = "Flytta datat nu."

I praktiken:

Kör **Migration extension** → få rätt mål och dimensionering

Använd **DMS** → genomför själva migreringen tryggt och kontrollerat

Poängen med **Azure SQL Migration extension** är att göra det **enkelt och säkert att planera en SQL-migrering till Azure** utan att gissa. Tillägget installeras på din **on-premises SQL Server** och samlar in verklig prestandadata, till exempel CPU-användning, minne, I/O och belastningsmönster under normal drift. Denna data sparas lokalt och analyseras sedan för att ge **konkreta rekommendationer** om vilken Azure-lösning som passar bäst: Azure SQL Database, Managed Instance eller SQL Server på VM – samt rätt storlek. Resultatet blir bättre dimensionering, lägre kostnad och mindre risk för prestandaproblem efter flytten. Kort sagt: den hjälper dig att **flytta rätt, inte bara flytta snabbt**.

IMPORTANT: Emphasize that Azure SQL Migration extension is a reliable Azure service powered by **Azure Database Migration Service** that orchestrates data movement activities to deliver a seamless migration experience.

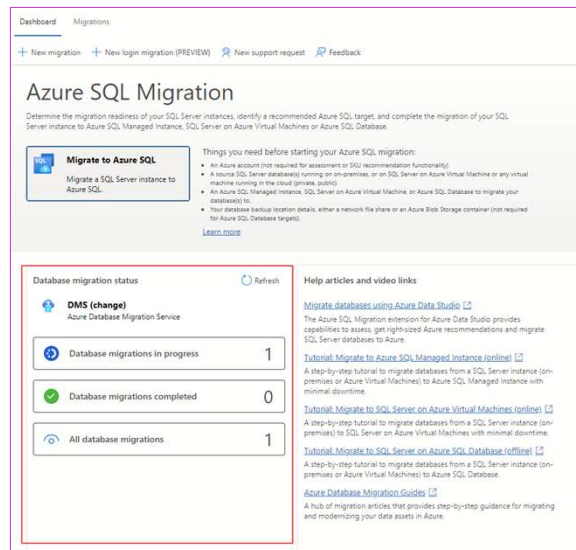
The [Azure SQL Migration extension for Azure Data Studio](#) helps in assessing your migration readiness, providing appropriate SKU recommendations for Azure resources, and facilitating the migration of your SQL Server database to Azure and is ideal for small to medium-sized databases. It's powered by the latest version of Data Migration Services, and also provides an advanced assessment feature that evaluates SQL Server databases that are ready for migration to Azure SQL.

Assessment features:

- A responsive UI for an end-to-end migration experience. The extension starts with a migration **readiness assessment** and **SKU recommendation** (based on performance data).
- Use the Azure SQL Migration extension to assess SQL Server databases running on Windows or Linux
- Provides an SKU recommendation engine that collects performance data from the on-premises source SQL Server instance and then generates right-sized SKU recommendations based on your Azure SQL target.
- Azure account is not required for assessments or SKU recommendations.

Resource: <https://learn.microsoft.com/en-us/training/modules/assess-sql-server-databases-for-migration-to-azure-sql/4-assess-database-azure-migration-extension>

Azure SQL Migration extension



© Copyright Microsoft Corporation. All rights reserved.

För att få fram **den här vyn i Azure SQL Migration extension** (som på bilden) gör du så här, steg för steg:

Förutsättningar

Du har **Azure Data Studio** installerat

Du har åtkomst till ett **Azure-konto**

Du har en **SQL Server-instans** (on-prem eller i VM) att migrera

Steg-för-steg

Öppna Azure Data Studio

Gå till **Extensions** (fyrekantsikonen till vänster)

Sök efter **"Azure SQL Migration"**

Installera tillägget **Azure SQL Migration**

Starta om Azure Data Studio om du blir ombedd

Öppna exakt denna vy

I vänstermenyn: klicka på **Azure-ikonen**

Logga in på ditt **Azure-konto**

Expandera **Azure SQL Migration**

Klicka på **Azure SQL Migration → Dashboard**

Nu hamnar du på **Dashboard-vyn** som på bilden, där du ser:

Koppling till **Database Migration Service (DMS)**

Antal migreringar (pågående/klara)

Länkar till guider och tutorials

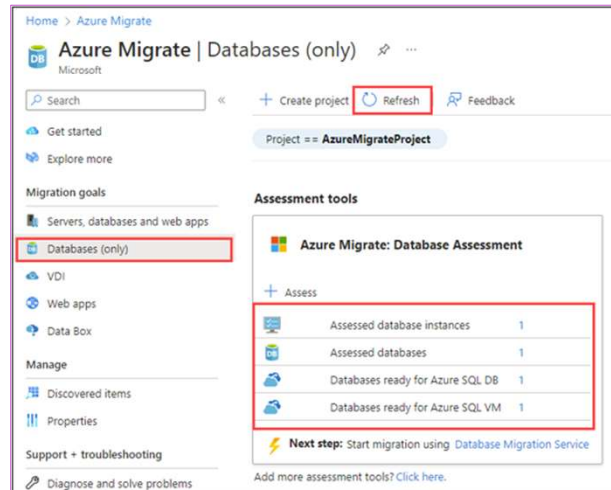
Knappen **Migrate to Azure SQL**

Kort sammanfattning**Plats:** Azure Data Studio**Tillägg:** Azure SQL Migration**Vy:** Azure SQL Migration → Dashboard

Vill du kan jag även beskriva **hur du kopplar DMS härifrån** eller **startar en ny migrering steg för steg**.

What is Azure Migrate?

Azure Migrate provides a dashboard for a suite of migration tools.



© Copyright Microsoft Corporation. All rights reserved.

Den här bilden visar **Azure Migrate**, som är **navet** som knyter ihop alla de senaste bilderna du har sett.

Azure Migrate är den övergripande **styr- och översiktsplattformen** för migrering till Azure. Här samlar du inventering, bedömning och rekommendationer för dina databaser. Med hjälp av **Azure Migrate: Database Assessment** analyseras dina on-prem SQL Server-instanser och databaser: kompatibilitet, storlek, prestanda och vilket Azure-mål som passar bäst (Azure SQL Database, SQL Managed Instance eller SQL på VM).

När bedömningen är klar pekar Azure Migrate vidare till **nästa steg: faktisk migrering**. Det är här de andra bilderna kommer in:

Azure SQL Migration extension används lokalt för att samla data och starta migreringen.

Database Migration Service (DMS) är motorn som utför själva flytten av databaserna till Azure.

Status och framdrift visas i SQL Migration-vyerna du sett tidigare.

Kort sagt:

Azure Migrate = planera och besluta.

Migration extension + DMS = genomföra migreringen.

Azure Migrate simplifies the process of migrating, modernizing, and optimizing your Azure environment by offering a wide range of comprehensive services. It covers all pre-migration steps, including discovery, assessments, and right-sizing of on-premises resources for infrastructure, data, and applications. Additionally, Azure Migrate allows integration with third-party tools, expanding its capabilities to support a wide range of use-cases.

Azure Migrate supports discovery and assessment of different SQL Server deployments, such as SQL Server Always On Failover Cluster Instances (FCI) and Always On Availability Groups (AG).

•Business case

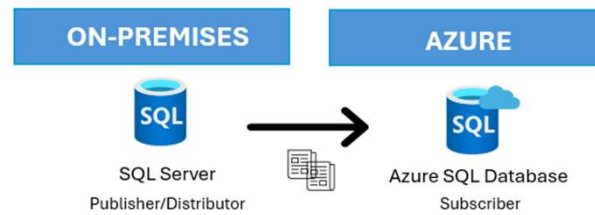
During the discovery phase, **SQL Server discovery and assessment** helps you to develop a comprehensive business case to assess the return on investment for migrating servers, SQL Server deployments, and ASP.NET web apps to Azure. You can also eliminate uncertainty and gain insights on Total Cost of Ownership (TCO), resource

utilization, and quick wins for migration and modernization.

Optimize costs

This assessment type uses agentless discovery, Azure readiness checks, and dependency analysis for efficient on-premises mapping and identification of resources ready for migration. It uses insights to estimate the cost of migrating your resources to Azure.

Online migration with transactional replication



Three steps to migrate using transactional replication:

Create a distribution database	Create a publication	Create a subscription
Performed on the source database that will be migrated to a SQL Server hosted on an Azure Virtual Machine	You can selectively choose the tables within a database that you wish to publish. It can be the whole database or specific tables and views	On the SQL Server on the Azure Virtual Machine, you create a subscription that will host the migrated data

© Copyright Microsoft Corporation. All rights reserved.

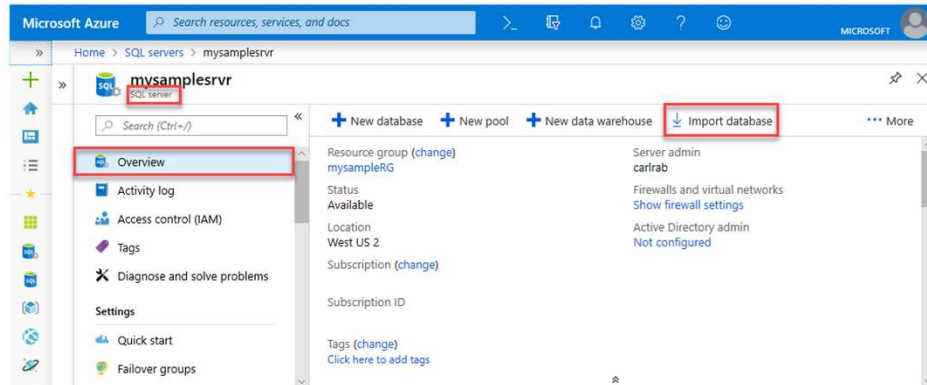
If you need a database to remain online to users throughout the migration process, you can use transactional replication to move the data. Transactional replication is the only online method available for migrating to Azure SQL Database.

The process begins with a snapshot of the publication

database objects and data. Once the initial snapshot is taken, any subsequent changes to the data or schema at the Publisher are typically delivered to Azure SQL Database in near

real-time as they
occur.

Migrate using the BACPAC

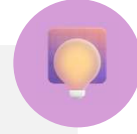


```
SqlPackage.exe /a:import /tcs:"Data Source=mynewserver20170403.database.windows.net;Initial
Catalog=myMigratedDatabase;User
Id=<your_server_admin_account_user_id>;Password=<your_server_admin_account_password>"
/sf:AdventureWorks2008R2.bacpac /p:DatabaseEdition=Premium /p:DatabaseServiceObjective=P6
```

© Copyright Microsoft Corporation. All rights reserved.

Explain the BACPAC command in the slide

Knowledge check



True or false: Azure Database Migration Service supports online migrations for Azure SQL Database.

☐ True

☒ False



Which purchasing option allows you to use your SQL Server Enterprise Cores licenses to pay for Azure SQL Database?

☐ Both vCore and DTU purchasing models

☒ vCore purchasing model

☐ DTU purchasing model



When should transactional replication be considered during migration?

☒ When you need to minimize downtime

☐ When you need to minimize compatibility errors

☐ When your source is Azure SQL Database

© Copyright Microsoft Corporation. All rights reserved.

Azure Database Migration Service supports online migrations for Azure SQL Database

✗ False: Online-migrering stöds inte till Azure SQL Database med DMS.

Which purchasing option allows use of SQL Server Enterprise Core licenses?

✓ vCore purchasing model: Endast vCore-modellen stöder Azure Hybrid Benefit-licenser.

✗ DTU purchasing model: DTU-modellen kan inte använda egna SQL-licenser.

✗ Both vCore and DTU: Hybrid Benefit gäller inte för båda modellerna.

When should transactional replication be considered during migration?

✓ Minimize downtime: Replikering möjliggör nästan noll stillestånd vid migrering.

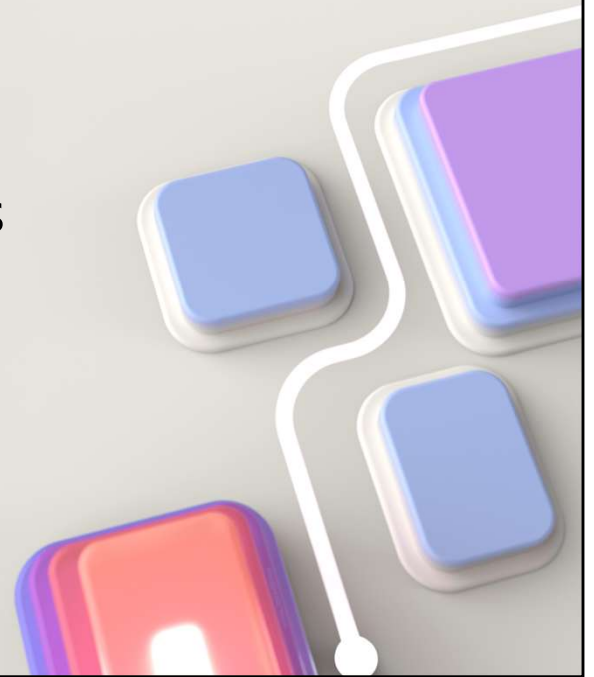
✗ Minimize compatibility errors: Replikering löser inte funktions- eller syntaxproblem.

✗ Source is Azure SQL Database: Transaktionell replikering används främst från SQL Server.

Migrate SQL Server workloads to Azure SQL Managed Instance

<https://learn.microsoft.com/training/modules/migrate-sql-workloads-azure-managed-instances/>

© Copyright Microsoft Corporation. All rights reserved.



Objectives



Explore options for migrating to Azure SQL Managed Instance



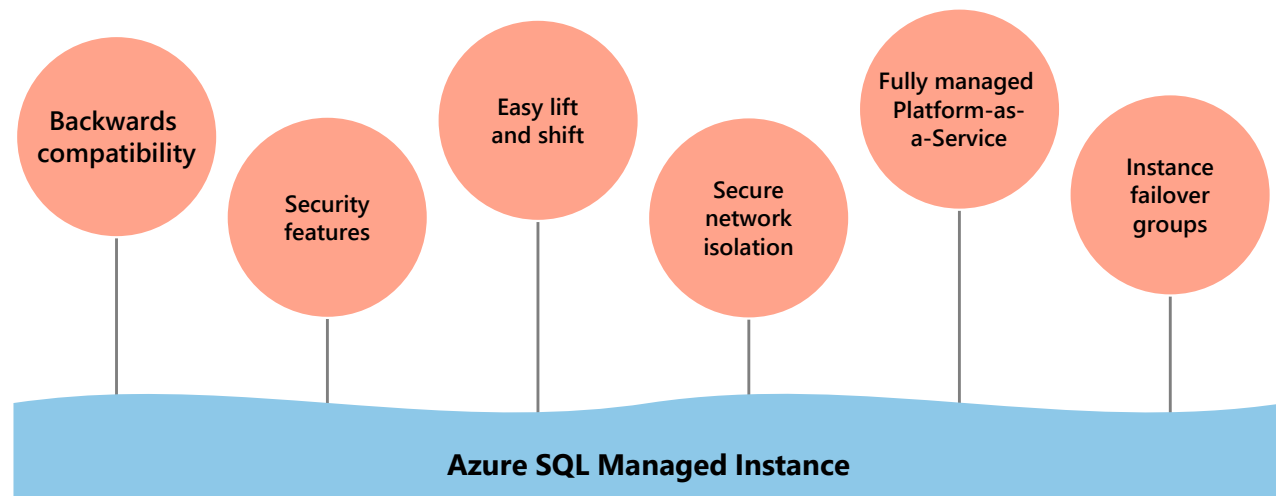
Synchronize data to Azure SQL Managed Instance



Considerations of migrating to Azure SQL Managed Instance

© Copyright Microsoft Corporation. All rights reserved.

Benefits of migrating to Azure SQL Managed Instance



© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/en-us/training/modules/migrate-sql-workloads-azure-managed-instances/2-evaluate-migration-scenarios>

Migration options supported

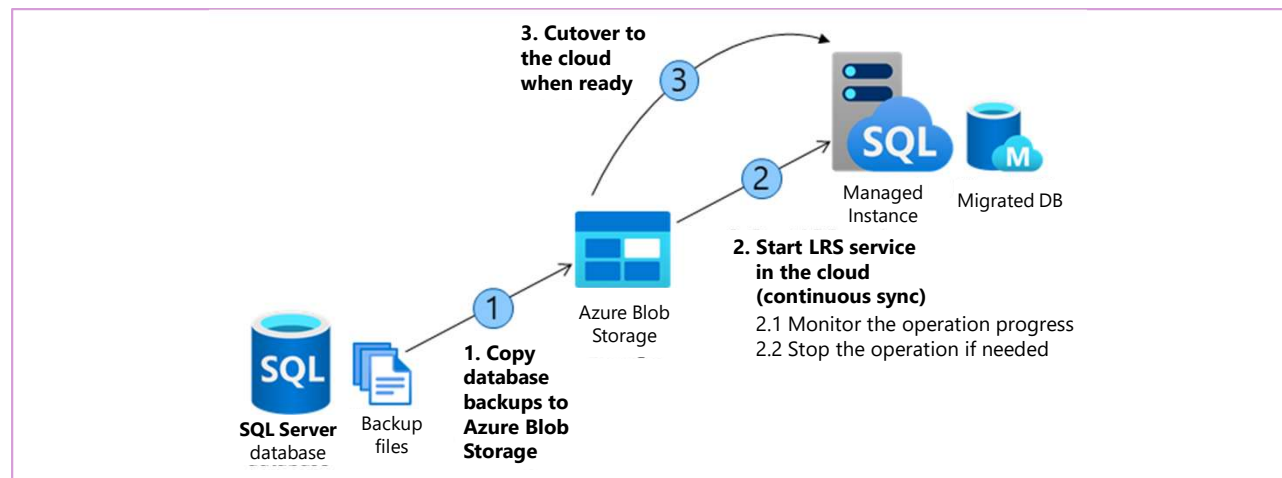
Log Replay Service	Online
Azure Database Migration Service	Online
Azure Migrate	Offline
Managed Instance link	Online
Transactional replication	Online
Native backup	Offline

© Copyright Microsoft Corporation. All rights reserved.

IMPORTANT: Although most of the tools facilitate [migration to Azure SQL Database](#) as well, there are some features that are exclusively supported by SQL Managed Instance. These are the ones we'll focus more on the next slides.

Reinforce that **Log Shipping** is natively built in as a part of [Azure Data Migration Service \(DMS\)](#), and [Log Replay Service \(LRS\)](#).

Log Replay Service (LRS)



© Copyright Microsoft Corporation. All rights reserved.

Log Replay Service (LRS) är en migreringsmetod som används för att flytta databaser från **SQL Server on-premises** till **Azure SQL Managed Instance** med mycket låg nedtid. Grunden är att utnyttja vanliga SQL Server-backuper och transaktionsloggar, i stället för direktkoppling mellan källan och Azure.

Processen sker i tre steg. Först tas en full backup (och eventuella diff- och logbackuper) från källdatabasen och laddas upp till **Azure Blob Storage**. Därefter startas LRS på Managed Instance, som kontinuerligt läser och återspelar ("replay") nya transaktionsloggar från bloblagringen. Under denna fas fortsätter applikationen att köra mot den lokala databasen, samtidigt som Azure hålls nästan helt synkroniserad. Slutligen, när man är redo, stoppas applikationen kort, sista loggarna spelas upp och man gör **cutover** till Azure.

LRS passar bra när man vill ha **kontrollerad migrering**, minimal påverkan på produktion och möjlighet att pausa eller övervaka processen. Den är särskilt användbar för större databaser och Managed Instance-migreringar.

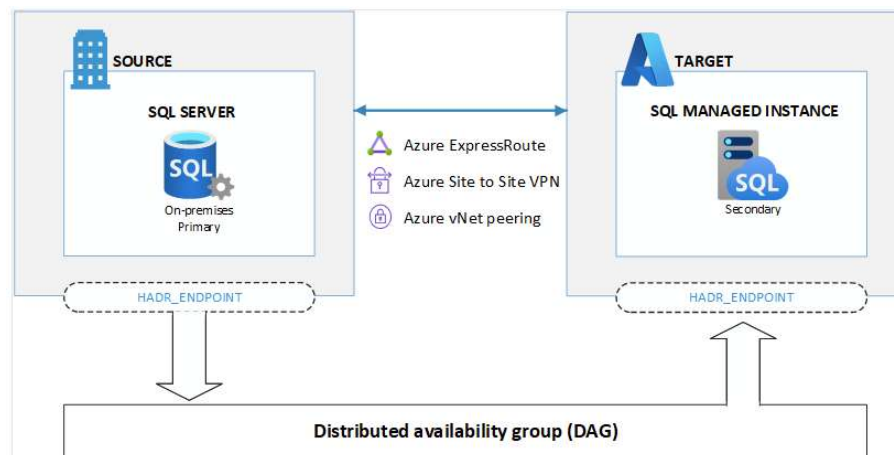
Log Replay Service (LRS) is a tool that enables custom migrations of databases from on-premises SQL Servers to SQL Managed Instance in the cloud. It uses log shipping technology and is useful in cases where more control is needed, when there's little tolerance for downtime, or when Azure Data Migration Service can't be used.

LRS can be used directly with PowerShell, CLI cmdlets, or API, to manually build and orchestrate database migrations to SQL Managed Instance. Some of the reasons to consider using LRS include:

- More control over the database migration project
- Little tolerance for downtime on migration cutover

- Inability to install DMS executable in the environment
- Lack of file access to database backups
- Inability to open networking ports from the environment to Azure

Azure SQL Managed Instance link feature



© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Managed Instance link feature används för att skapa en **hybridlösning** mellan en lokal SQL Server och Azure SQL Managed Instance. Funktionen bygger på **Always On availability groups** och mer specifikt en **Distributed Availability Group (DAG)**, där den lokala SQL Servern är primär och Managed Instance fungerar som sekundär replika i Azure.

I praktiken innebär det att transaktioner som skrivs lokalt replikeras nästan i realtid till Azure över ExpressRoute, Site-to-Site VPN eller VNet-peering. Azure-instansen kan användas för **read-only-arbetslast**, rapportering eller tester utan att belasta den lokala miljön. När man är redo kan man göra en kontrollerad **failover** och låta Azure bli primär, vilket ger mycket låg nedtid.

Managed Instance link passar bra för **stegvis migrering**, hybriddrift och disaster recovery-scenarier. Till skillnad från LRS och DMS är detta inte en engångsmigrering, utan en **kontinuerlig synkronisering** mellan on-prem och Azure. Det gör den idealisk när man vill flytta till molnet utan "big bang"-byten.

Azure SQL Managed Instance link feature offers a true online migration option compared to all other services and tools available. In addition, by partially running workloads on Azure, organizations can adopt a phased approach to cloud migration while still taking advantage of the benefits that Azure provides.

While the link is designed to replicate one database per link, it can be configured to replicate multiple databases from a single SQL Server instance to one or more SQL managed instances, or even replicate the same database to multiple SQL managed instances. This is achieved by setting up multiple links, each corresponding to a database-managed instance pair.

Azure SQL Managed Instance link feature allows you to replicate your SQL Server databases hosted anywhere to Azure, and fail over to the cloud in the event of a disaster or major business disruption. Azure SQL Managed Instance link also ensures seamless failover between the primary and secondary databases.

The link feature currently offers the following functionality:

- **One-way replication (SQL Server versions 2016 and 2019):** Use the link feature to replicate data one way from a SQL Server instance to your managed instance. Although you can manually fail over to your managed instance if there's a disaster, doing so breaks the link, and failing back isn't supported.
- **Disaster recovery (SQL Server 2022):** Use the link feature to replicate data from a SQL Server 2022 instance to your managed instance, manually fail over to your managed instance during a disaster, and fail back to the SQL Server instance after you've mitigated the disaster.

Backup and restore



```

BACKUP DATABASE [TargetDatabaseName] TO URL =
'https://<mystorageaccountname>.blob.core.windows.net/<mycontainername>/TargetDatabaseName.bak'

RESTORE DATABASE [TargetDatabaseName] FROM URL =
'https://<mystorageaccountname>.blob.core.windows.net/<mycontainername>/TargetDatabaseName.bak'
  
```

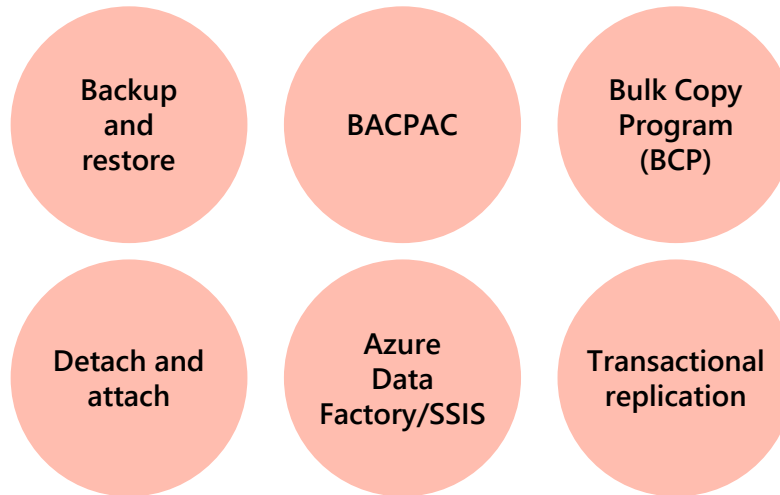
© Copyright Microsoft Corporation. All rights reserved.

One of the key capabilities of Azure SQL Managed Instance to enable quick and easy database migration is the native restore of database backup (.bak) files stored on [Azure Storage](#). Backing up and restoring are asynchronous operations based on the size of your database.

This method is also supported by SQL Server running on Azure VM.

1. Backup the database to an URL that points to an Azure storage account
2. Create a credential that accesses the storage account from SQL Managed Instance using Shared Access Signature (SAS)
3. Run the RESTORE FILELISTONLY FROM URL command to identify which backup to restore (optional)
4. Use the RESTORE DATABASE command to restore the database onto the managed instance
5. You can run SQL commands to check for the status of the restore operation to complete (optional)

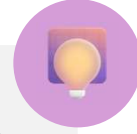
Other options to move data to and from SQL Managed Instance



© Copyright Microsoft Corporation. All rights reserved.

Similar to Azure SQL Database, these tools can also be used with Azure SQL Managed Instance.

Knowledge check



Can transactional replication be configured from the Azure portal?

- ☐ No, transactional replication can only be configured through SQL Server Management Studio.
- ☐ Yes.
- ☒ No.



What is the primary use case for the Log Replay Service for Azure SQL Managed Instance?

- ☒ To migrate databases from on-premises SQL Servers or virtual machines to Azure SQL Managed Instance with minimal downtime.
- ☐ To migrate data from other cloud providers to Azure SQL Managed Instance.
- ☐ To migrate data from Azure SQL Database to Azure SQL Managed Instance.



What is the purpose of using SQL Managed Instance link feature during migration?

- ☐ To create an availability group listener on the source and target.
- ☒ To minimize downtime during migration by extending an on-premises database into the cloud.
- ☐ To create a Windows Server Failover Cluster on the source and target.

© Copyright Microsoft Corporation. All rights reserved.

Fråga 1:

Can transactional replication be configured from the Azure portal?

✗ No, transactional replication can only be configured through SQL Server Management Studio.

Fel – kan även konfigureras via T-SQL, inte endast SSMS.

✗ Yes.

Fel – Azure-portalen har inget stöd för att konfigurera replikeringen.

✓ No.

Rätt – transaktionell replikering konfigureras inte via Azure-portalen.

Fråga 2:

What is the primary use case for the Log Replay Service for Azure SQL Managed Instance?

✓ To migrate databases from on-premises SQL Servers or virtual machines to Azure SQL Managed Instance with minimal downtime.

Rätt – LRS synkar loggar kontinuerligt för nästan noll nedtid.

✗ To migrate data from other cloud providers to Azure SQL Managed Instance.

Fel – LRS är inte ett generellt moln-till-moln-verktyg.

✗ To migrate data from Azure SQL Database to Azure SQL Managed Instance.

Fel – LRS använder backup- och loggfiler från SQL Server.

Fråga 3:

What is the purpose of using SQL Managed Instance link feature during migration?

✗ To create an availability group listener on the source and target.

Fel – lyssnare är inte huvudsyftet med MI link.

✓ To minimize downtime during migration by extending an on-premises database into the cloud.

Rätt – kontinuerlig replikering möjliggör kontrollerad failover.

✗ To create a Windows Server Failover Cluster on the source and target.

Fel – WSFC krävs inte mellan on-prem och Managed Instance.

Förklaring (ca 100 ord)

Azure Database Migration Service är den generella och enklaste vägen för databasmigreringar till Azure och passar de flesta standardfall. Log Replay Service är mer teknisk och passar när man vill ha full kontroll, mycket låg nertid och redan använder traditionella SQL-backuper. SQL Managed Instance link är den mest avancerade lösningen och används när man vill köra on-premises och Azure parallellt under en längre tid, exempelvis för tester eller stegvis övergång. Skillnaden ligger främst i **synkmetod, komplexitet och hur "hybrid" lösningen är** – från DMS (enkel), via LRS (kontrollerad), till MI link (realtid och hybrid).

References

What is Azure SQL Managed Instance?

<https://learn.microsoft.com/azure/sql-database/sql-database-managed-instance>

What is Azure SQL Database?

<https://learn.microsoft.com/azure/sql-database/sql-database-technical-overview>

Azure database migration guides

<https://learn.microsoft.com/data-migration/>

© Copyright Microsoft Corporation. All rights reserved.

Learning Path Recap

In this learning path, we:

Learned how to set up and configure resources manually, which can provide a greater understanding of the underlying processes and systems.

We also learned how to recommend an appropriate database offering based on requirements.

Lastly, we explored the tools and features available to move the database to Azure SQL Database and Azure SQL Managed Instance.

© Copyright Microsoft Corporation. All rights reserved.



© Copyright Microsoft Corporation. All rights reserved.